Підсумковий звіт
з лабораторних робіт 1-6

Виконавець: Базилевич Олексій
Група: К-27
Викладач: Андрій Ляшко

2026

Зміст

Лабораторна робота № 1 3
Звіт: LAB1_BazylevichAlex-1.pdf 4
Лабораторна робота № 2 50
Умова 51
Звіт: LAB2 part1 Оцінка часової складності (робочий зошит)-1.pdf 53
Звіт: LAB2 part2 Оброблення даних про файли (робочий зошит)-1.pdf 63
Звіт: LAB2 part3 Оброблення даних про успішність (робочий зошит)-1.pdf 86
Звіт: LAB2 part4 Оцінка параметрів функції (робочий зошит)-1.pdf 101
Лабораторна робота № 3 106
Умова 107
Звіт: Lab3_1.pdf 109
Звіт: Lab3_2.pdf 130
Лабораторна робота № 4 135
Умова 136
Скріншот: 16_plots.png 140
Звіт: Lab4_2-1.pdf 141
Звіт: Lab4_3-1.pdf 143
Звіт: Lab4_4-1.pdf 144
Звіт: Lab4_5-1.pdf 146
Скріншот: pie_chart.png 148
Скріншот: task2_complexity.png 149
Скріншот: task3_histogram.png 150
Скріншот: task4_cardioid.png 151
Скріншот: task5_hyperboloid.png 152
Лабораторна робота № 5 153
Скріншот: photo_1_2026-04-12_04-44-48.jpg 154
Скріншот: photo_2_2026-04-12_04-44-48.jpg 155
Скріншот: photo_3_2026-04-12_04-44-48.jpg 156
Скріншот: photo_4_2026-04-12_04-44-48.jpg 157
Скріншот: photo_5_2026-04-12_04-44-48.jpg 158
Скріншот: photo_6_2026-04-12_04-44-48.jpg 159
Скріншот: photo_7_2026-04-12_04-44-48.jpg 160
Скріншот: photo_8_2026-04-12_04-44-48.jpg 161
Код: Lab5_1.ipynb 162
Лабораторна робота № 6 165
Умова 166
Код: lab6.py 167

Лабораторна робота № 1
без варіантів

LAB1

Exported 2/15/2026, 10:55:12 PM

LAB 1 Елементи мови Python. Основні інструкції та структури даних. Робота з файловою системою

The document was converted with **Code-Format**: https://code-format.com/

Нехай протягом навчання програмні проєкти з різних предметів зберігалися в папках ... . Є перелік папок `folders`, в яких зберігаються ці проєкти (задає виконавець, виходячи з вмісту своїх носіїв даних).
Примітка. До цього переліку мають входити не окремі папки з проєктами, а папки, що містять, наприклад, всі проєкти такої-то дисципліни такого-то семестру тощо. Інакше кажучи, не папка одного solution, а папка, в якій зберігаються solutions (якщо в термінології Visual Studio).

Щоб профільтрувати допоміжні файли середовищ, введемо "білий список" розширень (`WHITE_LIST`) та список розширень файлів з текстом програм(задається) (`CODE_LIST`).

З'ясувати

1. Скільки різних розширень зустрічається у папках з заданого списку (тут і далі: включаючи вкладені), скільки файлів з кожним розширенням. (Для цієї задачі розглядаємо всі розширення, а не тільки "білого списку".)
   Відповідну інформацію вивести на екран охайною табличкою. На рядку виводиться розширення, кількість файлів та ознака (C,W,O відповідно для файлів з текстом програм (відповідно до `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

   Наприклад, `.txt 155 W .ipynb 73 W .cpp 45 C .exe 3 O`

2. За датами створення файлів для кожного місяця, починаючи з вересня 2024-го року, знайти таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний обсяг у байтах, кількість їх рядків (враховувати тільки для файлів категорії C; білі та порожні рядки не рахувати), середній розмір файлу, середня кількість рядків.
   Відповідну інформацію вивести на екран охайною табличкою в хронологічному порядку.

   Наприклад, `2024 09 23 123456 256 5368 11 2024 10 23 123456 256 5368 11 2024 11 134 123456 256 921 11`

3. Для кожного місяця та кожного розширення, що зустрічаються при перегляді файлів, розміщених у папках зі списку, знайти таку інформацію: рік, місяць, розширення, кількість файлів, сумарний обсяг у байтах, кількість їх рядків (тільки для розширень категорії C; білі та порожні рядки не рахувати; для інших розширень цю клітину таблиці залишаємо незаповненою)

   Цю інформацію записати в csv-файл з іменем `lab1_1.csv` (кодування тільки utf-8), на початку додати рядок з заголовками: year, mon, ext, cnt, size, lines, avg_size, avg_line.

Здається:

The document was converted with **Code-Format**: https://code-format.com/

1. Цей робочий зошит (заповнений, вилучати клітини, які були в ньому з самого початку, не дозволяється) та pdf-звіт.
   Перед здачею необхідно зробити Kernel -> Restart Kernel and Run All Cells... Для отримання pdf-звіта блокнот можна просто роздрукувати.
   Увага! Повідомлень про помилки виконання бути не повинно.
2. Файли зі скриптами для вирішення трьох задач (див. вище).
3. Файл `lab1_1.csv`

Оцінювання (максимум 15 балів):

Обов'язково має бути повністю та правильно виконаний (без повідомлень про помилки) та заповнений робочий зошит (блокнот), має бути pdf-звіт та `lab1_1.csv`, що відповідає виконаному блокноту.

За відсутності скриптів оцінка не може перевищувати 12 балів.

Правильність виконання оцінюється з точки зору дотримання вимог та отриманого кінцевого результату.

Якщо було вибрано папки майже без коду, то оцінка не може перевищувати 6 балів.

За неохайне виведення знімається 3 бали.

In [1]:

```
from importlib.resources import files

CODE_LIST = {'.cpp', '.hpp', '.c', '.h', '.py', '.pyw', '.cs', '.js',
'.java'}
WHITE_LIST = CODE_LIST | {'.html', '.json', '.txt', '.ipynb'}
```

In [2]:

```
import sys
print(sys.platform, sys.version, sep='\n')
```

```
win32
3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 64
bit (AMD64)]
```

Інструкції імпортування зазвичай записують на початку. Далі вони будуть додатково продубльовані по місцю використання.

The document was converted with **Code-Format**: https://code-format.com/

In [3]:

```
import datetime
from pathlib import Path
from datetime import datetime as dt
import csv
```

Вихідні відомості

In [4]:

```
language = 'python3'
```

Бібліотека `datetime` містить стандартні засоби для зображення дат, зокрема клас [datetime.date]

In [5]:

```
import datetime
year = datetime.date.today().year
```

In [6]:

```
year
```

```
2026
```

In [7]:

```
f'Ця робота використовує мову програмування {language} і була виконана
у {year} році.'
```

```
'Ця робота використовує мову програмування python3 і була виконана у
2026 році.'
```

In [8]:

```
author = "Bazylevich Alex"
group = "K-27"
lab_instructor = "Andrii Liashko"
```

The document was converted with **Code-Format**: https://code-format.com/

Запишіть f-рядок, значенням якого є текст (розташовується на двох рядках)
`Виконавець: ....,  група ...  Викладач:` Значення виконавця, його групи та викладача мають підставлятися зі змінних `author`, `group` та `lab_instructor`.

▸ Підказка.
Доцільно використати рядок у трикратних лапках.

In [9]:

```
info = f"""Виконавець: {author}, група {group},
Викладач: {lab_instructor}"""# тут має бути записаний питомий f-рядок
print(info)
```

```
Виконавець: Bazylevich Alex, група K-27,
Викладач: Andrii Liashko
```

Готуємось "гуляти" по папках (підготовка-дослідження)

Нам знадобиться клас `Path` з бібліотеки `pathlib`. Документація.

In [10]:

```
from pathlib import Path
```

Дізнаємось поточний робочий каталог. (**c**urrent **w**ork **d**ir)

In [11]:

```
Path().cwd()
```

```
WindowsPath('C:/Users/Alex/PycharmProjects/labs')
```

In [12]:

```
str(Path().cwd())
```

```
'C:\\Users\\Alex\\PycharmProjects\\labs'
```

The document was converted with **Code-Format**: https://code-format.com/

Шлях до якої-небудь папки з помірною кількістю файлів з різними розширеннями та складеними папками зробити значенням змінної `root`.
У кожного має бути свій шлях.

Наприклад, можна взяти папку з проєктом мови C++.

In [13]:

```
root = Path('/media/tania/Apacer PHD/gits/Courses/C++1/projects/
lab07')
root
```

```
WindowsPath('/media/tania/Apacer PHD/gits/Courses/C++1/projects/
lab07')
```

Обійти папку, вивести всі наявні в ній файли

Цікавить шлях, тип файлу (каталог чи звичайний файл), для звичайного файлу цікавить розширення та розмір у байтах.

Сімлінки та інше будемо пропускати.

▸ Підказка 1

У класі є метод `glob`, що здатен вирішити задачу. Див. [документацію](#) бібліотеки.

In [14]:

```
help(Path.glob)
```

```
Help on function glob in module pathlib:

glob(self, pattern, *, case_sensitive=None, recurse_symlinks=False)
    Iterate over this subtree and yield all existing files (of any
    kind, including directories) matching the given relative pattern.
```

The document was converted with **Code-Format**: https://code-format.com/

- Підказка 2

Для рекурсивного обходу каталогу можна використати шаблон `**/*`, за яким будуть обходитись усі елементи, крім самої папки, що обходиться (.).

- Підказка 3

Якщо `s` задає послідовність або те, по чому можна ітерувати, то обхід можна реалізувати за допомогою циклу `for`

```
for el in s: do something with s
```

Обійти всі наявні в `root` файли та вивести їх імена

The document was converted with **Code-Format**: https://code-format.com/

In [15]:

```
from pathlib import Path
root = Path("D:/mother")
for el in root.glob('**/*'):
    print(el)
```

The document was converted with **Code-Format**: https://code-format.com/

```
D:\mother\adobik
D:\mother\ishodniki
D:\mother\music
D:\mother\pics
D:\mother\adobik\Adobe Premiere Pro Audio Previews
D:\mother\adobik\Adobe Premiere Pro Auto-Save
D:\mother\adobik\ForMother.prin
D:\mother\adobik\ForMother.prproj
D:\mother\ishodniki\20250809_091518.mp4
D:\mother\ishodniki\20250809_115037.mp4
D:\mother\ishodniki\20250809_115126.mp4
D:\mother\ishodniki\20250809_115241.mp4
D:\mother\ishodniki\20250809_115312.mp4
D:\mother\ishodniki\20250809_165645.mp4
D:\mother\ishodniki\20250809_170449.mp4
D:\mother\ishodniki\20250809_204014.mp4
D:\mother\ishodniki\20250810_091816.mp4
D:\mother\ishodniki\20250811_111141.mp4
D:\mother\ishodniki\20250811_121637.mp4
D:\mother\ishodniki\20250811_121755.mp4
D:\mother\ishodniki\20250811_121923.mp4
D:\mother\ishodniki\20250811_122050.mp4
D:\mother\ishodniki\20250811_122217.mp4
D:\mother\ishodniki\20250811_122335.mp4
D:\mother\ishodniki\20250811_122357.mp4
D:\mother\ishodniki\20250811_123006.mp4
D:\mother\ishodniki\20250811_124338.mp4
D:\mother\ishodniki\20250811_124416.mp4
D:\mother\ishodniki\20250811_124603.mp4
D:\mother\music\soundcheck-314418.mp3
D:\mother\pics\kisspng-rixos-hotels-rixos-the-palm-dubai-business-
resort-finding-elite-5b34fd6e325162.5785607815301994062061-removebg-
preview.png
D:
\mother\pics\MV5BMWQzNWU5MmMtM2FjNC00MGMwLWI1ODAtZTAwZmEwOGU1OTA3XkEyX
kFqcGc@._V1_.jpg
D:\mother\adobik\Adobe Premiere Pro Audio Previews\ForMother.PRV
D:\mother\adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-
cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_17-42-24.prproj
D:\mother\adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-
cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_17-49-36.prproj
D:\mother\adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-
cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_17-55-36.prproj
D:\mother\adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-
cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_18-01-36.prproj
D:\mother\adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-
cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_18-07-36.prproj
```

The document was converted with **Code-Format**: https://code-format.com/

```
D:\mother\adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-
cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_18-13-36.prproj
D:\mother\adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-
cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_18-19-36.prproj
D:\mother\adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-
cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_18-25-36.prproj
D:\mother\adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-
cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_18-31-36.prproj
D:\mother\adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-
cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_18-45-19.prproj
D:\mother\adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-
cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_18-51-29.prproj
```

Є бажання виводити їх шляхи відносно заданої папки.

Тобто необхідно побудувати шлях, відносно шляху `root`. Допоможе метод `relative_to`.

In [16]:

```
help(Path.relative_to)
```

```
Help on function relative_to in module pathlib:

relative_to(self, other, *, walk_up=False)
    Return the relative path to another path identified by the passed
    arguments.  If the operation is not possible (because this is not
    related to the other path), raise ValueError.

    The *walk_up* parameter controls whether `..` may be used to
resolve
    the path.
```

Примітка. Насправді це метод класу `PurePath`, який є базовим для `Path`.

Після завершення циклу змінна `el`, яка позначала поточний елемент обходу, продовжує існувати. Проведемо з нею кілька експериментів.

Виведемо її значення.

The document was converted with **Code-Format**: https://code-format.com/

In [17]:

```
el
```

```
WindowsPath('D:/mother/adobik/Adobe Premiere Pro Auto-Save/ForMother--
f76663f3-cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_18-51-29.prproj')
```

У наступній клітині побудуємо та виведемо відповідний відносний шлях.

▶ Підказка

`el.relative_to(root)`

In [18]:

```
print(el.relative_to(root))
```

```
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-51-29.prproj
```

Явно перетворимо відносний шлях на рядок (значення типу `str`).

▶ Підказка

Щоб об'єкт `obj` перетворити на об'єкт типу `T`, можна використати конструктор типу `T`, а саме `T(obj)`.

▶ Відповідь

`str(el.relative_to(root))`

In [19]:

```
# заповнити
str(el.relative_to(root))
```

```
'adobik\\Adobe Premiere Pro Auto-Save\\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-51-29.prproj'
```

The document was converted with **Code-Format**: https://code-format.com/

Тепер у нашому обході будемо виводити шляхи, відносно шляху `root`. У наступну клітину слід записати модифікацію обходу, який здійснювався раніше.

Примітка. Якщо ми збираємось відносний шлях тільки виводити, то перед виведенням перетворювати його явно на рядок потреби немає. Це чудово зробить сама функція `print`.

▶ Підказка

`for el in` *`вираз, що задає обхід`*`: print(відносний шлях)`

The document was converted with **Code-Format**: https://code-format.com/

In [20]:

```
# заповнити
for el in root.glob('**/*'):
    print(el.relative_to(root))
```

The document was converted with **Code-Format**: https://code-format.com/

```
adobik
ishodniki
music
pics
adobik\Adobe Premiere Pro Audio Previews
adobik\Adobe Premiere Pro Auto-Save
adobik\ForMother.prin
adobik\ForMother.prproj
ishodniki\20250809_091518.mp4
ishodniki\20250809_115037.mp4
ishodniki\20250809_115126.mp4
ishodniki\20250809_115241.mp4
ishodniki\20250809_115312.mp4
ishodniki\20250809_165645.mp4
ishodniki\20250809_170449.mp4
ishodniki\20250809_204014.mp4
ishodniki\20250810_091816.mp4
ishodniki\20250811_111141.mp4
ishodniki\20250811_121637.mp4
ishodniki\20250811_121755.mp4
ishodniki\20250811_121923.mp4
ishodniki\20250811_122050.mp4
ishodniki\20250811_122217.mp4
ishodniki\20250811_122335.mp4
ishodniki\20250811_122357.mp4
ishodniki\20250811_123006.mp4
ishodniki\20250811_124338.mp4
ishodniki\20250811_124416.mp4
ishodniki\20250811_124603.mp4
music\soundcheck-314418.mp3
pics\kisspng-rixos-hotels-rixos-the-palm-dubai-business-resort-
finding-elite-5b34fd6e325162.5785607815301994062061-removebg-
preview.png
pics\MV5BMWQzNWU5MmMtM2FjNC00MGMwLWI1ODAtZTAwZmEwOGU1OTA3XkEyXkFqcGc@.
_V1_.jpg
adobik\Adobe Premiere Pro Audio Previews\ForMother.PRV
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_17-42-24.prproj
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_17-49-36.prproj
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_17-55-36.prproj
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-01-36.prproj
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-07-36.prproj
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
```

The document was converted with **Code-Format**: https://code-format.com/

```
c525-c8cdfb7f21c5-2025-08-15_18-13-36.prproj
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-19-36.prproj
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-25-36.prproj
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-31-36.prproj
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-45-19.prproj
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-51-29.prproj
```

Складові шляху

Інформацію про частини шляху можна знайти тут.

Альтернатива: використати довідку.

Знайдемо всі атрибути, доступні для класу `Path`. (У Python під атрибутами розуміють і класичні атрибути (описують дані об'єкта), і методи (це наслідок технічної реалізації методів).)

Функція `dir` поверне список.

The document was converted with **Code-Format**: https://code-format.com/

In [21]:

```
dir(Path)
```

The document was converted with **Code-Format**: https://code-format.com/

```
['__bytes__',
 '__class__',
 '__delattr__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__firstlineno__',
 '__format__',
 '__fspath__',
 '__ge__',
 '__getattribute__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__le__',
 '__lt__',
 '__module__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__rtruediv__',
 '__setattr__',
 '__sizeof__',
 '__slots__',
 '__static_attributes__',
 '__str__',
 '__subclasshook__',
 '__truediv__',
 '_copy_from',
 '_copy_from_file',
 '_copy_from_file_fallback',
 '_copy_from_symlink',
 '_delete',
 '_drv',
 '_filter_trailing_slash',
 '_format_parsed_parts',
 '_from_dir_entry',
 '_from_parsed_parts',
 '_from_parsed_string',
 '_hash',
 '_info',
 '_parse_path',
 '_parse_pattern',
```

The document was converted with **Code-Format**: https://code-format.com/

```
'_parts_normcase',
'_parts_normcase_cached',
'_raw_path',
'_raw_paths',
'_remove_leading_dot',
'_remove_trailing_slash',
'_root',
'_str',
'_str_normcase',
'_str_normcase_cached',
'_tail',
'_tail_cached',
'absolute',
'anchor',
'as_posix',
'as_uri',
'chmod',
'copy',
'copy_into',
'cwd',
'drive',
'exists',
'expanduser',
'from_uri',
'full_match',
'glob',
'group',
'hardlink_to',
'home',
'info',
'is_absolute',
'is_block_device',
'is_char_device',
'is_dir',
'is_fifo',
'is_file',
'is_junction',
'is_mount',
'is_relative_to',
'is_reserved',
'is_socket',
'is_symlink',
'iterdir',
'joinpath',
'lchmod',
'lstat',
'match',
'mkdir',
```

The document was converted with **Code-Format**: https://code-format.com/

```
 'move',
 'move_into',
 'name',
 'open',
 'owner',
 'parent',
 'parents',
 'parser',
 'parts',
 'read_bytes',
 'read_text',
 'readlink',
 'relative_to',
 'rename',
 'replace',
 'resolve',
 'rglob',
 'rmdir',
 'root',
 'samefile',
 'stat',
 'stem',
 'suffix',
 'suffixes',
 'symlink_to',
 'touch',
 'unlink',
 'walk',
 'with_name',
 'with_segments',
 'with_stem',
 'with_suffix',
 'write_bytes',
 'write_text']
```

Серед них ті, що починаються з двох підкреслень, є службовими. Ті, що починаються з одного підкреслення, — допоміжними (не є відкритими). Виведемо всі, крім них.

The document was converted with **Code-Format**: https://code-format.com/

In [22]:

```
cnt = 0
for a in dir(Path):
    if not a.startswith('_'):
        print(a, end=', ')
        cnt += 1
        if (cnt > 5):
            cnt = 0
            print()
```

```
absolute, anchor, as_posix, as_uri, chmod, copy,
copy_into, cwd, drive, exists, expanduser, from_uri,
full_match, glob, group, hardlink_to, home, info,
is_absolute, is_block_device, is_char_device, is_dir, is_fifo,
is_file,
is_junction, is_mount, is_relative_to, is_reserved, is_socket,
is_symlink,
iterdir, joinpath, lchmod, lstat, match, mkdir,
move, move_into, name, open, owner, parent,
parents, parser, parts, read_bytes, read_text, readlink,
relative_to, rename, replace, resolve, rglob, rmdir,
root, samefile, stat, stem, suffix, suffixes,
symlink_to, touch, unlink, walk, with_name, with_segments,
with_stem, with_suffix, write_bytes, write_text,
```

Повертаємось до нашого примірника шляху.

In [23]:

```
el
```

```
WindowsPath('D:/mother/adobik/Adobe Premiere Pro Auto-Save/ForMother--
f76663f3-cb36-a0d2-c525-c8cdfb7f21c5-2025-08-15_18-51-29.prproj')
```

Визначимо, чи задає цей шлях папку, файл, яке в нього ім'я, розширення, який його розмір.

▸ Підказка

Використати атрибути `is_dir`, `is_file`, `name`, `suffix`, `stat().st_size`

Записати вираз, що перевіряє, чи задає шлях `el` папку.

The document was converted with **Code-Format**: https://code-format.com/

In [24]:

```
# заповнити
print(el.is_dir())
```

```
False
```

Записати вираз, що перевіряє, чи задає шлях `el` файл.

In [25]:

```
# заповнити
print(el.is_file())
```

```
True
```

Записати вираз, що визначає ім'я файлу `el`.

In [26]:

```
print(el.name)
#заповнити
```

```
ForMother--f76663f3-cb36-a0d2-c525-
c8cdfb7f21c5-2025-08-15_18-51-29.prproj
```

Записати вираз, що визначає розширення файлу `el`.

In [27]:

```
# заповнити
print(el.suffix)
```

```
.prproj
```

Записати вираз, що визначає розмір файлу `el`.

The document was converted with **Code-Format**: https://code-format.com/

In [28]:

```
# заповнити
print(el.stat().st_size)
```

```
117145
```

Для розв'язання загальної задачі нам буде необхідно вміти визначати рік та місяць створення файла.

Не тільки розмір файлу, але й інша інформація про файл може бути знайдена за допомогою метода `stat` (документація)

Нас цікавить дата створення файла. Тут виникає невелика залежність від ОС. (документація)

Знайдемо час створення файлу (у секундах) і запам'ятаємо в змінній `ctime`.

In [29]:

```
if sys.platform.startswith('win'):
    ctime = el.stat().st_birthtime
else:
    ctime = el.stat().st_ctime
ctime
```

```
1755273089.0334153
```

Тепер слід перетворити на звичайну дату, щоб з'ясувати рік та місяць.

Використаємо клас `datetime`, точніше його метод `fromtimestamp`. (документація)

Нижче клас `datetime` імпортований з одноіменної бібліотеки `datetime` під псевдонімом `dt`.

In [30]:

```
from datetime import datetime as dt
```

The document was converted with **Code-Format**: https://code-format.com/

In [31]:

```
cdate = dt.fromtimestamp(ctime)
cdate
```

```
datetime.datetime(2025, 8, 15, 18, 51, 29, 33415)
```

In [32]:

```
cdate.year, cdate.month
```

```
(2025, 8)
```

Напишіть **кросплатформену** функцію `created_at`, що за шляхом до файлу визначає рік та місяць його створення. Функція має отримувати шлях до файлу та повертати пару цілих чисел.

Приклад функції, що повертає два значення. `def f(arg): return arg, arg+arg`

Питання для обговорення. В який момент краще розгалужити обчислення дати створення?

▶ Відповідь

`def _created_at_Win(...): ...`

`def _created_at_Linux(...): ...`

`if sys.platform.startswith('win'): created_at = _created_at_Win else: created_at = _created_at_Linux`

The document was converted with **Code-Format**: https://code-format.com/

In [33]:

```
def _created_at_Linux(path):
    stat = path.stat()
    d = dt.fromtimestamp(stat.st_ctime)
    return d.year, d.month

def _created_at_Windows(path):
    stat = path.stat()
    d = dt.fromtimestamp(stat.st_ctime)
    return d.year, d.month

if sys.platform.startswith('win'):
    created_at = _created_at_Windows
else:
    created_at = _created_at_Linux
```

Перевіримо працездатність функції.

In [34]:

```
created_at(el)
```

```
(2025, 8)
```

In [35]:

```
y, m = created_at(el)
print(y, m)
```

```
2025 8
```

Повертаємось до задачі обходу та виведення

Тепер для шляху `el` виведемо:
відповідний йому відносний шлях, далі на тому ж рядку `file` або `dir`
для файлу на новому рядку з абзацного відступу виведемо його розширення в 10 символів та розмір в 10 символів

The document was converted with **Code-Format**: https://code-format.com/

In [36]:

```
s = '.txt'
size = 12345
print(f'\t {s:10} {size:10}')
```

```
         .txt            12345
```

Записати код, що виводить інформацію про файл як у прикладі нижче

In [37]:

```
# заповнити

s = el.suffix
size = el.stat().st_size
type = "file" if el.is_file() else "dir"
print(f'\t {el.relative_to(root)} --- {type} {s:10} {size:10}')
```

```
         adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-
a0d2-c525-c8cdfb7f21c5-2025-08-15_18-51-29.prproj --- file .prproj
117145
```

Приклад формату виведення: `build/.cmake/api/v1/query/client-vscode/query.jsonfile --- file .json 144`

Зберемо обхід, за якого будемо виводити інформацію тільки про папки та звичайні файли.

У базовий обхід додаємо виведення інформації про файли.

The document was converted with **Code-Format**: https://code-format.com/

In [38]:

```
# заповнити
for el in root.glob('**/*'):
    s = el.suffix
    size = el.stat().st_size
    type = "file" if el.is_file() else "dir"
    print(f'{el.relative_to(root)} --- {type}\n {s:10} {size:10}')
```

The document was converted with **Code-Format**: https://code-format.com/

```
adobik --- dir
                  4096
ishodniki --- dir
                  4096
music --- dir
                     0
pics --- dir
                  4096
adobik\Adobe Premiere Pro Audio Previews --- dir
                     0
adobik\Adobe Premiere Pro Auto-Save --- dir
                  4096
adobik\ForMother.prin --- file
 .prin           87260
adobik\ForMother.prproj --- file
 .prproj        117145
ishodniki\20250809_091518.mp4 --- file
 .mp4         17529647
ishodniki\20250809_115037.mp4 --- file
 .mp4         16801819
ishodniki\20250809_115126.mp4 --- file
 .mp4         23116860
ishodniki\20250809_115241.mp4 --- file
 .mp4         11557065
ishodniki\20250809_115312.mp4 --- file
 .mp4         14598997
ishodniki\20250809_165645.mp4 --- file
 .mp4         28005339
ishodniki\20250809_170449.mp4 --- file
 .mp4         24920533
ishodniki\20250809_204014.mp4 --- file
 .mp4         11143206
ishodniki\20250810_091816.mp4 --- file
 .mp4         35624802
ishodniki\20250811_111141.mp4 --- file
 .mp4         73634827
ishodniki\20250811_121637.mp4 --- file
 .mp4         25906725
ishodniki\20250811_121755.mp4 --- file
 .mp4         14785825
ishodniki\20250811_121923.mp4 --- file
 .mp4         10444496
ishodniki\20250811_122050.mp4 --- file
 .mp4         14766755
ishodniki\20250811_122217.mp4 --- file
 .mp4         19874161
ishodniki\20250811_122335.mp4 --- file
```

The document was converted with **Code-Format**: https://code-format.com/

```
 .mp4         11447511
ishodniki\20250811_122357.mp4 --- file
 .mp4         18378385
ishodniki\20250811_123006.mp4 --- file
 .mp4         10688844
ishodniki\20250811_124338.mp4 --- file
 .mp4         20932397
ishodniki\20250811_124416.mp4 --- file
 .mp4         14512261
ishodniki\20250811_124603.mp4 --- file
 .mp4         14302768
music\soundcheck-314418.mp3 --- file
 .mp3          7322112
pics\kisspng-rixos-hotels-rixos-the-palm-dubai-business-resort-
finding-elite-5b34fd6e325162.5785607815301994062061-removebg-
preview.png --- file
 .png            57260
pics\MV5BMWQzNWU5MmMtM2FjNC00MGMwLWI1ODAtZTAwZmEwOGU1OTA3XkEyXkFqcGc@.
_V1_.jpg --- file
 .jpg            21719
adobik\Adobe Premiere Pro Audio Previews\ForMother.PRV --- dir
 .PRV                0
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_17-42-24.prproj --- file
 .prproj         29368
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_17-49-36.prproj --- file
 .prproj         30835
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_17-55-36.prproj --- file
 .prproj         30934
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-01-36.prproj --- file
 .prproj         31889
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-07-36.prproj --- file
 .prproj         32705
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-13-36.prproj --- file
 .prproj         33964
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-19-36.prproj --- file
 .prproj         35209
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-25-36.prproj --- file
 .prproj         36983
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-31-36.prproj --- file
```

This document was converted with Code-format https://code-format.com/

```
 .prproj         38177
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-45-19.prproj --- file
 .prproj         39466
adobik\Adobe Premiere Pro Auto-Save\ForMother--f76663f3-cb36-a0d2-
c525-c8cdfb7f21c5-2025-08-15_18-51-29.prproj --- file
 .prproj        117145
```

Визначимо, файли з якими розширеннями присутні в папці, що обходиться.

▸ Підказка 1 (про алгоритм).
Створимо порожню множину. У процесі обходу папки будемо додавати в неї знайдені розширення. Наприкінці виведемо знайдені розширення в алфавітному порядку.

▸ Підказка 2 (про деталі реалізації).

- Типом множин є `set` (документація).
- Щоб додати елемент до множини, можна використати метод `add` (документація).
- Відсортувати допоможе функція `sorted`(документація). Зверніть увагу на те, як задати сортування у спадному порядку.

Значенням змінної `suffixes` зробіть множину наявних розширень файлів.

In [47]:

```
# заповнити
suffixes = set()
for el in root.glob('**/*'):
    suffixes.add(el.suffix)
```

In [40]:

```
suffixes
```

```
{'', '.PRV', '.jpg', '.mp3', '.mp4', '.png', '.prin', '.prproj'}
```

Перетворіть її на відсортований список `suffixes_sorted`.

In [49]:

```
suffixes_sorted = sorted(suffixes)# заповнити
```

Виведіть знайдені розширення у відсортованому порядку.

The document was converted with **Code-Format**: https://code-format.com/

In [50]:

```
for el in suffixes_sorted:
    print(el, end=', ')
print()
```

```
, .PRV, .jpg, .mp3, .mp4, .png, .prin, .prproj,
```

Скільки різних розширень було знайдено? (Виведіть це значення. Вручну нічого не рахуємо!)

▸ Підказка

Функція `len` допоможе знайти кількість елементів довільного контейнера.

In [51]:

```
# заповнити
print(len(suffixes_sorted))
```

```
8
```

Знайдемо скільки файлів з певним розширенням наявно у досліджуваній папці.

Для кожного розширення визначимо, скільки файлів з цим розширенням присутні в папці, що обходиться.

▸ Підказка 1 (про алгоритм).

Створимо словник, ключами в якому будуть розширення файлів. Значеннями словника будуть лічильники кількості файлів з відповідним розширенням. У процесі обходу папки будемо збільшувати лічильники.

▸ Підказка 2 (про деталі реалізації).

- Типом словників є `dict` (документація).
- Наявність ключа в словнику можна перевірити за допомогою оператора `in`. Або у нагоді може стати метод `setdefault` .

Значенням змінної `d` зробіть словник, що знає кількість файлів для кожного наявного розширення.

The document was converted with **Code-Format**: https://code-format.com/

In [59]:

```
d = dict()
for el in root.glob('**/*'):
    if el.is_file():
        ext = el.suffix
        d.setdefault(ext, 0)
        d[ext] += 1
```

In [60]:

```
d
```

```
{'.prin': 1, '.prproj': 12, '.mp4': 21, '.mp3': 1, '.png': 1, '.jpg':
1}
```

In [62]:

```
suffixes_cnts = list(d.items())
for el in suffixes_cnts:
    print(el, end=', ')
print()
```

```
('.prin', 1), ('.prproj', 12), ('.mp4', 21), ('.mp3', 1), ('.png', 1),
('.jpg', 1),
```

Знайдемо найбільш популярні розширення.

▸ Підказки

1. Серед вбудованих функцій Python зверніть увагу на `max` (документація).
2. Анонімні функції теж будуть у нагоді.

Є список пар.

The document was converted with **Code-Format**: https://code-format.com/

In [63]:

```
lst = [(1, 2), (1, 5), (8, 2), (2, 4)]
lst
```

```
[(1, 2), (1, 5), (8, 2), (2, 4)]
```

Відсортуємо пари лексикографічно.

In [64]:

```
lst.sort()
lst
```

```
[(1, 2), (1, 5), (2, 4), (8, 2)]
```

Відсортуємо пари за ключем. В якості ключа візьмемо обернену пару.

In [65]:

```
lst.sort(key = lambda el: (el[1], el[0]))
lst
```

```
[(1, 2), (8, 2), (2, 4), (1, 5)]
```

Знайти найбільшу кількість файлів, що мають однакове розширення. Присвоїти значення змінній `max_count`.

▸ Підказка

Ключем має бути другий елемент пари.

In [66]:

```
max_count = max(d.items(), key=lambda x: x[1])
max_count
```

```
('.mp4', 21)
```

The document was converted with **Code-Format**: https://code-format.com/

In [67]:

```
max_count = max_count[1]
max_count
```

```
21
```

Вивести всі розширення, що відповідають цій найбільшій кількості входжень.

▸ Підказка

Як варіант: обійти пари `.items()`, що зберігаються в словнику, і вивести потрібні розширення.

In [68]:

```
for ext, count in d.items():
    if count == max_count:
        print(ext)
```

```
.mp4
```

Класифікація розширень

Як за розширенням визначити, чи потрапляє воно в задані переліки?

▸ Підказка

Використайте оператор `in`

Запишіть логічний вираз, що задає ознаку того, що розширення .txt наявне у білому списку.

▸ Відповідь

`'.txt' in WHITE_LIST`

In [69]:

```
'.txt' in WHITE_LIST
```

```
True
```

The document was converted with **Code-Format**: https://code-format.com/

Напишіть функцію `kind`, що за розширенням повертає рядок з символом W,C,O (див. умову)

In [72]:

```
sorted_stats = sorted(d.items(), key=lambda x: x[1], reverse=True)

print(f"{'EXT':<10} | {'COUNT':<10} | {'KIND':<5}")
print("-" * 30)

def kind(extension):
    if extension in WHITE_LIST:
        return 'W'
    elif extension in CODE_LIST:
        return 'C'
    else:
        return 'O'

for ext, count in sorted_stats:
    k = kind(ext)
    print(f"{ext:<10} | {count:<10} | {k:<5}")
```

```
EXT        | COUNT      | KIND
------------------------------
.mp4       | 21         | O
.prproj    | 12         | O
.prin      | 1          | O
.mp3       | 1          | O
.png       | 1          | O
.jpg       | 1          | O
```

Модифікуйте словник так, щоб для кожного розширення він зберігав:

1. кількість файлів
2. їх сумарний розмір
3. їх сумарну кількість рядків тексту (тільки для файлів з текстом програм, для інших — 0)

Значеннями словника можуть бути списки з трьох елементів (кількість файлів, сумарний розмір, сумарна кількість рядків тексту)

In [73]:

```
sample = {'.cpp':[23, 123456, 56]}   # приклад
```

The document was converted with **Code-Format**: https://code-format.com/

При перегляді файлів слід оновлювати поля значення. Наприклад `sample['.cpp'][0]+=1` збільшує кількість файлів з розширенням на 1 `sample['.cpp'][1]+=1234` додає розмір файлу (1234) до сумарного розміру

Якщо треба рахувати інформацію не тільки в розрізі розширень, а ще й місяців, то в якості ключів можна використати трійки: рік, місяць, розширення.

In [74]:

```
sample = {(2024, 9, '.cpp'):[23, 123456, 56]}   # приклад
sample
```

```
{(2024, 9, '.cpp'): [23, 123456, 56]}
```

In [75]:

```
sample[2024, 9, '.cpp']
```

```
[23, 123456, 56]
```

Кількість рядків файлу

Створити шлях до якого-небудь власного файлу з текстом програми. (Підставити власний шлях.)

In [81]:

```
sample = root.joinpath('C:/Users/Alex/source/repos/Lab5_c++/Lab5_c++/
main.cpp')
```

In [82]:

```
sample.stat().st_size
```

```
1085
```

Знайдемо кількість рядків файлу

The document was converted with **Code-Format**: https://code-format.com/

In [83]:

```
if sample.stat().st_size < 2**20:
    lines = 0
    with open(sample) as f:
        for _ in f:  # ітеруємо рядками    _  тут ім'я змінної, що
обходить рядки файлу, можна змінити
            lines += 1
    print(lines)
```

```
53
```

Якщо рядок порожній, то він хибний. Якщо рядок містить тільки білі символи, то це можна визначити за допомогою методу `isspace` (методи класу `str`)

In [84]:

```
help(str.isspace)
```

```
Help on method descriptor isspace:

isspace(self, /) unbound builtins.str method
    Return True if the string is a whitespace string, False otherwise.

    A string is whitespace if all characters in the string are
whitespace and there
    is at least one character in the string.
```

Модифікувати підрахунок кількості рядків файлу так, щоб додатково рахувалась кількість рядків файлу, що містять небілі символи (непорожні і не тільки з білих символів). У кінці вивести кількість рядків та кількість рядків з небілими символами.

The document was converted with **Code-Format**: https://code-format.com/

In [86]:

```
if sample.stat().st_size < 2**20:
    lines = 0
    notempty_lines = 0
    with open(sample) as f:
        for s in f:
            lines += 1
            if s.strip():
                notempty_lines += 1
    print(lines, notempty_lines)
```

```
53 41
```

Написати функцію, що визначає для файла кількість його рядків з небілими символами.

`def lines_counter(path): ??? return ???`

In [96]:

```
def lines_counter(path):
    count = 0
    try:
        with open(path, encoding='utf-8', errors='ignore') as f:
            for s in f:
                if s.strip():
                    count += 1
    except Exception:
        return 0

    return count
```

Перевіримо нашу фукцію зоча б трішечки.

In [97]:

```
lines_counter(sample)
```

```
41
```

Як зберегти табличну інформацію в csv-файл

The document was converted with **Code-Format**: https://code-format.com/

Приклад даних. Табличні дані (як список контейнерів). (Використовувати списки чи кортежі тут різниці не має.) Та перелік заголовків до таблиці.

In [90]:

```
data = [(2025, 1, 13, 23456, 4.5, 99, 'текст1'),
        (2025, 7, 35, 23456, 4.5, 99, 'текст2'),
        (2025, 4, 34, 23456, 4.5, 99, 'текст3'),
        (2025, 11, 24, 23456, 4.5, 99, 'текст1')]
headers = ['h1', 'h2', 'h3', 'h4', 'h5', 'h6', 'h7']
```

Роботу виконуватиме бібліотека (документація), більш конкретно `writer` (документація) . Для читання є `reader` (документація).

In [91]:

```
import csv
```

In [92]:

```
with open('data.csv', 'w', encoding='utf-8', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(headers)
    for row in data:
        writer.writerow(row)
```

In [93]:

```
with open('data.csv', encoding='utf-8', newline='') as f:
    reader = csv.reader(f)
    headers = next(reader)
    lst = list()
    for row in reader:
        lst.append(row)
        print(row)
```

```
['2025', '1', '13', '23456', '4.5', '99', 'текст1']
['2025', '7', '35', '23456', '4.5', '99', 'текст2']
['2025', '4', '34', '23456', '4.5', '99', 'текст3']
['2025', '11', '24', '23456', '4.5', '99', 'текст1']
```

Збирання задачі

The document was converted with **Code-Format**: https://code-format.com/

Нехай протягом навчання програмні проєкти з різних предметів зберігалися в папках ... . Є перелік папок `folders`, в яких зберігаються ці проєкти (задає виконавець, виходячи з вмісту своїх носіїв даних).
Примітка. До цього переліку мають входити не окремі папки з проєктами, а папки, що містять, наприклад, всі проєкти такої-то дисципліни такого-то семестру тощо. Інакше кажучи, не папка одного solution, а папка, в якій зберігаються solutions (якщо в термінології Visual Studio).

In [99]:

```
folders = ["D:/mother", "C:/Users/Alex/source/repos/Lab5_c++/Lab5_c+
+"]   # задати перелік
folders
```

```
['D:/mother', 'C:/Users/Alex/source/repos/Lab5_c++/Lab5_c++']
```

Щоб профільтрувати допоміжні файли середовищ, введемо "білий список" розширень (`WHITE_LIST`) та список розширень файлів з текстом програм(задається) (`CODE_LIST`).

Ці переліки були задані в умові.

In [100]:

```
for el in WHITE_LIST: print(el, end=' ')
```

```
.html .json .js .h .pyw .cpp .c .ipynb .hpp .cs .py .java .txt
```

In [101]:

```
for el in CODE_LIST: print(el, end=' ')
```

```
.js .h .pyw .cpp .c .hpp .cs .py .java
```

The document was converted with **Code-Format**: https://code-format.com/

Задача 1 Скільки різних розширень зустрічається у папках з заданого списку (тут і далі: включаючи вкладені), скільки файлів з кожним розширенням.
(Для цієї задачі розглядаємо всі розширення, а не тільки "білого списку".)

Відповідну інформацію вивести на екран охайною табличкою. На рядку виводиться розширення, кількість файлів та ознака (C,W,O відповідно для файлів з текстом програм (відповідно до `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

Наприклад, `.txt 155 W .ipynb 73 W .cpp 45 C .exe 3 O`

1. Створити порожній словник `d`. Його ключами будуть розширення, а значеннями відповідні кількості файлів.
2. Обійти всі папки з переліку `folders`. Для кожної папки запустити рекурсивний обхід, під час якого рахувати кількості файлів окремо для кожного розширення.
3. Перетворити словник на список пар і відсортувати цей список по другій координаті в спадному порядку.
4. Обійти список пар і для кожного елемента вивести запитувану інформацію за допомогою f-рядка (налаштовуючи ширину поля виведення(, щоб досягти охайної таблиці.

Використати раніше розроблену функцію `kind`

The document was converted with **Code-Format**: https://code-format.com/

In [102]:

```
from pathlib import Path
d = {}

for folder_path in folders:
    root = Path(folder_path)
    if root.exists():
        for el in root.glob('**/*'):
            if el.is_file():
                ext = el.suffix
                d[ext] = d.get(ext, 0) + 1

sorted_d = sorted(d.items(), key=lambda x: x[1], reverse=True)

print(f"{'EXT':<15} | {'COUNT':<10} | {'KIND':<5}")
print("-" * 35)

for ext, count in sorted_d:
    k = kind(ext)
    print(f"{ext:<15} | {count:<10} | {k:<5}")
```

```
EXT             | COUNT      | KIND
-----------------------------------
.mp4            | 21         | O
.prproj         | 12         | O
.tlog           | 8          | O
.prin           | 1          | O
.mp3            | 1          | O
.png            | 1          | O
.jpg            | 1          | O
.vcxproj        | 1          | O
.filters        | 1          | O
.user           | 1          | O
.cpp            | 1          | W
.recipe         | 1          | O
.ilk            | 1          | O
.log            | 1          | O
.obj            | 1          | O
.idb            | 1          | O
.pdb            | 1          | O
.lastbuildstate | 1          | O
```

In [ ]:

The document was converted with **Code-Format**: https://code-format.com/

In [ ]:

```
```

Задача 2 За датами створення файлів для кожного місяця, починаючи з вересня 2024-го року, знайти таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний обсяг у байтах, кількість їх рядків (враховувати тільки для файлів категорії C; білі та порожні рядки не рахувати), середній розмір файлу, середня кількість рядків.

Відповідну інформацію вивести на екран охайною табличкою в хронологічному порядку.

Наприклад, `2024 09 23 123456 256 5368 11 2024 10 23 123456 256 5368 11 2024 11 134 123456 256 921 11`

1. Створити порожній словник `d`. Його ключами будуть пари (рік, місяць), а значеннями відповідні кількості файлів, їх сумарний обсяг, кількість їх рядків.
2. Обійти всі папки з переліку `folders`. Для кожної папки запустити рекурсивний обхід, під час якого обробляти тільки файли з розширеннями з білого списку. При обробленні файлу визначати рік та місяць його створення та оновлювати інформацію в словнику (за потреби додавати ключі).
3. Перетворити словник на список пар і відсортувати цей список за хронологією.
4. Обійти список пар і для кожного елемента вивести запитувану інформацію за допомогою f-рядка (налаштовуючи ширину поля виведення), щоб досягти охайної таблиці. Середні значення обчислювати під час обходу, не зберігаючи в структурі даних.

Використати раніше розроблені раніше функції `created_at`, `lines_counter`

The document was converted with **Code-Format**: https://code-format.com/

In [109]:

```
import datetime
import os
from pathlib import Path

def created_at(path):
    timestamp = os.path.getctime(path)
    return datetime.datetime.fromtimestamp(timestamp)

d = {}
START_DATE = (2024, 9)

for folder_path in folders:
    root = Path(folder_path)
    if root.exists():
        for el in root.glob('**/*'):
            if el.is_file() and el.suffix in WHITE_LIST:
                dt = created_at(el)
                ym = (dt.year, dt.month)

                if ym < START_DATE:
                    continue

                if ym not in d:
                    d[ym] = [0, 0, 0]

                d[ym][0] += 1
                d[ym][1] += el.stat().st_size

                if el.suffix in CODE_LIST:
                    d[ym][2] += lines_counter(el)

sorted_stats = sorted(d.items())

print(f"{'YEAR':<6} {'MON':<4} {'COUNT':<8} {'SIZE':<12} {'LINES':<8}
{'AVG_SIZE':<10} {'AVG_LINE':<10}")
print("-" * 75)

for (year, month), stats in sorted_stats:
    count, total_size, total_lines = stats
    avg_size = total_size / count if count else 0
    avg_lines = total_lines / count if count else 0

    print(f"{year:<6} {month:<4} {count:<8} {total_size:<12}
{total_lines:<8} {avg_size:<10.0f} {avg_lines:<10.1f}")
```

The document was converted with **Code-Format**: https://code-format.com/

```
YEAR   MON  COUNT    SIZE         LINES    AVG_SIZE   AVG_LINE
---------------------------------------------------------------------
-----
2025   11   1        1085         41       1085       41.0
```

In [ ]:

```

```

In [ ]:

```

```

In [ ]:

```

```

In [ ]:

```

```

Задача 3 Для кожного місяця та кожного розширення, що зустрічаються при перегляді файлів, розміщених у папках зі списку, знайти таку інформацію: рік, місяць, розширення, кількість файлів, сумарний обсяг у байтах, кількість їх рядків (тільки для розширень категорії C; білі та порожні рядки не рахувати; для інших розширень цю клітину таблиці залишаємо незаповненою)

Цю інформацію записати в csv-файл з іменем `lab1_1.csv` (кодування тільки utf-8), на початку додати рядок з заголовками: year, mon, ext, cnt, size, lines, avg_size, avg_line.

1. Створити порожній словник `d`. Його ключами будуть пари (рік, місяць, розширення), а значеннями відповідні кількості файлів, їх сумарний обсяг, кількість їх рядків.
2. Обійти всі папки з переліку `folders`. Для кожної папки запустити рекурсивний обхід, під час якого обробляти тільки файли з розширеннями з білого списку. При обробленні файлу визначати рік та місяць його створення та оновлювати інформацію в словнику (за потреби додавати ключі).
3. Перетворити словник на список пар і відсортувати цей список за хронологією.
4. Обійти список пар і для кожного елемента вивести запитувану інформацію за допомогою f-рядка (налаштовуючи ширину поля виведення), щоб досягти охайної таблиці. Середні значення обчислювати під час обходу, не зберігаючи в структурі даних.

Використати раніше розроблені раніше функції `created_at`, `lines_counter`

Примітка. Не забороняється спочатку виконати задачу 3, а потім за її даними отримати результат задачі 2. Але це може виявитись зробити складніше, ніж у пропонованій вище схемі.

The document was converted with **Code-Format**: https://code-format.com/

In [111]:

```
import csv
import datetime
import os
from pathlib import Path

def created_at(path):
    timestamp = os.path.getmtime(path)
    return datetime.datetime.fromtimestamp(timestamp)

d = {}

for folder_path in folders:
    root = Path(folder_path)
    if root.exists():
        for el in root.glob('**/*'):
            if el.is_file() and el.suffix in WHITE_LIST:
                dt = created_at(el)
                key = (dt.year, dt.month, el.suffix)

                if key not in d:
                    d[key] = [0, 0, 0]

                d[key][0] += 1
                d[key][1] += el.stat().st_size

                if el.suffix in CODE_LIST:
                    d[key][2] += lines_counter(el)

sorted_stats = sorted(d.items())

filename = 'lab1_1.csv'
with open(filename, 'w', encoding='utf-8', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['year', 'mon', 'ext', 'cnt', 'size', 'lines',
'avg_size', 'avg_line'])

    for (year, month, ext), stats in sorted_stats:
        count, total_size, total_lines = stats

        avg_size = int(total_size / count) if count else 0

        if ext in CODE_LIST:
            lines_val = total_lines
            avg_line_val = int(total_lines / count) if count else 0
        else:
            lines_val = ''
            avg_line_val =
```

The document was converted with Code-Format: https://code-format.com/

```
        writer.writerow([year, month, ext, count, total_size,
lines_val, avg_size, avg_line_val])
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

The document was converted with **Code-Format**: https://code-format.com/

Лабораторна робота № 2
варіант № 99

Метод найменших квадратів

Постановка задачі

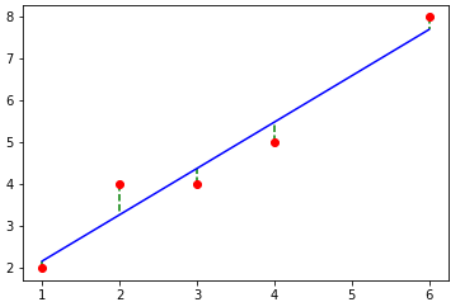
У результаті серії експериментів було отримано дані

$$(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$$

x_i — вхідні дані експерименту

y_i — відповідні вихідні дані.

Припущення експертів: $y_i = f(x_i)$ для $f(x) = ax + b$



Задача: знайти коефіцієнти a, b лінійної функції f.

Мінімізуємо суму квадратів відстаней від отриманих значень до розрахункових.
Задача: знайти значення параметрів a, b, які мінімізують значення

$$F(a, b) = \sum_{i=0}^{n} \left(f\left(x_i\right) - y_i\right)^2 = \sum_{i=0}^{n} \left(ax_i + b - y_i\right)^2$$

Математичні розрахунки

$$F(a,b)=\sum_{i=0}^{n}\left(f\left(x_i\right)-y_i\right)^2=\sum_{i=0}^{n}\left(ax_i+b-y_i\right)^2\longrightarrow\min$$

Шукаємо точку локального екстремуму функції $F(a,b)$

$$\begin{cases}F'_a(a,b)=0\\F'_b(a,b)=0\end{cases}\quad\Leftrightarrow\quad\begin{cases}\sum\limits_{i=0}^{n}2(ax_i+b-y_i)x_i=0\\\sum\limits_{i=0}^{n}2(ax_i+b-y_i)=0\end{cases}\quad\Leftrightarrow\quad\begin{cases}\sum\limits_{i=0}^{n}(ax_i+b-y_i)x_i=0\\\sum\limits_{i=0}^{n}(ax_i+b-y_i)=0\end{cases}$$

$$\begin{cases}\left(\sum\limits_{i=0}^{n}x_i\right)b+\left(\sum\limits_{i=0}^{n}x_ix_i\right)\quad a=\left(\sum\limits_{i=0}^{n}x_iy_i\right)\\\left(\sum\limits_{i=0}^{n}1\right)b+\left(\sum\limits_{i=0}^{n}x_i\right)\quad a=\left(\sum\limits_{i=0}^{n}y_i\right)\end{cases}\quad\Leftrightarrow\quad\begin{cases}\left(\sum\limits_{i=0}^{n}1\right)b+\left(\sum\limits_{i=0}^{n}x_i\right)\quad a=\left(\sum\limits_{i=0}^{n}y_i\right)\\\left(\sum\limits_{i=0}^{n}x_i\right)b+\left(\sum\limits_{i=0}^{n}x_ix_i\right)\quad a=\left(\sum\limits_{i=0}^{n}x_iy_i\right)\end{cases}$$

Розглядаємо матриці

$$X=\begin{pmatrix}1&x_0\\1&x_1\\\cdot&\cdot\\\cdot&\cdot\\1&x_n\end{pmatrix}\quad Y=\begin{pmatrix}y_0\\y_1\\\cdot\\\cdot\\y_n\end{pmatrix}\quad C=\begin{pmatrix}b\\a\end{pmatrix}$$

Тоді систему можна записати в матричному вигляді

$$AC=B,\quad \text{де}\quad A=X^TX,B=X^TY$$

Знайдемо значення функції в точках $x_0,x_1,\ldots,x_n$

$$XC=\begin{pmatrix}b+ax_0\\b+ax_1\\\cdot\\\cdot\\\cdot\\b+ax_n\end{pmatrix}=\begin{pmatrix}f(x_0)\\f(x_1)\\\cdot\\\cdot\\\cdot\\f(x_n)\end{pmatrix}$$

Послідовність обчислень

$$data=\begin{pmatrix}1&x_0&y_0\\1&x_1&y_1\\\cdot&\cdot&\cdot\\\cdot&\cdot&\cdot\\\cdot&\cdot&\cdot\\1&x_n&y_n\end{pmatrix}\qquad X=\begin{pmatrix}1&x_0\\1&x_1\\\cdot&\cdot\\\cdot&\cdot\\1&x_n\end{pmatrix}\qquad Y=\begin{pmatrix}y_0\\y_1\\\cdot\\\cdot\\y_n\end{pmatrix}$$

$X=data[:,0{:}2]$, $Y=data[:,2{:}3]$, $A=X^TX$, $B=X^TY$, розв'язуємо рівняння $AC=B$

$$Z=XC=\begin{pmatrix}f(x_0)\\f(x_1)\\\cdot\\\cdot\\\cdot\\f(x_n)\end{pmatrix}$$

LAB2 part1 Оцінка часової складності (робочий зошит)

Exported 2/28/2026, 2:28:37 PM

LAB2 Частина 1. Оцінка параметрів функції

Виконавець: **Базилевич Олексій**, група **К27**
Викладач: **Андрій Ляшко**

Нам знадобляться бібліотеки `numpy`, `scipy`, `matplolib`.
Бібліотеки є сторонніми, отже потребують встановлення. Краще у віртуальне оточення. Мали бути встановлені, якщо ні (клітина з імпортуванням виконується з помилкою), то треба встановити. Можна прямо з Jupyter. Тільки не під час пари. Встановлення `matplolib` вимагатиме часу.

Наступну клітину можна розкоментовувати та виконувати тільки, якщо в якості ядра (kernel) вибране віртуальне оточення!

In [1]:

```
#!pip install scipy, matplotlib
```

Якщо встановлювати не в оточення, то хоча б тільки для користувача (з ключем --user).

In [2]:

```
#!pip3.14 install scipy, matplotlib --user    # Linux
```

In [3]:

```
#!py -3.14 -m pip install scipy, matplotlib --user    # Windows
```

The document was converted with **Code-Format**: https://code-format.com/

In [4]:

```
import numpy as np
import scipy, scipy.linalg as la
import matplotlib, matplotlib.pyplot as plt
print(f'numpy {np.__version__}, scipy {scipy.__version__}, matplotlib
{matplotlib.__version__}')
```

```
numpy 2.4.2, scipy 1.17.1, matplotlib 3.10.8
```

Постановка задачі

У результаті кількох експериментів з обчислення швидкодії програми була отримана низка даних вигляду (n, t), де
n — розмір оброблюваних вхідних даних,
t — кількість часу, що була в середньому необхідна для їх обробки.

Дослідник знає, що часова складність програми повинна мати вигляд $T(n) = \Theta(n^a)$. Знайти значення параметра a.

Зауваження. Якщо $T(n) = \Theta(n^a)$, то можна вважати, що $T(n) \approx cn^a$.

Підготовка даних

Спочатку підготуємо "експериментальні" дані. Для цього слід задати значення параметрів c та a та згенерувати "експериментальні" дані в точках також вручну заданої майже логарифмічної шкали n, що містить розміри оброблюваних даних.

В якості шкали використаємо послідовність значень 1, 2, 4, 10, 20, 50, 100, 200, 500, 1000.

Створити змінні c, a зі значеннями 4.5 та 2.2 відповідно.

In [5]:

```
c = 4.5
a = 2.2
```

The document was converted with **Code-Format**: https://code-format.com/

Створити багатовимірний масив `n` розмірності 1, який зберігає шкалу значень.

Допоможе функція `np.array` [Документація](#).

▸ Підказка.
Шкалу у функцію можна передати як кортеж (послідовність значень, записана через кому і взята в круглі дужки; наприклад, (1, 2, 3)).

In [6]:

```
n = np.array((1, 2, 4, 10, 20, 50, 100, 200, 500, 1000))
n
```

```
array([   1,    2,    4,   10,   20,   50,  100,  200,  500, 1000])
```

Далі треба згенерувати результати експериментів. Для цього напишемо функцію `T_shifted`, що за розміром даних буде повертати кількість часу, схожу на час роботи програми на цих даних. Далі ця функція буде застосовуватись до багатовимірних масивів, тому в ній можна використовувати математичні оператори та математичні функції з бібліотеки `numpy`.

Використаємо, наприклад, таку функцію.

$$T_{shifted}(n) = cn^a + 2.5n^{a-1} + 0.23\ln(n)$$

In [7]:

```
def T_shifted(n):
    return c*n**a+2.5*n**(a-1)+0.23*np.log(n)
```

За допомогою функції `T_shifted` згенеруємо результати експериментів та запам'ятаємо їх у змінній `t`.

▸ Підказка.
Застосуйте функцію безпосередньо до масиву.

The document was converted with **Code-Format**: https://code-format.com/

In [8]:

```
t = T_shifted(n)
t
```

```
array([7.00000000e+00, 2.65794860e+01, 1.08518496e+02, 7.53353861e+02,
       3.36873279e+03, 2.48748869e+04, 1.13663920e+05, 5.20815885e+05,
       3.90327333e+06, 1.79247769e+07])
```

Перехід до логарифмічної шкали та застосування методу найменших квадратів (МНК)

Якщо $T(n) \approx cn^a$, то $\log T(n) \approx \log c + a \log n$.

(Основа логарифму значення насправді не має. В обчисленнях можна використовувати натуральний логарифм, що обчислюється функцією `np.log`.)

Введемо позначку $b = \ln c$. Тоді попереднє співвідношення перепишеться у вигляді $\log T(n) \approx b + a \log n$. Оцінку параметрів a та b останнього співвідношення можна здійснити за допомогою МНК (див. у файлі LSM.pdf).

- Перейти до логарифмів розмірів вхідних даних, тобто від n до $x = \ln n$. Отримані значення записати в змінну `x`.
- Перейти до логарифмів результатів експериментів, тобто від t до $y = \ln t$. Отримані значення записати в змінну `y`.

▸ Підказка.
Функція обчислення логарифму `np.log` є універсальною: її можна застосовувати до багатовимірних масивів.

In [9]:

```
x = np.log(n)
y = np.log(t)
```

The document was converted with **Code-Format**: https://code-format.com/

In [10]:

```
x, y
```

```
(array([0.        , 0.69314718, 1.38629436, 2.30258509, 2.99573227,
        3.91202301, 4.60517019, 5.29831737, 6.2146081 , 6.90775528]),
 array([ 1.94591015,  3.28013972,  4.68692063,  6.62453505,
8.12229193,
        10.12161402, 11.6410013 , 13.16315187, 15.17732607, 16.7016945
]))
```

Створити матрицю x (див. LSM.pdf).

$$X = \begin{pmatrix} 1 & x_0 & 1 & x_1 & \dots & & \dots & 1 & x_n \end{pmatrix}$$

▸ Підказки.

Спосіб 1.

- Створити матрицю одиниць розміру `(x.size, 2)` (можна використати функцію `np.ones` або `np.full`).
- Присвоїти її другому стовпчику значення `x`. Другий стовпчик це другий рядок транспонованої матриці.

Спосіб 2.

- Створити вектор-стовпчик одиниць розміру `x.size` (можна використати функцію `np.ones`).

  Вектор-стовпчик це двовимірний масив з другою розмірністю 1.
  Наприклад, вектор-стовпчик

  $$\begin{pmatrix} 1 \\ 25 \\ 39 \end{pmatrix}$$

  має форму `(3, 1)`.
- Перетворити `x` на вектор-стовпчик, змінивши форму за допомогою методу `reshape`.
- Зчепити вектор-стовпчик одиниць та перетворений на вектор-стопчик вектор `x` за допомогою функції `np.concatenate` та результат присвоїти змінній x.

The document was converted with **Code-Format**: https://code-format.com/

In [11]:

```
X = np.ones((x.size, 2))
X.T[1] = x
X
```

```
array([[1.        , 0.        ],
       [1.        , 0.69314718],
       [1.        , 1.38629436],
       [1.        , 2.30258509],
       [1.        , 2.99573227],
       [1.        , 3.91202301],
       [1.        , 4.60517019],
       [1.        , 5.29831737],
       [1.        , 6.2146081 ],
       [1.        , 6.90775528]])
```

Створити матрицю Y (див. LSM.pdf)

$$Y = \left(y_0\ y_1\ \cdots\ y_n\right)$$

тобто перетворити вектор y на вектор-стовпчик і запам'ятати в змінній Y.

▸ Підказка.

Застосувати до вектору y метод `reshape`.

In [12]:

```
Y = np.reshape(y, (y.size, 1))
Y
```

```
array([[ 1.94591015],
       [ 3.28013972],
       [ 4.68692063],
       [ 6.62453505],
       [ 8.12229193],
       [10.12161402],
       [11.6410013 ],
       [13.16315187],
       [15.17732607],
       [16.7016945 ]])
```

The document was converted with **Code-Format**: https://code-format.com/

Оцінку параметрів a та b можна здійснити за допомогою МНК (див. LSM.pdf). Матриця

$$C = \begin{pmatrix} b & a \end{pmatrix}$$

є розв'язком матричного рівняння *де*

$$AC = B, \quad \text{де} \quad A = X^T X, \quad B = X^T Y$$

Створити матриці A та B (див. LSM.pdf).

$$A = X^T X, \quad B = X^T Y$$

▶ Підказки.

- Отримати транспоновану матрицю можна через атрибут T.
- Оператор матричного множення записується так @

In [13]:

```
A = X.T@X
B = X.T@Y
```

In [14]:

```
A, B
```

```
(array([[ 10.        ,  34.31563284],
        [ 34.31563284, 167.6006952 ]]),
 array([[ 91.46458524],
        [420.99652603]]))
```

За допомогою функції `solve` з бібліотеки `scipy.linalg`, що містить засоби лінійної алгебри, розв'язуємо лінійну систему рівнянь, задану в матричному вигляді.

Увага! У більш нових версіях бібліотеки результатом буде не вектор, а вектор-стовпчик.

In [15]:

```
C = la.solve(A, B)
C
```

```
array([[1.77104215],
       [2.14928759]])
```

The document was converted with **Code-Format**: https://code-format.com/

Слід згадати, що

$$C = (b\ a) = (\ln c\ a)$$

In [16]:

```
a_est = C[1, 0]
c_est = np.exp(C[0, 0])
```

In [17]:

```
print(f'{c = :}, отримали оцінку {c_est:.3f}, різниця {abs(c-
c_est):.4f}\n\
{a = :}, отримали оцінку {a_est:.3f}, різниця {abs(a-a_est):.4f}')
```

```
c = 4.5, отримали оцінку 5.877, різниця 1.3770
a = 2.2, отримали оцінку 2.149, різниця 0.0507
```

Візуальна оцінка якості наближення

Для цього побудуємо графік знайденої функції та накладемо на нього вхідні дані.

Почнемо з того, що побудуємо функцію `T_estimated`, що обчислює час роботи програми за формулою $T(n) = cn^a$ з використанням знайдених оцінок `a_est`, `c_est` параметрів a та c.

Аналогічно функції `T_shifted`, вона буде застосовна до багатовимірних масивів.

In [18]:

```
def T_estimated(n):
    return c_est*n**a_est
```

На вході у нас були значення часу роботи програми в таких точках

In [19]:

```
n
```

```
array([   1,    2,    4,   10,   20,   50,  100,  200,  500, 1000])
```

The document was converted with **Code-Format**: https://code-format.com/

Можна бачити, що шкала була нерівномірна (і це було правильно! для тієї задачі, що розв'язувалась). Але для візуалізації будемо будувати графік поточково, тому потрібні значення в проміжних точках.

Почнемо з того, що визначимо точки, які використаємо в поточковій побудові графіку. Отриманий масив точок, які рівномірно розташовані по всій шкалі, запам'ятаємо в змінній `scale`.

▶ Підказки

- Знайти мінімальне та максимальне значення шкали розмірів вхідних даних. Використати для цього методи `min` та `max` класу багатовимірних масивів.
- За допомогою функції `np.linspace` отриманий числовий проміжок розбити на 500 рівних частин та точки поділу записати в масив `scale`.

In [20]:

```
scale = np.linspace(np.min(n), np.max(n), num=501)
```

З використанням функції `T_estimated` обчислимо розрахункові значення функції часової складності на всій шкалі і запам'ятаємо в змінній `times`.

▶ Підказка.
Функція `T_estimated` може бути застосована до масиву `scale` безпосередньо.

In [21]:

```
times = T_estimated(scale)
```

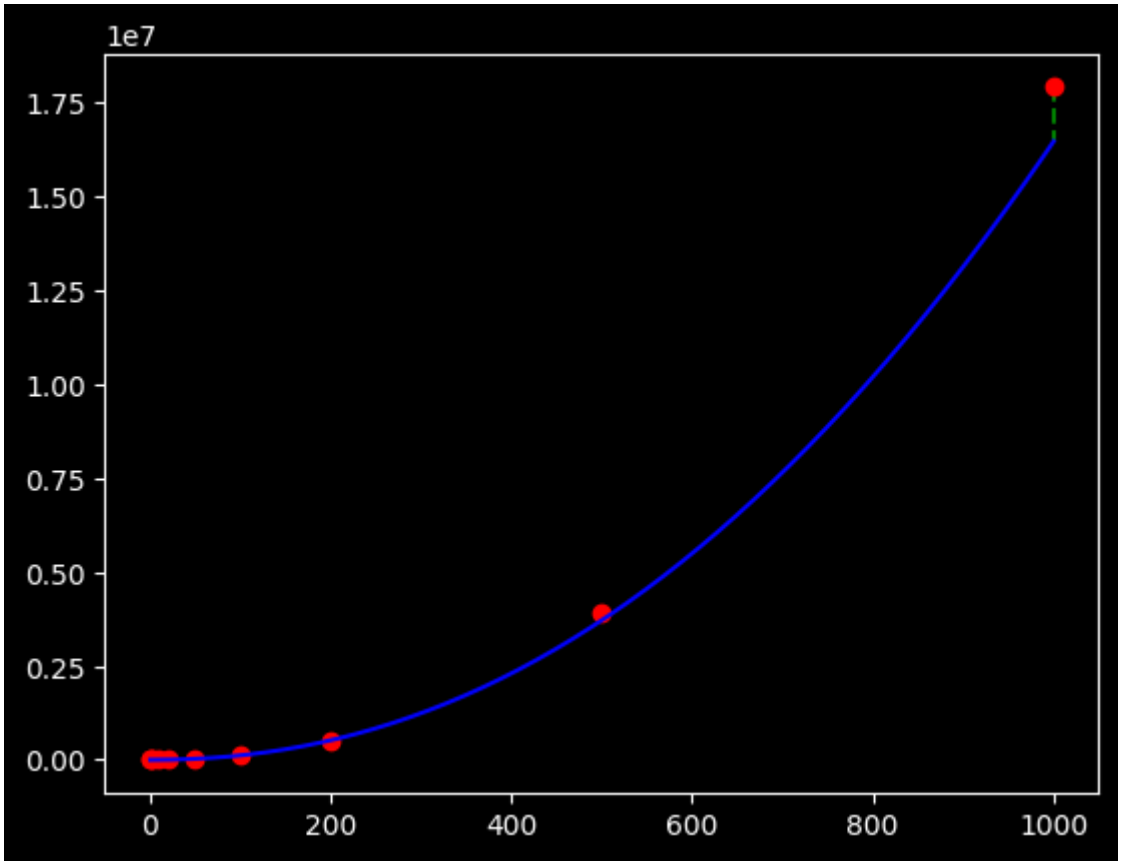
Візуалізуємо отримані результати.

На графіку червоні точки відповідають "експериментальним" даним. Синім зображено графік отриманої залежності часу роботи від розмірів даних.

The document was converted with **Code-Format**: https://code-format.com/

In [22]:

```
for i, px in np.ndenumerate(n):
    py = t[i]
    pz = T_estimated(px)
    plt.plot((px, px),(py, pz), color="green", linestyle='dashed')
plt.plot(n, t, color="red", marker='o', linestyle='None')
plt.plot(scale, times, color='blue')
#plt.show()
#plt.savefig('1.png')
```



Як зміниться якість наближення, якщо не розглядати експериментальні дані в малих точках?

Частину 1 зроблено!

The document was converted with **Code-Format**: https://code-format.com/

LAB2 part2 Оброблення даних про файли (робочий зошит)

Exported 2/28/2026, 2:28:42 PM

LAB2 Частина 2. Оброблення даних про файли

Виконавець: **Базилевич Олексій**, група **К-27**
Викладач: **Андрій Ляшко**

Нам знадобиться бібліотека `pandas`.
Бібліотека є сторонньою, отже потребує встановлення. Краще у віртуальне оточення. Мала бути встановлена, якщо ні (клітина з імпортуванням виконується з помилкою), то треба встановити. Можна прямо з Jupyter. Тільки не під час пари.

Наступну клітину можна розкоментовувати та виконувати тільки, якщо в якості ядра (kernel) вибране віртуальне оточення!

In [1]:

```
#!pip install pandas
```

Якщо встановлювати не в оточення, то хоча б тільки для користувача (з ключем --user).

In [2]:

```
#!pip3.14 install pandas --user    # Linux
```

In [3]:

```
#!py -3.14 -m pip install pandas --user    # Windows
```

The document was converted with **Code-Format**: https://code-format.com/

In [4]:

```
from pathlib import Path

import numpy as np
import pandas as pd
print(f'pandas {pd.__version__}')
```

```
pandas 3.0.1
```

Постановка задачі

Згадка про минуле (Лабораторну роботу №1)
Протягом навчання програмні проєкти з різних предметів зберігалися в папках з переліку `folders`, який оброблявся в лабораторній роботі №1.
Щоб профільтрувати допоміжні файли середовищ, було введено "білий список" розширень (`WHITE_LIST`) та список розширень файлів з текстом програм(задається) (`CODE_LIST`).
Також було написано функцію `kind`, що за розширенням повертає рядок з символом C,W або O відповідно для файлів з текстом програм (відповідно до переліку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список).

У результаті виконання лабораторної роботи для кожного місяця та кожного розширення, що зустрічаються при перегляді файлів, розміщених у папках зі списку, було знайдено таку інформацію:

`рік, місяць, розширення, кількість файлів, сумарний обсяг у байтах, кількість їх рядків.`

Кількість рядків наводиться тільки для розширень категорії C; білі та порожні рядки не рахуються; для інших розширень ця клітина таблиці залишається незаповненою.

За цією інформацією було побудовано csv-файл з іменем `lab1_1.csv` (кодування тільки utf-8), на початку якого розміщено рядок з заголовками: year, mon, ext, cnt, size, lines.

In [5]:

```
CODE_LIST = {'.cpp', '.hpp', '.c', '.h', '.py', '.pyw', '.cs', '.js',
'.java'}
WHITE_LIST = CODE_LIST | {'.html', '.json', '.txt', '.ipynb'}
```

In [6]:

```
headers = ['year', 'mon', 'ext', 'cnt', 'size', 'lines']
```

The document was converted with **Code-Format**: https://code-format.com/

За інформацією з csv-файлу з'ясувати

1. Скільки різних розширень зустрічається у папках з заданого списку, скільки файлів з кожним розширенням. (Для цієї задачі розглядаємо всі розширення, а не тільки "білого списку".)
   Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.
2. За датами створення файлів для кожного місяця, починаючи з вересня 2024-го року, знайти таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний розмір у кілобайтах, кількість їх рядків (враховувати тільки для файлів категорії C), середній розмір у байтах, середню кількість рядків.
   Відповідну інформацію вивести на екран у табличному вигляді в порядку, оберненому до хронологічного.

Зверніть увагу на підписи стовпчиків, що будуть виводитись.

Всі значення округлити та привести до цілого типу.

Завантаження даних

Задаємо шлях до файлу `lab1_1.csv`.

In [7]:

```
path = Path('lab1_1.csv')
```

The document was converted with **Code-Format**: https://code-format.com/

Завантажимо дані за допомогою функції `pd.read_csv` ([документація тут](#)).

Зверніть увагу на те, що:

1. файл записано в кодуванні `utf-8`
2. перший рядок файлу містить заголовки year, mon, ext, cnt, size, lines
3. значення у стовпчику lines можуть бути відсутні

▶ Підказки.
Функції варто передати параметри, що задають:

1. кодування, в якому записано текстовий файл
2. номер рядка, в якому записано заголовки
3. тип, в якому слід записати в пам'ять останній стовпчик

▶ Ще більше підказок

1. параметр sep задає роздільники полів
2. параметр encoding задає кодування
3. параметр header задає номер рядка, в якому записано заголовки (нумерація починається з 0)
4. параметр dtype може бути словником, що задає типи стовпчиків (можливо, не всіх); назві стовпчика відповідає назва типу
5. тип Int64 містить цілі числа та особливе значення NA

In [8]:

```
data = pd.read_csv(path, encoding='utf8', sep=",", header=0,
dtype={"lines": "Int64"}, names=headers)
data.head(3)
```

	year	mon	ext	cnt	size	lines
0	2021	12	NaN	2	3422	<NA>
1	2021	12	.bas	2	7979	<NA>
2	2021	12	.bat	294	85220	<NA>

The document was converted with **Code-Format**: https://code-format.com/

In [9]:

```
data.tail(3)
```

	year	mon	ext	cnt	size	lines
1021	2026	2	.xlsx	14	293131	<NA>
1022	2026	2	.yaml	18	2025769	<NA>
1023	2026	2	.zip	4	2240635049	<NA>

Подивимось інформацію про завантажені дані. Документація на метод `info`.

In [10]:

```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1024 entries, 0 to 1023
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   year    1024 non-null   int64
 1   mon     1024 non-null   int64
 2   ext     989 non-null    str
 3   cnt     1024 non-null   int64
 4   size    1024 non-null   int64
 5   lines   94 non-null     Int64
dtypes: Int64(1), int64(4), str(1)
memory usage: 49.1 KB
```

The document was converted with **Code-Format**: https://code-format.com/

Бачимо, що відсутні значення є не тільки в стовпчику `lines`, що природньо, бо кількість рядків обчислювалась не для всіх розширень, але і в стовпчику `ext` з розширеннями.

Замінимо відсутні розширення порожнім рядком.

Використаємо метод `fillna`.

Слід замінити тільки відсутні розширення, тому значення параметра `value` слід зробити словником, що для кожного стовпчика, який слід оброблювати, задає значення-заповнювач.
Не забудьте, що перетворення слід виконувати "на місці".

▸ Підказка (тільки за повного безвихіддя)

```
data.fillna(value={'ext':''}, inplace=True)
```

Альтернатива. Можна спробувати перетворити тільки стовпчик `data.ext`.

▸ Підказка (тільки за повного безвихіддя)

Використаємо метод `fillna` класу `Series`, щоб виконати заміну, і присвоїмо результат стовпчику `data.ext`. `data.ext = data.ext.fillna(value='')`

Перший спосіб більш економний за пам'яттю. Другий спосіб корисний тоді, коли для різних стовпчиків використовують різні алгоритми заповнення відсутніх значень.

In [11]:

```
data.fillna(value={'ext':''}, inplace=True)
data.head(3)
```

	year	mon	ext	cnt	size	lines
0	2021	12		2	3422	<NA>
1	2021	12	.bas	2	7979	<NA>
2	2021	12	.bat	294	85220	<NA>

The document was converted with **Code-Format**: https://code-format.com/

In [12]:

```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1024 entries, 0 to 1023
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   year    1024 non-null   int64
 1   mon     1024 non-null   int64
 2   ext     1024 non-null   str
 3   cnt     1024 non-null   int64
 4   size    1024 non-null   int64
 5   lines   94 non-null     Int64
dtypes: Int64(1), int64(4), str(1)
memory usage: 49.1 KB
```

Тепер незаповнені дані залишились тільки у стовпчику `lines`.

`Ні в якому разі там не повинно бути нулів!`

Групування та групові операції

In [13]:

```
data.head()
```

	year	mon	ext	cnt	size	lines
0	2021	12		2	3422	<NA>
1	2021	12	.bas	2	7979	<NA>
2	2021	12	.bat	294	85220	<NA>
3	2021	12	.c	1	4402	141
4	2021	12	.cbp	105	172765	<NA>

The document was converted with **Code-Format**: https://code-format.com/

Задача 1 Скільки різних розширень зустрічається в папках з заданого списку, скільки файлів з кожним розширенням?

Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

Скільки різних розширень зустрічається в папках з заданого списку?

Найпростіше: взяти стовпчик таблиці, що зберігає розширення, та знайти кількість унікальних значень.

▸ Підказки

1. Для отримання стовпчика таблиці `t` з іменем `name` можна використати вираз `t.name` або `t['name']`
2. Кількість унікальних елементів ряду можна знайти за допомогою методу `nunique` ([докуемнтаці тут](#)).

In [14]:

```
data.ext.nunique()
```

```
284
```

Скільки файлів з кожним розширенням?

Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

The document was converted with **Code-Format**: https://code-format.com/

Спочатку треба підсумувати інформацію по кожному розширенню. Допоможе метод `groupby` [документація тут](#).

Групувати слід за розширенням (параметр `by`) і для кожної групи обчислити суму кількостей файлів (метод `sum`).

Щоб не сумувати роки, місяці тощо, групування слід застосувати до вибірки потрібних стовпчиків. Маємо вибрати стовпчик, за яким групуємо, і стовпчик, який сумуємо.

▸ Підказка
Щоб вибрати потрібні стовпчики, можна використати операцію індексації по сукупності стовпчиків, наприклад, `dataFrame[['columnA','columnB']]`

Результат присвоїти змінній `task1`.

In [15]:

```
task1  = data[['ext', 'cnt']].groupby(by='ext').sum()
task1
```

	cnt
ext	
	77311
.0	17
.1	4
.14	2
.17	2
...	...
.xrc	2
.yaml	76
.yml	142
.zi	2
.zip	17

284 rows × 1 columns

The document was converted with **Code-Format**: https://code-format.com/

Відсортувати за спаданням кількості файлів. Допоможе метод `sort_values` (документація тут).

Не забудьте вказати, що сортування відбувається за спаданням та "на місці".

▸ Підказки

1. параметр `by` може задавати по яких стовпчиках сортувати
2. параметр `ascending` задає спадання-зростання
3. параметр `inplace` задає чи виконується перетворення на місці або створюється копія

In [16]:

```
task1.sort_values(by='cnt', ascending=False, inplace=True)
task1.head(3)
```

	cnt
ext	
.meta	111220
.png	84997
	77311

За умовою ще треба додати стовпчик з ознакою категорії розширення файлу W,C,O (див. умову). У лабораторній роботі №1 була написана функція `kind`, що за розширенням повертає рядок з шуканою ознакою. Використаємо її.

In [17]:

```
def kind(extension):
    if extension in WHITE_LIST:
        return 'W'
    elif extension in CODE_LIST:
        return 'C'
    else:
        return 'O'
```

Додамо до таблиці стовпчик з ознаками розширень файлів.

Зараз розширення є індексом. За допомогою методу метод `reset_index` (документація тут) зробимо його стовпчиком таблиці.

Не забувайте виконувати перетворення "на місці".

The document was converted with **Code-Format**: https://code-format.com/

In [18]:

```
task1.reset_index(inplace=True)
task1.head(3)
```

	ext	cnt
0	.meta	111220
1	.png	84997
2		77311

Тепер вже додаємо стовпчик з ознаками.

▶ Підказки

1. Щоб додати до таблиці `t` стовпчик `name`, можна виконати присвоювання виразу `t['name']`
2. Присвоювати можна скаляр (не наш випадок) або інший стовпчик.
3. Щоб отримати стовпчик з ознаками, можна застосувати метод `apply` ([документація тут](#)) до стовпчика з розширеннями.

In [19]:

```
task1['kind']=task1['ext'].apply(kind)
task1.head(3)
```

	ext	cnt	kind
0	.meta	111220	O
1	.png	84997	O
2		77311	O

За допомогою методу `set_index` (документація тут) знов зробимо розширення індексом таблиці.

Тут: Значенням параметра `keys` має стати ім'я стовпчика, що слід використати як індекс.

Не забувайте виконувати перетворення "на місці".

The document was converted with **Code-Format**: https://code-format.com/

In [20]:

```
task1.set_index(keys='ext', inplace=True)
task1.head(3)
```

	cnt	kind
ext		
.meta	111220	O
.png	84997	O
	77311	O

Змінимо написи.

In [21]:

```
task1.index.name='Розширення'
task1.columns=['Кількість файлів','Ознака']
task1.head()
```

	Кількість файлів	Ознака
Розширення		
.meta	111220	O
.png	84997	O
	77311	O
.cs	70314	W
.py	47833	W

The document was converted with **Code-Format**: https://code-format.com/

In [22]:

```
task1.head(20)
```

	Кількість файлів	Ознака
Розширення		
.meta	111220	O
.png	84997	O
	77311	O
.cs	70314	W
.py	47833	W
.pyc	16264	O
.txt	11587	W
.md	10759	O
.tex	6362	O
.pyi	4451	O
.json	3220	W
.mvfrm	2997	O
.dll	2950	O
.bin	2620	O
.dat	2192	O
.js	1696	W
.pdb	1631	O
.rsp	1542	O
.asmdef	1508	O
.mat	1326	O

The document was converted with **Code-Format**: https://code-format.com/

Задача 2 За датами створення файлів для кожного місяця знайти таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний розмір у кілобайтах, кількість їх рядків (враховувати тільки для файлів категорії C), середній розмір у байтах, середню кількість рядків.

Відповідну інформацію вивести на екран у табличному вигляді в порядку, оберненому до хронологічного.

Всі значення округлити та привести до цілого типу.

План:

1. Додати до початкової таблиці `data` стовпчик `kind`, що містить ознаки розширень.
2. Вибрати дані, що стосуються розширень з "білого списку".
3. Згрупувати вибрані дані в межах року та місяця.
4. За згрупованою інформацією обчислити потрібні агрегати.
5. Виконати перетворення стовпчиків та відсортувати.

Щоб за невдачі повторно не завантажувати дані, створимо їх копію.

In [23]:

```
data1 = data.copy()
```

1. Додати до початкової таблиці `data` стовпчик `kind`, що містить ознаки розширень.

Аналогічно тому, що виконувалось раніше, додати до таблиці `data1` стовпчик `kind`, що містить ознаки.

In [24]:

```
data1['kind']=data1.ext.apply(kind)
data1.head(3)
```

	year	mon	ext	cnt	size	lines	kind
0	2021	12		2	3422	<NA>	O
1	2021	12	.bas	2	7979	<NA>	O
2	2021	12	.bat	294	85220	<NA>	O

The document was converted with **Code-Format**: https://code-format.com/

2. Вибрати дані, що стосуються розширень з "білого списку".

Запишіть вираз, що задає вибірку тільки інформації про розширення з "білого списку". Щоб далі не копіювати сам вираз, присвойте його значення змінній `selection`. (Це є зручно, коли експериментуємо, у продакшн потреби в цій змінній насправді не буде, замість неї можна буде використовувати вираз безпосередньо.)

▸ Підказка

- Розширення знаходиться в білому списку, якщо його ознакою є не O.
- Обчислити умову, що розширення знаходиться в білому списку, і використати її як аргумент операції індексації.

In [25]:

```
selection = data1[(data1.kind != 'O') & ((data1.year > 2024) |
((data1.year == 2024) & (data1.mon >= 9)))]
selection.head()
```

	year	mon	ext	cnt	size	lines	kind
288	2024	9	.c	1	27620	750	W
291	2024	9	.cpp	7	92626	1918	W
294	2024	9	.h	4	1160	57	W
295	2024	9	.json	5	2307	<NA>	W
301	2024	9	.txt	3	1827	<NA>	W

3. Згрупувати вибрані дані за ознаками в межах року та місяця.

   Допоможе метод `groupby` [документація тут](#).

Групувати слід за роком та місяцем (параметр `by` має бути списком). Оскільки далі будемо обчислювати статистики для розміру та кількості рядків, то перед групуванням з таблиці слід вибрати стовпчики, за якими групуємо і за якими обчислюємо статистики.

In [26]:

```
task2 = selection[['year', 'mon', 'cnt', 'size',
'lines']].groupby(by=['year','mon'])
```

The document was converted with **Code-Format**: https://code-format.com/

Далі необхідно знайти кількість файлів за кожен місяць, їх сумарний та середній розмір, а також сумарну та середню кількість рядків.

У результаті групування отримався об'єкт `DataFrameGroupBy` (документація тут).

До нього застосовний метод `agg`, що обчислює статистики (агрегатні характеристики) (документація тут).

Для стовпчика `cnt` будемо шукати суму значень (`sum`), для стовпчика `size` будемо шукати суму значень та середнє значення (`mean`), для стовпчика `lines` будемо шукати суму значень та середнє значення.

Першим параметром методу `agg` зробимо словник, що задає які статистики для яких стовпчиків слід обчислювати. Ключами словника мають бути імена стовпчиків (рядки), а значеннями списки статистик або окремі статистики (статистики також задаємо рядками-іменами)).

Результат агрегування зробимо значенням змінної `res2`.

In [27]:

```
res2 = task2.agg({'cnt':'sum', 'size':['sum', 'mean'], 'lines':
['sum', 'mean']})
res2.head(3)
res2.head()
```

		cnt	size		lines	
		sum	sum	mean	sum	mean
year	**mon**					
2024	**9**	20	125540	25108.000000	2725	908.333333
	10	1864	5440894	777270.571429	50299	12574.75
	11	610	747916	186979.000000	13819	4606.333333
	12	3	8167	4083.500000	211	105.5
2025	**1**	10	67140	13428.000000	1801	600.333333

Оскільки відбувалось групування по кільком стовпчикам та для окремих стовпчиків обчислювалось кілька характеристик, то отримали складені індекси (multindex) і рядків, і стовпчиків.

Поточний індекс стовпчиків.

The document was converted with **Code-Format**: https://code-format.com/

In [28]:

```
res2.columns
```

```
MultiIndex([(  'cnt',  'sum'),
            ( 'size',  'sum'),
            ( 'size', 'mean'),
            ('lines',  'sum'),
            ('lines', 'mean')],
           )
```

Перейдемо до простого індексу стовпчиків.

In [29]:

```
res2.columns = res2.columns.to_flat_index()
res2.columns
```

```
Index([   ('cnt', 'sum'),   ('size', 'sum'),  ('size', 'mean'),
        ('lines', 'sum'), ('lines', 'mean')],
      dtype='object')
```

In [30]:

```
res2.head(3)
```

		(cnt, sum)	(size, sum)	(size, mean)	(lines, sum)	(lines, mean)
year	**mon**					
2024	**9**	20	125540	25108.000000	2725	908.333333
	10	1864	5440894	777270.571429	50299	12574.75
	11	610	747916	186979.000000	13819	4606.333333

Змінимо назви стовпчиків.

The document was converted with **Code-Format**: https://code-format.com/

In [31]:

```
res2.columns=['Кількість', 'Сумарний розмір', 'Середній розмір',
'Кількість рядків', 'Середня кількість рядків',]
res2.head(3)
```

		Кількість	Сумарний розмір	Середній розмір	Кількість рядків	Середня кількість рядків
year	**mon**					
2024	**9**	20	125540	25108.000000	2725	908.333333
	10	1864	5440894	777270.571429	50299	12574.75
	11	610	747916	186979.000000	13819	4606.33333

Виразимо сумарний розмір у kb.

Для цього слід поділити значення на 1024, округлити. Додатково перетворимо на цілий тип (метод `astype` дозволяє виконувати перетворення типів, [документація тут](#)). Також не забудемо змінити підпис стовпчика.

Переходимо до кілобайтів.

In [32]:

```
res2['Сумарний розмір'] = (res2['Сумарний
розмір']/1024).round().astype(int)
```

Змінюємо підпис стовпчиків

In [33]:

```
res2.columns=['Кількість', 'Сумарний розмір, kb', 'Середній розмір,
b', 'Кількість рядків', 'Середня кількість рядків',]
```

The document was converted with **Code-Format**: https://code-format.com/

In [34]:

```
res2.head(3)
```

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
2024	**9**	20	123	25108.000000	2725	908.333333
	10	1864	5313	777270.571429	50299	12574.75
	11	610	730	186979.000000	13819	4606.33333

Округлити середній розмір та середню кількість рядків, перейти до цілих значень.

In [35]:

```
res2.info()
```

```
<class 'pandas.DataFrame'>
MultiIndex: 18 entries, (np.int64(2024), np.int64(9)) to
(np.int64(2026), np.int64(2))
Data columns (total 5 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Кількість                18 non-null     int64
 1   Сумарний розмір, kb      18 non-null     int64
 2   Середній розмір, b       18 non-null     float64
 3   Кількість рядків         18 non-null     Int64
 4   Середня кількість рядків 17 non-null     Float64
dtypes: Float64(1), Int64(1), float64(1), int64(2)
memory usage: 1001.0 bytes
```

The document was converted with **Code-Format**: https://code-format.com/

Ось тільки тепер замінимо відсутні значення кількості рядків нулями!

Якщо б це було зроблено раніше, то середня кількість рядків була б обрахована неправильно (на неї вплинули б нульові значення замість відсутніх).

▸ Підказка

Використати метод `fillna` (документація тут). Оскільки відсутні дані тільки в одному стовпчику, то все просто. Тільки не забудьте, що треба перетворювати наявний датасет, а не створювати новий.

In [36]:

```
res2.fillna(0, inplace=True)
res2.info()
```

```
<class 'pandas.DataFrame'>
MultiIndex: 18 entries, (np.int64(2024), np.int64(9)) to
(np.int64(2026), np.int64(2))
Data columns (total 5 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Кількість                 18 non-null     int64
 1   Сумарний розмір, kb       18 non-null     int64
 2   Середній розмір, b        18 non-null     float64
 3   Кількість рядків          18 non-null     Int64
 4   Середня кількість рядків  18 non-null     Float64
dtypes: Float64(1), Int64(1), float64(1), int64(2)
memory usage: 1001.0 bytes
```

Тепер можна перетворювати типи стовпчиків.

In [37]:

```
fname = 'Середній розмір, b'
res2[fname] = res2[fname].round().astype(int)

fname = 'Середня кількість рядків'
res2[fname] = res2[fname].round().astype(int)
```

The document was converted with **Code-Format**: https://code-format.com/

In [38]:

```
res2
```

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
2024	**9**	20	123	25108	2725	908
	10	1864	5313	777271	50299	12575
	11	610	730	186979	13819	4606
	12	3	8	4084	211	106
2025	**1**	10	66	13428	1801	600
	2	82	488	83310	11065	3688
	3	35387	345468	58959871	4707841	1569280
	4	33455	280612	57469303	4403387	1467796
	5	411	9962	1275176	27228	5446
	6	41	3841	1310986	2019	1010
	7	16	122	31157	3457	1152
	8	5	2	2437	0	0
	9	219	1256	257303	21947	7316
	10	429	2525	517021	39730	13243
	11	1224	4661	795394	31862	7966
	12	151	3781	774301	77363	38682
2026	**1**	20000	452666	42139076	7662090	1094584
	2	19979	80508	10304997	294211	73553

Відсортуємо в порядку, оберненому до хронологічного.

У даному випадку слід сортувати індекс. Допоможе метод `sort_index` ([документація тут](#)).

The document was converted with **Code-Format**: https://code-format.com/

In [39]:

```
res2.sort_index(ascending=[False, False], inplace=True)
res2
```

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
2026	**2**	19979	80508	10304997	294211	73553
	1	20000	452666	42139076	7662090	1094584
2025	**12**	151	3781	774301	77363	38682
	11	1224	4661	795394	31862	7966
	10	429	2525	517021	39730	13243
	9	219	1256	257303	21947	7316
	8	5	2	2437	0	0
	7	16	122	31157	3457	1152
	6	41	3841	1310986	2019	1010
	5	411	9962	1275176	27228	5446
	4	33455	280612	57469303	4403387	1467796
	3	35387	345468	58959871	4707841	1569280
	2	82	488	83310	11065	3688
	1	10	66	13428	1801	600
2024	**12**	3	8	4084	211	106
	11	610	730	186979	13819	4606
	10	1864	5313	777271	50299	12575
	9	20	123	25108	2725	908

Частину 2 зроблено!

The document was converted with **Code-Format**: https://code-format.com/

The document was converted with **Code-Format**: https://code-format.com/

LAB2 part3 Оброблення даних про успішність (робочий зошит)

Exported 2/28/2026, 2:28:48 PM

LAB2 Частина 3. Оброблення даних про успішність

Виконавець: **Базилевич Олексій**, група **К-27**
Викладач: **Андрій Ляшко**

Нам знадобиться бібліотека `pandas`.
Бібліотека є сторонньою, отже потребує встановлення. Краще у віртуальне оточення. Мала бути встановлена, якщо ні (клітина з імпортуванням виконується з помилкою), то треба встановити. Можна прямо з Jupyter. Тільки не під час пари.

Наступну клітину можна розкоментовувати та виконувати тільки, якщо в якості ядра (kernel) вибране віртуальне оточення!

In [137]:

```
#!pip install pandas
```

Якщо встановлювати не в оточення, то хоча б тільки для користувача (з ключем --user).

In [138]:

```
#!pip3.14 install pandas --user    # Linux
```

In [139]:

```
#!py -3.14 -m pip install pandas --user    # Windows
```

The document was converted with **Code-Format**: https://code-format.com/

In [140]:

```
from pathlib import Path
import pandas as pd
from scipy.interpolate import insert

print(f'pandas {pd.__version__}')
```

```
pandas 3.0.1
```

Постановка задачі

У файлі `data_new.csv` знаходяться дані з поточними результатами студентів за ЛР, що складається з двох частин. Відомо, що файли записаний у форматі csv з кодуванням utf-8, може містити білі рядки та містить заголовок. Також відомо, що інформація про жодного студента не записана на кількох рядках одночасно.

Завдання: З використанням виключно засобів бібліотеки `pandas`

1. Завантажити наявну у файлах інформацію в об'єкт `DataFrame`.
   Заголовками стовпчиків мають бути: `group`, `name`, `var`, `part1`, `part2`.
2. Знайти про скількох студентів наявна інформація у файлі.
3. Знайти студентів, хто здав обидві частини. Скільки таких студентів?
   Цікавить тільки група, ПІБ, номер варіанту, загальна кількість балів. Тут і далі: здали = отримали бали, можливо нулеві.
4. Знайти тих студентів, хто не здав жодної частини. Скільки таких студентів?
   Цікавить тільки група, ПІБ, номер варіанту.
5. Знайти тих студентів, хто здав першу частину.
   Цікавить тільки група, ПІБ, номер варіанту, бали за першу частину.
6. Знайти тих студентів групи К14, що здали хоч одну частину лабораторної. Цікавить уся наявна інформація.
7. Для кожної групи знайти кількість студентів та кількість студентів, що щось здали.

Крок 1 Завантажити наявну у файлах інформацію в об'єкт `DataFrame`.

Заголовками стовпчиків мають бути: `group`, `name`, `var`, `part1`, `part2`.

Задаємо шлях до файлу `data_new.csv`.

In [141]:

```
path = Path('data_new.csv')
```

The document was converted with **Code-Format**: https://code-format.com/

Завантажимо дані за допомогою функції `pd.read_csv` ([документація тут](#)).

Зверніть увагу на те, що:

1. файл записано в кодуванні `utf-8`
2. перший рядок файлу містить заголовки, проте під час завантаження слід підставити власні заголовки group, name, var, part1, part2
3. значення в деяких стовпчиках можуть бути відсутні

▶ Підказки.

Функції варто передати параметри, що задають:

1. кодування, в якому записано текстовий файл
2. номер рядка, в якому записано заголовки
3. тип, в якому слід записати в пам'ять останній стовпчик
4. назви стовпчиків

▶ Ще більше підказок

1. параметр sep задає роздільники полів
2. параметр encoding задає кодування
3. параметр header задає номер рядка, в якому записано заголовки (нумерація починається з 0)
4. параметр dtype може бути словником, що задає типи стовпчиків (можливо, не всіх); назві стовпчика відповідає назва типу
5. тип Int64 містить цілі числа та особливе значення NA
6. параметр names задає назви стовпчиків

In [142]:

```
data = pd.read_csv(path, encoding='utf8', sep=",", header=0,
dtype={"lines": "Int64"}, names=['group', 'name', 'var', 'part1',
'part2'])
data.head()
```

	group	name	var	part1	part2
0	K10	Прізвище3 Ім'я3	1	NaN	NaN
1	K10	Прізвище9 Ім'я9	2	NaN	NaN
2	K10	Прізвище16 Ім'я16	3	NaN	NaN
3	K10	Прізвище18 Ім'я18	4	NaN	NaN
4	K10	Прізвище25 Ім'я25	5	NaN	9.0

The document was converted with **Code-Format**: https://code-format.com/

In [143]:

```
data.tail()
```

	group	name	var	part1	part2
126	K14	Прізвище112 Ім'я112	132	NaN	NaN
127	K14	Прізвище117 Ім'я117	133	NaN	NaN
128	K14	Прізвище120 Ім'я120	134	NaN	NaN
129	K14	Прізвище126 Ім'я126	135	NaN	NaN
130	K14	Прізвище127 Ім'я127	136	NaN	NaN

Подивимось інформацію про завантажені дані. Документація на метод `info`.

In [144]:

```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 131 entries, 0 to 130
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   group   131 non-null    str
 1   name    131 non-null    str
 2   var     131 non-null    int64
 3   part1   13 non-null     float64
 4   part2   13 non-null     float64
dtypes: float64(2), int64(1), str(2)
memory usage: 5.2 KB
```

Крок 2 Знайти про скількох студентів наявна інформація у файлах.

▸ Підказка.

Вибрати стовпчик з прізвищами студентів і знайти кількість наявних у ньому значень. Оскільки ПІБ студентів унікальні, то цього буде достатньо.

The document was converted with **Code-Format**: https://code-format.com/

In [145]:

```
data['name'].nunique()
```

```
131
```

Крок 3 Знайти студентів, хто здав обидві частини. Скільки таких студентів?

Цікавить тільки група, ПІБ, номер варіанту, загальна кількість балів.
Тут і далі: здали = отримали бали, можливо нулеві.

▸ Підказка.
Вибрати стовпчик з прізвищами студентів і знайти кількість наявних у ньому значень.
Оскільки ПІБ студентів унікальні, то цього буде достатньо.

3.1 Для рядків таблиці визначити, чи правда що в рядку обидва поля `part1`, `part2` заповнені.
Підказка. Метод `notna` обчислює ознаку наявності значення поля.

▸ Ще підказки

- Вибрати з таблиці стовпчик `part1` та для його елементів з'ясувати чи заповнені вони.
- Виконати аналогічне для стовпчика `part2`.
- Скомбінувати отримані результати (поля обох стовпчиків у рядку мають бути заповнені одночасно).

The document was converted with **Code-Format**: https://code-format.com/

In [146]:

```
ans = data['part1'].notna() & data['part2'].notna()
ans
```

```
0      False
1      False
2      False
3      False
4      False
       ...
126    False
127    False
128    False
129    False
130    False
Length: 131, dtype: bool
```

3.2 Щоб знайти шукану кількість студентів достатньо знайти кількість рядків, для яких обчислена ознака істинна.

▸ Підказка

Логічне значення `True` інтерпретується як `1`. Тому можна просумувати отримані логічні значення. Метод `sum` стане у нагоді.

In [147]:

```
count = ans.sum()
count
```

```
np.int64(12)
```

3.3 Щоб знайти шуканих студентів, слід профільтрувати загальну таблицю за обчисленою ознакою.

Копію (використати метод `copy`) отриманого результату зберегти в змінну `task3`.

Примітка. Надалі до отриманої вибірки будемо додавати стовпчики, тому важливо, щоб вибірка не використовувала дані спільно з первинною таблицею.

The document was converted with **Code-Format**: https://code-format.com/

In [148]:

```
task3 = data[ans].copy()
task3.head()
```

	group	name	var	part1	part2
32	K11	Прізвище0 Ім'я0	38	6.0	9.0
38	K11	Прізвище51 Ім'я51	44	6.0	9.0
46	K11	Прізвище84 Ім'я84	52	6.0	9.0
58	K12	Прізвище12 Ім'я12	64	6.0	9.0
64	K12	Прізвище40 Ім'я40	70	6.0	9.0

3.4 До отриманої таблиці додати суму балів під іменем `total`.

In [149]:

```
task3.insert(loc=5, column='total',
value=task3['part1']+task3['part2'])
task3.head()
```

	group	name	var	part1	part2	total
32	K11	Прізвище0 Ім'я0	38	6.0	9.0	15.0
38	K11	Прізвище51 Ім'я51	44	6.0	9.0	15.0
46	K11	Прізвище84 Ім'я84	52	6.0	9.0	15.0
58	K12	Прізвище12 Ім'я12	64	6.0	9.0	15.0
64	K12	Прізвище40 Ім'я40	70	6.0	9.0	15.0

3.5 З отриманої таблиці вибрати стовпчики `group`, `name`, `var`, `total`.

The document was converted with **Code-Format**: https://code-format.com/

In [150]:

```
task3[['group', 'name', 'var', 'total']]
```

	group	name	var	total
32	K11	Прізвище0 Ім'я0	38	15.0
38	K11	Прізвище51 Ім'я51	44	15.0
46	K11	Прізвище84 Ім'я84	52	15.0
58	K12	Прізвище12 Ім'я12	64	15.0
64	K12	Прізвище40 Ім'я40	70	15.0
88	K13	Прізвище4 Ім'я4	94	15.0
89	K13	Прізвище6 Ім'я6	95	15.0
92	K13	Прізвище24 Ім'я24	98	15.0
95	K13	Прізвище33 Ім'я33	101	15.0
103	K13	Прізвище97 Ім'я97	109	15.0
106	K13	Прізвище107 Ім'я107	112	15.0
119	K14	Прізвище62 Ім'я62	125	15.0

Крок 4 Знайти тих студентів, хто не здав жодної частини. Скільки таких студентів?

Цікавить тільки група, ПІБ, номер варіанту.

▸ Підказки

- Метод `isna` обчислює ознаку відсутності значення поля.
- Далі використати вибірку за умовою, а також вибрати тільки потрібні стовпчики.

The document was converted with **Code-Format**: https://code-format.com/

In [151]:

```
not_ans = data['part1'].isna() & data['part2'].isna()
task31 = data[not_ans].copy()
task31[['group', 'name', 'var']]
```

	group	name	var
0	K10	Прізвище3 Ім'я3	1
1	K10	Прізвище9 Ім'я9	2
2	K10	Прізвище16 Ім'я16	3
3	K10	Прізвище18 Ім'я18	4
5	K10	Прізвище30 Ім'я30	6
...	...	...	...
126	K14	Прізвище112 Ім'я112	132
127	K14	Прізвище117 Ім'я117	133
128	K14	Прізвище120 Ім'я120	134
129	K14	Прізвище126 Ім'я126	135
130	K14	Прізвище127 Ім'я127	136

117 rows × 3 columns

In [152]:

```
count1 = not_ans.sum()
count1
```

```
np.int64(117)
```

Крок 5 Знайти тих студентів, хто здав першу частину.

Цікавить тільки група, ПІБ, номер варіанту, бали за першу частину.

▸ Підказки

- Метод `isna` обчислює ознаку відсутності значення поля.
- Далі використати вибірку за умовою, а також вибрати тільки потрібні стовпчики.

Code converted with CodeFormat.Co, It's free and ad-free. https://code-format.com

In [153]:

```
not_ans1 = data['part1'].notna()
task311 = data[not_ans1].copy()
task311[['group', 'name', 'var', 'part1']]
```

	group	name	var	part1
32	K11	Прізвище0 Ім'я0	38	6.0
38	K11	Прізвище51 Ім'я51	44	6.0
46	K11	Прізвище84 Ім'я84	52	6.0
58	K12	Прізвище12 Ім'я12	64	6.0
64	K12	Прізвище40 Ім'я40	70	6.0
88	K13	Прізвище4 Ім'я4	94	6.0
89	K13	Прізвище6 Ім'я6	95	6.0
92	K13	Прізвище24 Ім'я24	98	6.0
95	K13	Прізвище33 Ім'я33	101	6.0
103	K13	Прізвище97 Ім'я97	109	6.0
106	K13	Прізвище107 Ім'я107	112	6.0
110	K14	Прізвище5 Ім'я5	116	6.0
119	K14	Прізвище62 Ім'я62	125	6.0

Крок 6 Знайти студентів групи K14, що здали хоч одну частину лабораторної. Цікавить уся наявна інформація.

Підказки

- Знайти рядки таблиці, що стосуються студентів групи K14.
- Знайти рядки таблиці, в яких хоча б один зі стовпчиків `part1`, `part2` заповнений.
- Скомбінувати попередні умови: відповідна вибірка буде шуканою.

6.1 Знайти рядки таблиці, що стосуються студентів групи K14.

▸ Підказка

Вибрати рядки таблиці, де значення поля `group` дорівнює K14.

The document was converted with **Code-Format**: https://code-format.com/

In [154]:

```
k14only = (data['group'] == 'K14')
taskK14only = data[k14only].copy()
taskK14only[['group', 'name', 'var', 'part1', 'part2']]
```

	group	name	var	part1	part2
110	K14	Прізвище5 Ім'я5	116	6.0	NaN
111	K14	Прізвище7 Ім'я7	117	NaN	NaN
112	K14	Прізвище10 Ім'я10	118	NaN	NaN
113	K14	Прізвище15 Ім'я15	119	NaN	NaN
114	K14	Прізвище21 Ім'я21	120	NaN	NaN
115	K14	Прізвище34 Ім'я34	121	NaN	NaN
116	K14	Прізвище1 Ім'я1	122	NaN	NaN
117	K14	Прізвище47 Ім'я47	123	NaN	NaN
118	K14	Прізвище56 Ім'я56	124	NaN	NaN
119	K14	Прізвище62 Ім'я62	125	6.0	9.0
120	K14	Прізвище67 Ім'я67	126	NaN	NaN
121	K14	Прізвище77 Ім'я77	127	NaN	NaN
122	K14	Прізвище82 Ім'я82	128	NaN	NaN
123	K14	Прізвище86 Ім'я86	129	NaN	NaN
124	K14	Прізвище98 Ім'я98	130	NaN	NaN
125	K14	Прізвище101 Ім'я101	131	NaN	NaN
126	K14	Прізвище112 Ім'я112	132	NaN	NaN
127	K14	Прізвище117 Ім'я117	133	NaN	NaN
128	K14	Прізвище120 Ім'я120	134	NaN	NaN
129	K14	Прізвище126 Ім'я126	135	NaN	NaN
130	K14	Прізвище127 Ім'я127	136	NaN	NaN

The document was converted with **Code-Format**: https://code-format.com/

6.2 Знайти рядки таблиці, в яких хоча б один зі стовпчиків `part1`, `part2` заповнений.

▸ Підказки

- Вибрати з таблиці стовпчики `part1`, `part2`.
- В отриманій вибірці для клітин таблиці з'ясувати чи заповнені вони. Метод `notna` стане у нагоді.
- Далі слід знайти, чи наявні в рядках істинні значення:
  можна явно записати умову з використанням АБО;
  можна використати метод `any`. Не забудьте врахувати, що перевіряються поля рядка, а не стовпчика. Тому слід правильно вказати вісь, уздовж якої виконується перевірка.

In [155]:

```
ans62 = (data['part1'].notna()) | (data['part2'].notna())
task62 = data[ans62].copy()
task62[['group', 'name', 'var', 'part1', 'part2']]
```

	group	name	var	part1	part2
4	K10	Прізвище25 Ім'я25	5	NaN	9.0
32	K11	Прізвище0 Ім'я0	38	6.0	9.0
38	K11	Прізвище51 Ім'я51	44	6.0	9.0
46	K11	Прізвище84 Ім'я84	52	6.0	9.0
58	K12	Прізвище12 Ім'я12	64	6.0	9.0
64	K12	Прізвище40 Ім'я40	70	6.0	9.0
88	K13	Прізвище4 Ім'я4	94	6.0	9.0
89	K13	Прізвище6 Ім'я6	95	6.0	9.0
92	K13	Прізвище24 Ім'я24	98	6.0	9.0
95	K13	Прізвище33 Ім'я33	101	6.0	9.0
103	K13	Прізвище97 Ім'я97	109	6.0	9.0
106	K13	Прізвище107 Ім'я107	112	6.0	9.0
110	K14	Прізвище5 Ім'я5	116	6.0	NaN
119	K14	Прізвище62 Ім'я62	125	6.0	9.0

6.3 Скомбінувати попередні умови: відповідна вибірка буде шуканою.

This document was converted with Code-Format https://code-format.com

In [156]:

```
k14 = (data['group'] == 'K14')
k14ans = (data['part1'].notna()) | (data['part2'].notna())
taskK14 = data[k14 & k14ans].copy()
taskK14[['group', 'name', 'var', 'part1', 'part2']]
```

	group	name	var	part1	part2
110	K14	Прізвище5 Ім'я5	116	6.0	NaN
119	K14	Прізвище62 Ім'я62	125	6.0	9.0

Крок 7 Для кожної групи знайти кількість студентів та кількість студентів, що щось здали.

7.1 Обчислити для кожного студента ознаку, чи здав він хоча б одну з частин, та додати відповідний стовпчик до таблиці. Назвати доданий стовпчик `done`.

In [157]:

```
data.insert(5, 'done', data['part1'].notna() | data['part2'].notna())
data.head()
```

	group	name	var	part1	part2	done
0	K10	Прізвище3 Ім'я3	1	NaN	NaN	False
1	K10	Прізвище9 Ім'я9	2	NaN	NaN	False
2	K10	Прізвище16 Ім'я16	3	NaN	NaN	False
3	K10	Прізвище18 Ім'я18	4	NaN	NaN	False
4	K10	Прізвище25 Ім'я25	5	NaN	9.0	True

7.2 З використанням методу `groupby` виконати групування по групах.

Далі за допомогою методу `agg` для кожної групи для стовпчика `done` обчислити кількість (`count`) та суму (`sum`) значень. Це буде кількість студентів групи та кількість студентів групи, що щось здали.
Результат присвоїти змінній `res`

The document was converted with **Code-Format**: https://code-format.com/

In [167]:

```
res = data.groupby('group')['done'].agg(['count',
'sum']).reset_index()
res
```

	group	count	sum
0	K10	26	1
1	K11	31	3
2	K12	31	2
3	K13	22	6
4	K14	21	2

Перейменуємо заголовки колонок.

In [168]:

```
res.columns = ('group', 'number', 'works')
res
```

	group	number	works
0	K10	26	1
1	K11	31	3
2	K12	31	2
3	K13	22	6
4	K14	21	2

Розрахувати кількість студентів, що нічого не здали. Результат обчислення записати в стовпчик `lazy`.

The document was converted with **Code-Format**: https://code-format.com/

In [170]:

```
res['lazy'] = res['number'] - res['works']
res
```

	group	number	works	lazy
0	K10	26	1	25
1	K11	31	3	28
2	K12	31	2	29
3	K13	22	6	16
4	K14	21	2	19

Частину 3 зроблено!

The document was converted with **Code-Format**: https://code-format.com/

LAB2 part4 Оцінка параметрів функції (робочий зошит)
Exported 2/28/2026, 2:29:26 PM

LAB2 Частина 4. Оцінка параметрів функції

Виконавець: **Базилевич Олексій**, група **К27**
Викладач: **Андрій Ляшко**

Знадобляться бібліотеки `numpy`, `scipy`, `pandas`, `matplolib`.

In [1]:

```
from pathlib import Path
import numpy as np
import scipy, scipy.linalg as la
import pandas as pd
import matplotlib, matplotlib.pyplot as plt
print(f'numpy {np.__version__}, pandas {pd.__version__}, scipy
{scipy.__version__}, matplotlib {matplotlib.__version__}')
```

```
numpy 2.4.2, pandas 3.0.1, scipy 1.17.1, matplotlib 3.10.8
```

Постановка задачі

У результаті кількох експериментів з обчислення швидкодії програми була отримана низка даних вигляду (n, t), де
n — розмір оброблюваних вхідних даних,
t — кількість часу, що була в середньому необхідна для їх обробки.

Дослідник знає, що часова складність програми повинна мати вигляд $T(n) = \Theta(n^a)$. Знайти значення параметра a.

Зауваження. Якщо $T(n) = \Theta(n^a)$, то можна вважати, що $T(n) \approx cn^a$.

The document was converted with **Code-Format**: https://code-format.com/

На відміну від частини 1, у цій задачі результати експериментів задані у csv-файлі. Кожен рядок файлу містить розмір оброблюваних даних та час роботи на них. Для деяких розмірів виконували кілька експериментів, тому розміри можуть повторюватись. Під час оцінювання параметрів для кожного наявного розміру слід використовувати середнє значення часу роботи.

In [2]:

```
path = Path('lab2_var099.csv')
```

Завантажити дані в DataFrame за допомогою функції `pd.read_csv` ([документація тут](#)).

In [3]:

```
raw = pd.read_csv(path)
```

Виконати необхідні перетворення даних засобами `pandas`

In [4]:

```
df_mean = raw.groupby('n', as_index=False).mean()
df_mean.insert(0, 'ones', 1.0)
df_mean['ln_n'] = np.log(df_mean['n'])
df_mean['ln_t'] = np.log(df_mean['t'])
df_data = df_mean[['ones', 'ln_n', 'ln_t']]
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

Побудувати матрицю $data$ (див. LSM.pdf). Має бути багатовимірним масивом типу `ndarray`.

▶ Підказка.

У класі `DataFrame` є метод `to_numpy`

The document was converted with **Code-Format**: https://code-format.com/

In [5]:

```
data = df_data.to_numpy()
data[:5]
```

```
array([[1.        , 2.30258509, 3.29583687],
       [1.        , 2.48490665, 3.40119738],
       [1.        , 3.91202301, 5.3099165 ],
       [1.        , 4.12713439, 5.72684775],
       [1.        , 4.20469262, 5.81039221]])
```

In [10]:

In [11]:

In [ ]:

In [ ]:

Знайти оцінки параметрів a, c функції $T(n) \approx cn^a$.

In [6]:

```
res = la.lstsq(data[:, :2], data[:, 2])[0]
a = res[1]
c = np.exp(res[0])
```

In [7]:

```
print(f'{c = :.6f}, {a = :.6f}')
```

```
c = 1.195303, a = 1.325032
```

Візуальна оцінка якості наближення

Для цього побудуємо графік знайденої функції та накладемо на нього вхідні дані.

The document was converted with **Code-Format**: https://code-format.com/

Побудувати функцію `T_estimated`, що обчислює час роботи програми за формулою $T(n) = cn^a$ з використанням знайдених оцінок `a`, `c` параметрів a та c.

In [8]:

```
def T_estimated(n):
    return c * (n ** a)
```

Для візуалізації будемо будувати графік поточково, тому потрібні значення в проміжних точках.

Почнемо з того, що визначимо точки, які використаємо в поточковій побудові графіку. Отриманий масив точок, які рівномірно розташовані по всій шкалі, запам'ятаємо в змінній `scale`.

In [9]:

```
scale = np.linspace(df_mean['n'].min(), df_mean['n'].max(), 100)
```

З використанням функції `T_estimated` обчислимо розрахункові значення функції часової складності на всій шкалі і запам'ятаємо в змінній `times`.

▸ Підказка.
Функція `T_estimated` може бути застосована до масиву `scale` безпосередньо.

In [10]:

```
times = T_estimated(scale)
```

Візуалізуємо отримані результати.

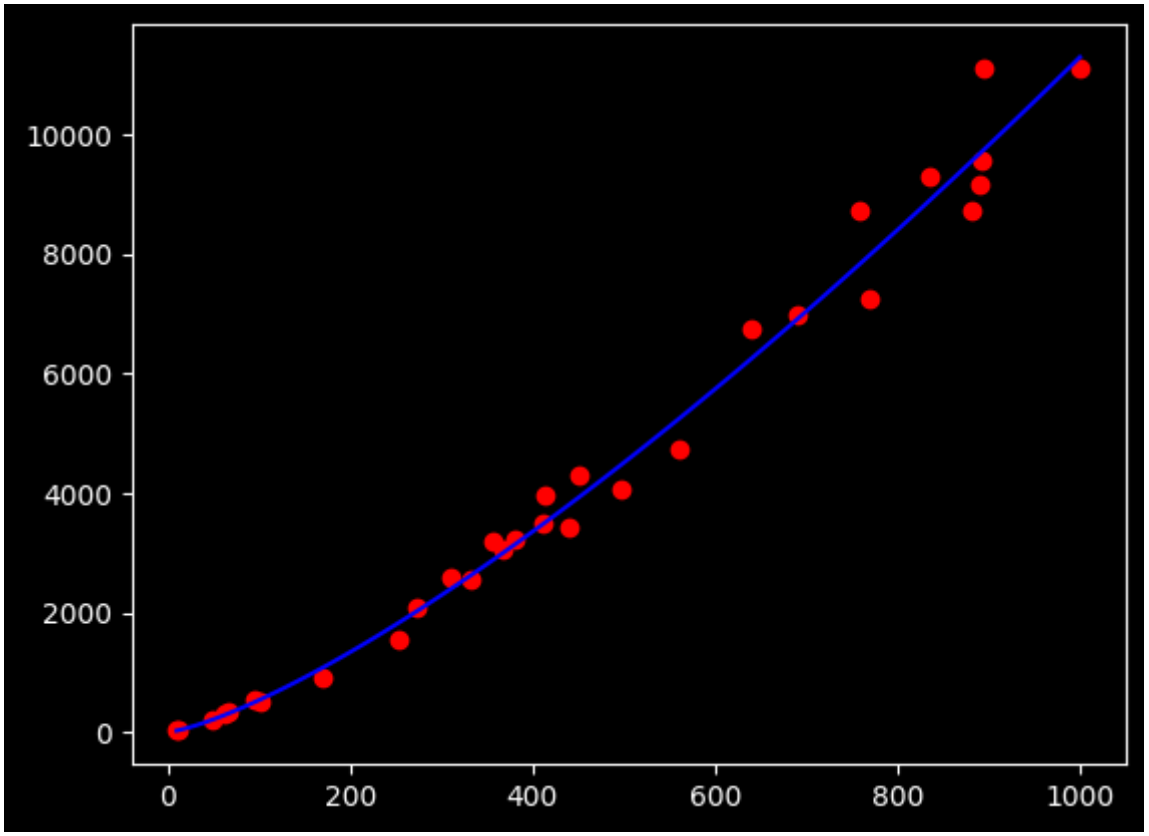
На графіку червоні точки відповідають "експериментальним" даним. Синім зображено графік отриманої залежності часу роботи від розмірів даних.

n — масив вхідних розмірів
t — масив, що містить значення часу роботи на даних відповідного розміру
(зміст цих змінних такий самий, що і в частині 1)

The document was converted with **Code-Format**: https://code-format.com/

In [11]:

```
n = df_mean['n']
t = df_mean['t']
plt.plot(n, t, color="red", marker='o', linestyle='None')
plt.plot(scale, times, color='blue')
```



Частину 4 зроблено!

The document was converted with **Code-Format**: https://code-format.com/

Лабораторна робота № 3
варіант № 4

Лабораторна робота №3
Номер варіанту отримується за формулою
(номер варіанту для лабораторної роботи №2) %24 +1.
Складається з двох задач.
Обидві мають бути виконані виключно програмно, у блокноті. Ручне збирання даних неприпустиме і оцінюється як 0 балів. Використання API сайту також неприпустиме!
За виконаним з чистого аркушем блокнотом створюється пдф-звіт. Пдф має відповідати блокноту (вимога критична).
Для кожної задачі здається: блокнот та звіт.
У блокноті обов'язково має бути зазначена інформація про виконавця (ПІБ, група, викладач), варіант та умова задачі. (Це критична вимога, порушення якої занулить бали.)

Задача 1 : 8 балів максимум. Задача 2 : 7 балів максимум.
Неохайний блокнот (або пдф-звіт) (виконаний не з чистого аркушу, невиконаний, є винятки) : -2 бали за кожен неохайний.
Має бути вся запитувана інформація.

Задача 1
За даними сайту https://www.numbeo.com/

Знайти країну де найдешевше та найдорожче

1) молоко
2) рис
3) яйця
4) сир
5) куряче філе
6) м'ясо
7) яблука
8) банани
9) апельсини
10) томати
11) картопля
12) цибуля
13) салат
14) бутильована вода
15) 1 поїздка на громадському транспорті
16) 1 литр бензину
17) 1 км на таксі

Надати інформацію про: кількість наявних даних, найменшу та найбільшу вартість (в EUR) та на якому місті вони досягаються

18) Знайти країну, в якій найдорожче та найдешевше харчування на 1 місяць (західне меню)
19) Знайти країну, в якій найдорожче та найдешевше харчування на 1 місяць (азіатське меню)
18+19: надати інформацію де та на якому меню досягається.

20) Знайти країну, в якій навищий та найменший індекс злочинності (Level of crime)
21) Знайти країну, в якій найвищий та найменший показник безпеки (прогулянки вдень та прогулянки вночі)
20+21: Надати інформацію на якій країні досягається + усю інформацію щодо різних типів злочинності у країні.

22) Знайти країну з найвищим та найнижчим рівнями забруднення води та повітря. Надати інформацію на якій країні досягається + усю інформацію щодо показників.

23) Знайти місто Англії, в якому найдешевше/ найдорожче молоко.
24) Знайти місто Франції, в якому найдешевше/ найдорожче білий хліб.
23+24: надати інформацію на якому місті досягається + інформацію по країні.

Задача 2
За даними сайту https://cppreference.com з'ясувати:
Member types, Member functions, Non-member functions.
За наявності враховувати підрозбиття на категорії.
Вивести: відповідні переліки через кому.
Також з'ясувати скільки перелічено конструкторів. Для кожного з'ясувати стандарти, в яких він існує (та вивести). Вивести конструктори, що існують в стандарті C++23.

1) vector
2) array
3) list
4) deque
5) forward_list
6) set
7) map
8) multuset
9) multimap
10) unordered_set
11) unordered_map
12) unordered_multuset
13) unordered_multimap
14) stack
15) queue
16) priority_queue
17) pair
18) optional
19) variant
20) basic_string
21) unique_ptr
22) shared_ptr
23) weak_ptr
24) any

Lab3_1

Exported 3/15/2026, 1:08:40 AM

Виконавець: **Базилевич Олексій Андрійович**, група **К-27**
Викладач: **Андрій Ляшко** Частина 1

Варіант 4 (Local Cheese)

Знайти країну де найдешевше та найдорожче

Надати інформацію про: кількість наявних даних, найменшу та найбільшу вартість (в EUR) та на якому місті вони досягаються

In [1]:

```
from pathlib import Path
import io
import json
import pandas as pd
import sys
import requests
import bs4
from bs4 import BeautifulSoup as BSoup, SoupStrainer, BeautifulSoup
import html5lib
```

In [4]:

```
url = "https://www.numbeo.com/cost-of-living/prices_by_country.jsp?
displayCurrency=EUR&itemId=12"
r = requests.get(url)
r.status_code
```

```
200
```

In [6]:

```
fname = Path('data.html')
cheese_data = r.text
with open(fname, "w", encoding="utf-8") as f:
    f.write(cheese_data)
```

The document was converted with **Code-Format**: https://code-format.com/

In [8]:

```
with open(fname,"r" ,encoding="utf-8") as f:
    data = f.read()
data
```

The document was converted with **Code-Format**: https://code-format.com/

```
'<!DOCTYPE html>\n<html lang="en-US">\n<head>\n<meta http-
equiv="Content-Type" content="text/html; charset=UTF-8">\n\n<meta
name="description" content="See current prices by country for prices
of items we do track. You can see prices only for countries where we
have decent number of contributors." />\n            <script defer
src="https://www.numbeo.com/common/numbeo_standard.js"></
script>\n<link rel="preconnect" href="https://
fonts.googleapis.com">\n<link rel="preconnect" href="https://
fonts.gstatic.com" crossorigin>\n<link href="https://
fonts.googleapis.com/css2?
family=Lato:ital,wght@0,100;0,300;0,400;0,700;0,900;1,100;1,300;1,400;
1,700;1,900&display=swap" rel="stylesheet">\n    <link href="https://
cdnjs.cloudflare.com/ajax/libs/jqueryui/1.10.3/themes/redmond/jquery-
ui.min.css" rel="Stylesheet" />\n    <link rel="shortcut icon"
type="image/x-icon" href="https://www.numbeo.com/favicon.ico" />\n
<script  defer  src="https://cdnjs.cloudflare.com/ajax/libs/
jquery/1.10.2/jquery.min.js"></script>\n    <script  defer
src="https://cdnjs.cloudflare.com/ajax/libs/jqueryui/1.10.3/jquery-
ui.min.js"></script>\n   <script src=\'https://www.gstatic.com/charts/
loader.js\'></script>\n<link rel="apple-touch-icon" href="https://
www.numbeo.com/images/apple-touch-icon.png">\n  <script>\n  if (typeof
whenDocReady === "function") {\n    // already declared, do nothing\n
} else {\n    function whenDocReady(fn) {\n      // see if DOM is
already available\n      if (document.readyState === "complete" ||
document.readyState === "interactive") {\n        // call on next
available tick\n        setTimeout(fn, 1);\n      } else {\n
document.addEventListener("DOMContentLoaded", fn);\n      }\n    }\n
}\n</script>\n<!-- Global site tag (gtag.js) - Google Analytics --
>\n<script async src="https://www.googletagmanager.com/gtag/js?id=G-
HXEZG0C6Q2"></script>\n<script>\n  window.dataLayer = window.dataLayer
|| [];\n  function gtag(){dataLayer.push(arguments);}\n  gtag(\'js\',
new Date());\ngtag(\'config\', \'G-HXEZG0C6Q2\');\ngtag(\'set\',
\'user_properties\', { ab: \'0\'});\n</script>\n<link rel="stylesheet"
href="https://cdn.datatables.net/v/ju/dt-1.10.22/datatables.min.css"/
>\n<script defer src="https://cdn.datatables.net/v/ju/dt-1.10.22/
datatables.min.js"></script>\n<script async src="https://
pagead2.googlesyndication.com/pagead/js/adsbygoogle.js?client=ca-
pub-8993465169534557"\n     crossorigin="anonymous"></script>\n\n\n\n
<link rel="stylesheet" href="https://www.numbeo.com/common/
style_desktop.css" />\n\n  <script>\n\n    function checkControls()
{\n\n      enableIfElementsHaveSomeValue("calculate", /* to check */
"itemId", "displayCurrency");\n\n    }\n\n  </script>\n\n<link
rel="canonical" href="https://www.numbeo.com/cost-of-living/
prices_by_country.jsp?itemId=12" />\n  <title>Prices by Country of
Local Cheese (1 kg) (Markets)</title>\n\n</
head>\n\n<body>\n\n<nav>\n<div class="logo-and-upper-ad">\n<div
class="innerWidth" id="content_and_logo">\n<table style="width: 100%;"
```

The document was converted with **Code-Format**: https://code-format.com/

```
>\n<tr>\n<td style="padding-top: 14px; padding-bottom: 16px; vertical-
align: bottom; max-width: fit-content"
class="container_around_logo_desktop">\n<a href="https://
www.numbeo.com/cost-of-living/" style="border: 0; text-
decoration:none;">\n<img src="https://www.numbeo.com/images/
numbeo_1B3B6F.svg" style="height: 60px; padding: 0px 0px; vertical-
align: bottom; border: 0;" alt="" /></a>\n</td>\n<td style="padding-
left:40px; padding-top: 5px; padding-bottom: 0px; vertical-align:
middle; min-width: 440px; text-align: left;" >\n<div style="display:
inline-block; width: 100%; margin-top: 10px; margin-bottom: 10px;
text-align: left; vertical-align: bottom">\n<nav style="display:
inline-block; vertical-align: middle;">\n<div
class="standard_margin_inlineblock" >      <form action="https://
www.numbeo.com/common/dispatcher.jsp" method="get"
id="menu_dispatch_form"  style="display: inline; padding: 0;
margin:0;">\n      <input type="hidden" name="where" value="https://
www.numbeo.com/cost-of-living/in/" style="display: inline; padding:0;
margin:0;" />\n<input id="city_selector_menu_city_id"  size="27"
type="text" placeholder="Select City" class="city-selector-menu"
name="city_selector_menu_city_id" />\n<input type="hidden"
id="menu_city_id" name="city_id"/>\n<input type="hidden"
id="name_menu_city_id" name="name_city_id"/>\n      </form>\n</div>\n
<script>\nwhenDocReady(function() {\n
\tjQuery(\'#city_selector_menu_city_id\').autocomplete({\n  \t
source: \'https://www.numbeo.com/common/CitySearchJson\',\n
minLength: 1,\n     delay: 3,\n     autoFocus: true,\n     select:
function(event, ui) {\n         event.preventDefault();\n
$(\'#name_menu_city_id\').val(ui.item.label);\n
$(\'#city_selector_menu_city_id\').val(ui.item.label);\n
$(\'#menu_city_id\').val(ui.item.value);
\njQuery(\'#menu_dispatch_form\').submit();     },\n     focus:
function(event, ui) {\n         event.preventDefault();\n     }\n\t
})   .keydown(function(e){\n\t    if (e.keyCode === 13){\n      if
($(\'#menu_city_id\').val() == \'\') { \n        e.preventDefault();\n
}\n      var val = $(\'#city_selector_menu_city_id\').val();\n\t\t
$(\'#name_menu_city_id\').val(val);\n\t   }\n  }); // end
autocompleter
definition\n$(\'#city_selector_menu_city_id\').toggleClass(\'city-
selector-menu\');
\n$(\'#city_selector_menu_city_id\').toggleClass(\'city-selector-
menu\');\n }); // end document ready \n</script>\n</nav>\n</div>\n</
td>\n<td style="vertical-align: middle; width: 468px; text-align:
right;" >\n</td>\n</tr>\n</table>\n</div> <!-- innerWidth in header --
>\n</div> <!-- logo-and-upper-ad -->\n</nav>\n<div
class="innerWidth">\n<div class="dropdown-outer">\n<nav class="before-
nav" >\n    <ul id="nav-one" class="nav">\n\n\t<li class="dir">\n<a
href="https://www.numbeo.com/cost-of-living/">Cost Of Living</a>\n
<ul>\n    <li><a href="https://www.numbeo.com/cost-of-living/">Cost Of
```

This document was created using Code Format https://code-format.com

```
Living</a></li>\n    <li><a href="https://www.numbeo.com/cost-of-
living/comparison.jsp">Cost of Living Comparison</a></li>\n    <li><a
href="https://www.numbeo.com/cost-of-living/calculator.jsp">Cost of
Living Calculator</a></li>\n    <li><a href="https://www.numbeo.com/
cost-of-living/rankings_current.jsp">Cost Of Living Index (Current)</
a></li>\n    <li><a href="https://www.numbeo.com/cost-of-living/
rankings.jsp">Cost Of Living Index</a></li>\n    <li><a href="https://
www.numbeo.com/cost-of-living/rankings_by_country.jsp">Cost Of Living
Index By Country</a></li>\n    <li><a href="https://www.numbeo.com/
food-prices/">Food Prices</a></li>\n    <li><a href="https://
www.numbeo.com/cost-of-living/prices_by_city.jsp">Prices By City</a></
li>\n    <li><a href="https://www.numbeo.com/cost-of-living/
prices_by_country.jsp">Prices By Country</a></li>\n    <li><a
href="https://www.numbeo.com/cost-of-living/
basket_of_goods.jsp">Basket of Goods and Services</a></li>\n    <li><a
href="https://www.numbeo.com/taxi-fare/">Taxi Fare Calculator</a></
li>\n    <li><a href="https://www.numbeo.com/gas-prices/">Gas Prices
Calculator</a></li>\n    <li><a href="https://www.numbeo.com/common/
motivation_and_methodology.jsp">Methodology and Motivation</a></li>\n
<li><a href="https://www.numbeo.com/salary-calculator/">Salary
Calculator</a></li>\n    <li><a href="https://www.numbeo.com/cost-of-
living/contribute.jsp">Update Data for Your City</a></li>\n  </ul></
li>\n\n\t<li class="dir">\n<a href="https://www.numbeo.com/property-
investment/">Property Prices</a>\n  <ul>\n    <li><a href="https://
www.numbeo.com/property-investment/">Property Prices</a></li>\n
<li><a href="https://www.numbeo.com/property-investment/
comparison.jsp">Property Prices Comparison</a></li>\n    <li><a
href="https://www.numbeo.com/property-investment/
rankings_current.jsp">Property Price Index (Current)</a></li>\n
<li><a href="https://www.numbeo.com/property-investment/
rankings.jsp">Property Price Index</a></li>\n    <li><a href="https://
www.numbeo.com/property-investment/rankings_by_country.jsp">Property
Price Index By Country</a></li>\n   </ul></li>\n\n\t<li
class="dir">\n<a href="https://www.numbeo.com/quality-of-
life/">Quality Of Life</a>\n  <ul>\n    <li><a href="https://
www.numbeo.com/quality-of-life/">Quality of Life</a></li>\n    <li><a
href="https://www.numbeo.com/quality-of-life/comparison.jsp">Quality
of Life Comparisons</a></li>\n    <li><a href="https://www.numbeo.com/
quality-of-life/rankings_current.jsp">Quality of Life Index
(Current)</a></li>\n    <li><a href="https://www.numbeo.com/quality-
of-life/rankings.jsp">Quality of Life Index</a></li>\n    <li><a
href="https://www.numbeo.com/quality-of-life/
rankings_by_country.jsp">Quality of Life Index by Country</a></li>\n
<li><a href="https://www.numbeo.com/crime/">Crime</a></li>\n    <li><a
href="https://www.numbeo.com/crime/rankings_current.jsp">Crime Index
(Current)</a></li>\n    <li><a href="https://www.numbeo.com/crime/
rankings.jsp">Crime Index</a></li>\n    <li><a href="https://
www.numbeo.com/crime/rankings_by_country.jsp">Crime Index by Country</
a></li>\n    <li><a href="https://www.numbeo.com/health-care/">Health
```

The document was converted with **Code-Format**: https://code-format.com/

Care</a></li>\n   <li><a href="https://www.numbeo.com/health-care/rankings_current.jsp">Health Care Index (Current)</a></li>\n   <li><a href="https://www.numbeo.com/health-care/rankings.jsp">Health Care Index</a></li>\n   <li><a href="https://www.numbeo.com/health-care/rankings_by_country.jsp">Health Care Index by Country</a></li>\n <li><a href="https://www.numbeo.com/pollution/">Pollution</a></li>\n <li><a href="https://www.numbeo.com/pollution/rankings_current.jsp">Pollution Index (Current)</a></li>\n   <li><a href="https://www.numbeo.com/pollution/rankings.jsp">Pollution Index</a></li>\n   <li><a href="https://www.numbeo.com/pollution/rankings_by_country.jsp">Pollution Index by Country</a></li>\n <li><a href="https://www.numbeo.com/traffic/">Traffic</a></li>\n <li><a href="https://www.numbeo.com/traffic/rankings_current.jsp">Traffic Index (Current)</a></li>\n   <li><a href="https://www.numbeo.com/traffic/rankings.jsp">Traffic Index</a></li>\n   <li><a href="https://www.numbeo.com/traffic/rankings_by_country.jsp">Traffic Index by Country</a></li>\n\t</ul></li>\n\t<li class="dir">\n<a href="https://www.numbeo.com/premium/index">Premium</a>\n <ul>\n   <li><a href="https://www.numbeo.com/common/api.jsp">Data API</a></li>\n   <li><a href="https://www.numbeo.com/premium/commercial-license">Data License</a></li>\n <li><a href="https://www.numbeo.com/premium/download-data">Bulk Data Download</a></li>\n   <li><a href="https://www.numbeo.com/cost-of-living/historical-data-analysis">Historical Data Explorer</a></li>\n <li><a href="https://www.numbeo.com/cost-of-living/estimator_main">Cost of Living Estimator</a></li>\n   <li><a href="https://www.numbeo.com/premium/">Insight Reports</a></li>\n <li><a href="https://www.numbeo.com/premium/international-salary-equivalent-calculator">Global Salary Equivalent Calculator</a></li>\n <li><a href="https://www.numbeo.com/premium/relocation-salary-calculator">Relocation Salary Calculator</a></li>\n\t</ul></li>\n\n\n</ul>\n<br style="clear: both"/>\n</nav>\n <!--dropdown-outer -->\n <!-- innerWidth -->\n \n \n \n \n \n<p>\n<nav itemscope itemtype="http://schema.org/BreadcrumbList" class="breadcrumb">\n<a href="https://www.numbeo.com/common/" class="discreet_link"><svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" class="image_without_border hide_smaller_than_600" style="vertical-align: text-bottom" viewBox="0 0 24 24"><path fill="#1B3B6F" d="M10 20v-6h4v6h5v-8h3L12 3L2 12h3v8z"/></svg></a><span itemprop="itemListElement" itemscope itemtype="http://schema.org/ListItem">\n<a href="https://www.numbeo.com/cost-of-living/" itemprop="item" class="breadcrumb_link">Cost of Living</a><meta itemprop="position" content="1" />\n > <span itemprop="itemListElement" itemscope itemtype="http://schema.org/ListItem">\n <a href="https://www.numbeo.com/cost-of-living/

The document was converted with CodeFormat https://code-format.com/

```
prices_by_country.jsp"  itemprop="item"  class="breadcrumb_link"><span
itemprop="name">Prices by Country</span></a><meta itemprop="position"
content="2" />\n</span> <span style="font-size: 75%; display: inline-
block;">&gt;</span> <span itemprop="itemListElement" itemscope
itemtype="http://schema.org/ListItem">\n<a href="https://
www.numbeo.com/cost-of-living/prices_by_country.jsp?itemId=12"
itemprop="item"  class="breadcrumb_link">\n<span itemprop="name">Local
Cheese (1 kg) (Markets)</span></a><meta itemprop="position"
content="3" />\n</span></nav>\n    <h1> Prices by Country of Local
Cheese (1 kg) (Markets)</h1>\n\n\n<!-- Column 1 start -->\n\n\n<div
id="page_container">\n\nThis page displays current price data by
country.\n<br/>Data are displayed only for countries with a sufficient
number of contributors.\n<br/>\n\n<div class="fancy_explanation"
style="margin-bottom: 0.8em; width: fit-content;">\n<a href="https://
www.numbeo.com/cost-of-living/country_price_rankings?
displayCurrency=EUR&amp;itemId=12">See bar chart of these data</a></
div>\n\n<div class="regionChartDivContainer">\n<div id="chart_div_12"
class="regionChartDiv" ></div><div
class="regionChartDivLabel">\nChart: Local Cheese (1 kg), Markets</
div>\n</div>\n <script>\n     google.load(\'visualization\', \'1\',
{\'packages\': [\'geochart\']});\n
google.setOnLoadCallback(drawRegionsMap_12);\n\n      function
drawRegionsMap_12() {\n        var data = new
google.visualization.DataTable();\n        data.addColumn(\'string\',
\'Country\');\n        data.addColumn(\'number\', \'Local Cheese (1
kg), Markets\');\n        data.addRows([\n        [\'Switzerland\',
23.56],\n        [\'Hong Kong (China)\', 19.71],\n
[\'Luxembourg\', 18.54],\n        [\'Austria\', 17.42],\n
[\'France\', 17.22],\n        [\'Singapore\', 15.72],\n
[\'Israel\', 15.69],\n        [\'Taiwan\', 15.58],\n
[\'Italy\', 14.97],\n        [\'Thailand\', 14.95],\n
[\'Belgium\', 13.53],\n        [\'Slovenia\', 13.10],\n
[\'Denmark\', 12.98],\n        [\'Germany\', 12.87],\n
[\'Spain\', 12.42],\n        [\'Slovakia\', 12.41],\n
[\'Netherlands\', 12.39],\n        [\'Greece\', 12.05],\n
[\'Malaysia\', 12.05],\n        [\'Uruguay\', 11.85],\n
[\'Cyprus\', 11.81],\n        [\'United States\', 11.34],\n
[\'China\', 11.27],\n        [\'South Korea\', 11.20],\n
[\'Norway\', 11.18],\n        [\'Czech Republic\', 11.10],\n
[\'Sweden\', 10.80],\n        [\'Croatia\', 10.65],\n
[\'Albania\', 10.64],\n        [\'Sri Lanka\', 10.54],\n
[\'Estonia\', 10.44],\n        [\'Lithuania\', 10.24],\n
[\'Portugal\', 10.24],\n        [\'Panama\', 10.17],\n
[\'Vietnam\', 10.04],\n        [\'Ireland\', 9.85],\n
[\'Russia\', 9.67],\n        [\'Canada\', 9.51],\n
[\'Tunisia\', 9.50],\n        [\'Latvia\', 9.49],\n        [\'Japan\',
9.44],\n        [\'Malta\', 9.43],\n        [\'United Arab Emirates\',
9.43],\n        [\'Chile\', 9.41],\n        [\'Bulgaria\', 9.38],\n
[\'Qatar\', 9.29],\n        [\'Puerto Rico\', 9.17],\n
```

The document was converted with **Code-Format**: https://code-format.com/

```
[\'Zimbabwe\', 9.16],\n        [\'Morocco\', 9.13],\n
[\'Kenya\', 8.70],\n        [\'Hungary\', 8.57],\n        [\'Serbia\',
8.57],\n        [\'Australia\', 8.57],\n        [\'Argentina\', 8.52],
\n        [\'Kazakhstan\', 8.50],\n        [\'Bahrain\', 8.49],\n
[\'United Kingdom\', 8.32],\n        [\'Turkey\', 8.24],\n
[\'Costa Rica\', 8.22],\n        [\'Moldova\', 8.16],\n
[\'Bosnia And Herzegovina\', 8.04],\n        [\'Mexico\', 8.03],\n
[\'Mauritius\', 7.99],\n        [\'Montenegro\', 7.99],\n
[\'Poland\', 7.97],\n        [\'Brazil\', 7.85],\n
[\'Lebanon\', 7.68],\n        [\'North Macedonia\', 7.68],\n
[\'Algeria\', 7.67],\n        [\'Finland\', 7.67],\n
[\'Ukraine\', 7.65],\n        [\'Romania\', 7.65],\n
[\'Indonesia\', 7.45],\n        [\'Dominican Republic\', 7.17],\n
[\'New Zealand\', 7.11],\n        [\'South Africa\', 7.05],\n
[\'Peru\', 7.03],\n        [\'Uzbekistan\', 7.01],\n
[\'Bangladesh\', 6.96],\n        [\'Belarus\', 6.73],\n
[\'Philippines\', 6.54],\n        [\'Saudi Arabia\', 6.47],\n
[\'Armenia\', 6.46],\n        [\'Georgia\', 6.39],\n        [\'Oman\',
6.23],\n        [\'Nepal\', 6.10],\n        [\'Kosovo (Disputed
Territory)\', 5.80],\n        [\'Venezuela\', 5.67],\n
[\'Jordan\', 5.59],\n        [\'Ecuador\', 5.41],\n
[\'Kuwait\', 5.24],\n        [\'Pakistan\', 5.10],\n        [\'Cuba\',
5.03],\n        [\'Nigeria\', 4.99],\n        [\'Colombia\', 4.72],\n
[\'Bolivia\', 4.32],\n        [\'India\', 4.14],\n        [\'Iraq\',
4.12],\n        [\'Azerbaijan\', 3.83],\n        [\'Iran\', 3.43],\n
[\'Egypt\', 2.98],\n        ]);\n\n        var options = {\n
colorAxis: {colors: [ \'#1a9641\', \'#a6d96a\', \'#ffffbf\',
\'#fdae61\', \'#d7191c\']}\n        };\n\n        var chart = new
google.visualization.GeoChart(document.getElementById(\'chart_div_12\'
));\n        chart.draw(data, options);\n    };\n    </
script>\n\n\n<form action="prices_by_country.jsp" method="get"
class="standard_margin">\n<input type="hidden" name="itemId"
value="12" />\nCurrency:          <select id="displayCurrency"
name="displayCurrency" onchange="if
(checkSelectedSomeTextNotDashed(\'displayCurrency\'))
this.form.submit();">\n             <option value="EUR"
selected="selected">EUR</option>\n             <option
value="USD">USD</option>\n             <option value="---">---</
option>\n             <option value="AED">AED</option>\n
<option value="AFN">AFN</option>\n             <option
value="ALL">ALL</option>\n             <option value="AMD">AMD</
option>\n             <option value="ANG">ANG</option>\n
<option value="AOA">AOA</option>\n             <option
value="ARS">ARS</option>\n             <option value="AUD">AUD</
option>\n             <option value="AWG">AWG</option>\n
<option value="AZN">AZN</option>\n             <option
value="BAM">BAM</option>\n             <option value="BBD">BBD</
option>\n             <option value="BDT">BDT</option>\n
```

The document was converted with Code Format! https://code-format.com/

```
<option value="BGN">BGN</option>\n            <option
value="BHD">BHD</option>\n            <option value="BIF">BIF</
option>\n            <option value="BMD">BMD</option>\n
<option value="BND">BND</option>\n            <option
value="BOB">BOB</option>\n            <option value="BRL">BRL</
option>\n            <option value="BSD">BSD</option>\n
<option value="BTC">BTC</option>\n            <option
value="BTN">BTN</option>\n            <option value="BWP">BWP</
option>\n            <option value="BYN">BYN</option>\n
<option value="BZD">BZD</option>\n            <option
value="CAD">CAD</option>\n            <option value="CDF">CDF</
option>\n            <option value="CHF">CHF</option>\n
<option value="CLF">CLF</option>\n            <option
value="CLP">CLP</option>\n            <option value="CNH">CNH</
option>\n            <option value="CNY">CNY</option>\n
<option value="COP">COP</option>\n            <option
value="CRC">CRC</option>\n            <option value="CUC">CUC</
option>\n            <option value="CVE">CVE</option>\n
<option value="CZK">CZK</option>\n            <option
value="DJF">DJF</option>\n            <option value="DKK">DKK</
option>\n            <option value="DOP">DOP</option>\n
<option value="EGP">EGP</option>\n            <option
value="ERN">ERN</option>\n            <option value="EUR">EUR</
option>\n            <option value="FJD">FJD</option>\n
<option value="FKP">FKP</option>\n            <option
value="GBP">GBP</option>\n            <option value="GEL">GEL</
option>\n            <option value="GGP">GGP</option>\n
<option value="GHS">GHS</option>\n            <option
value="GIP">GIP</option>\n            <option value="GMD">GMD</
option>\n            <option value="GNF">GNF</option>\n
<option value="GTQ">GTQ</option>\n            <option
value="GYD">GYD</option>\n            <option value="HKD">HKD</
option>\n            <option value="HNL">HNL</option>\n
<option value="HRK">HRK</option>\n            <option
value="HTG">HTG</option>\n            <option value="HUF">HUF</
option>\n            <option value="IDR">IDR</option>\n
<option value="ILS">ILS</option>\n            <option
value="IMP">IMP</option>\n            <option value="INR">INR</
option>\n            <option value="IQD">IQD</option>\n
<option value="IRR">IRR</option>\n            <option
value="ISK">ISK</option>\n            <option value="JEP">JEP</
option>\n            <option value="JMD">JMD</option>\n
<option value="JOD">JOD</option>\n            <option
value="JPY">JPY</option>\n            <option value="KES">KES</
option>\n            <option value="KGS">KGS</option>\n
<option value="KHR">KHR</option>\n            <option
value="KMF">KMF</option>\n            <option value="KPW">KPW</
option>\n            <option value="KRW">KRW</option>\n
<option value="KWD">KWD</option>\n            <option
```

The document was converted with **Code-Format**: https://code-format.com/

```
value="KYD">KYD</option>\n            <option value="KZT">KZT</
option>\n            <option value="LAK">LAK</option>\n
<option value="LBP">LBP</option>\n            <option
value="LKR">LKR</option>\n            <option value="LRD">LRD</
option>\n            <option value="LSL">LSL</option>\n
<option value="LYD">LYD</option>\n            <option
value="MAD">MAD</option>\n            <option value="MDL">MDL</
option>\n            <option value="MGA">MGA</option>\n
<option value="MKD">MKD</option>\n            <option
value="MNT">MNT</option>\n            <option value="MOP">MOP</
option>\n            <option value="MRU">MRU</option>\n
<option value="MUR">MUR</option>\n            <option
value="MVR">MVR</option>\n            <option value="MXN">MXN</
option>\n            <option value="MYR">MYR</option>\n
<option value="MZN">MZN</option>\n            <option
value="NAD">NAD</option>\n            <option value="NGN">NGN</
option>\n            <option value="NIO">NIO</option>\n
<option value="NOK">NOK</option>\n            <option
value="NPR">NPR</option>\n            <option value="NZD">NZD</
option>\n            <option value="OMR">OMR</option>\n
<option value="PAB">PAB</option>\n            <option
value="PEN">PEN</option>\n            <option value="PGK">PGK</
option>\n            <option value="PHP">PHP</option>\n
<option value="PKR">PKR</option>\n            <option
value="PLN">PLN</option>\n            <option value="PYG">PYG</
option>\n            <option value="QAR">QAR</option>\n
<option value="RON">RON</option>\n            <option
value="RSD">RSD</option>\n            <option value="RUB">RUB</
option>\n            <option value="RWF">RWF</option>\n
<option value="SAR">SAR</option>\n            <option
value="SBD">SBD</option>\n            <option value="SCR">SCR</
option>\n            <option value="SDG">SDG</option>\n
<option value="SEK">SEK</option>\n            <option
value="SGD">SGD</option>\n            <option value="SHP">SHP</
option>\n            <option value="SLE">SLE</option>\n
<option value="SLL">SLL</option>\n            <option
value="SOS">SOS</option>\n            <option value="SRD">SRD</
option>\n            <option value="STD">STD</option>\n
<option value="STN">STN</option>\n            <option
value="SVC">SVC</option>\n            <option value="SYP">SYP</
option>\n            <option value="SZL">SZL</option>\n
<option value="THB">THB</option>\n            <option
value="TJS">TJS</option>\n            <option value="TMT">TMT</
option>\n            <option value="TND">TND</option>\n
<option value="TOP">TOP</option>\n            <option
value="TRY">TRY</option>\n            <option value="TTD">TTD</
option>\n            <option value="TWD">TWD</option>\n
<option value="TZS">TZS</option>\n            <option
```

The document was converted with Code-Format: https://code-format.com/

```
value="UAH">UAH</option>\n            <option value="UGX">UGX</
option>\n            <option value="USD">USD</option>\n
<option value="UYU">UYU</option>\n            <option
value="UZS">UZS</option>\n            <option value="VES">VES</
option>\n            <option value="VND">VND</option>\n
<option value="VUV">VUV</option>\n            <option
value="WST">WST</option>\n            <option value="XAF">XAF</
option>\n            <option value="XAG">XAG</option>\n
<option value="XAU">XAU</option>\n            <option
value="XCD">XCD</option>\n            <option value="XCG">XCG</
option>\n            <option value="XDR">XDR</option>\n
<option value="XOF">XOF</option>\n            <option
value="XPD">XPD</option>\n            <option value="XPF">XPF</
option>\n            <option value="XPT">XPT</option>\n
<option value="YER">YER</option>\n            <option
value="ZAR">ZAR</option>\n            <option value="ZMW">ZMW</
option>\n            <option value="ZWG">ZWG</option>\n
<option value="ZWL">ZWL</option>\n        </select>\n\n</
form>\n\n<div style="max-width: 35em;">\n<table id="t2"  class="stripe
row-border order-column compact" >\n<thead>\n<tr>\n\t<th><div
style="font-size: 80%; vertical-align: middle;">Rank</div></
th>\n<th><div class="font_in_table_headers">Country</div></th><th><div
class="font_in_table_headers">Local Cheese <br/>(1 kg)</div></th></
tr>\n</thead>\n<tbody>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Kazakhstan">Kazakhstan</a></td><td
style="white-space: nowrap; text-align: right">8.50</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Portugal">Portugal</a></td><td style="white-space: nowrap;
text-align: right">10.24</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Greece">Greece</a></td><td style="white-
space: nowrap; text-align: right">12.05</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Latvia">Latvia</a></td><td
style="white-space: nowrap; text-align: right">9.49</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?country=Iran">Iran</
a></td><td style="white-space: nowrap; text-align: right">3.43</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Morocco">Morocco</a></td><td style="white-space: nowrap; text-
align: right">9.13</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Panama">Panama</a></td><td style="white-
space: nowrap; text-align: right">10.17</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Iraq">Iraq</a></td><td style="white-
```

The document was converted with **Code-Format**: https://code-format.com/

```
living/country_result.jsp?country=Iraq">Iraq</a></td><td style="white-
space: nowrap; text-align: right">4.12</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Chile">Chile</a></td><td
style="white-space: nowrap; text-align: right">9.41</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Nepal">Nepal</a></td><td style="white-space: nowrap; text-
align: right">6.10</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Argentina">Argentina</a></td><td
style="white-space: nowrap; text-align: right">8.52</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Ukraine">Ukraine</a></td><td style="white-space: nowrap; text-
align: right">7.65</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Bahrain">Bahrain</a></td><td style="white-
space: nowrap; text-align: right">8.49</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=India">India</a></td><td
style="white-space: nowrap; text-align: right">4.14</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Canada">Canada</a></td><td style="white-space: nowrap; text-
align: right">9.51</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Turkey">Turkey</a></td><td style="white-
space: nowrap; text-align: right">8.24</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Belgium">Belgium</a></td><td
style="white-space: nowrap; text-align: right">13.53</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Finland">Finland</a></td><td style="white-space: nowrap; text-
align: right">7.67</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Taiwan">Taiwan</a></td><td style="white-
space: nowrap; text-align: right">15.58</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=North+Macedonia">North Macedonia</
a></td><td style="white-space: nowrap; text-align: right">7.68</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=South+Africa">South Africa</a></td><td style="white-space:
nowrap; text-align: right">7.05</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Georgia">Georgia</a></td><td
style="white-space: nowrap; text-align: right">6.39</td></
```

The document was converted with Code-Format: https://code-format.com/

tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Peru">Peru</a></td><td style="white-space: nowrap; text-align: right">7.03</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Germany">Germany</a></td><td style="white-space: nowrap; text-align: right">12.87</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Puerto+Rico">Puerto Rico</a></td><td style="white-space: nowrap; text-align: right">9.17</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=United+States">United States</a></td><td style="white-space: nowrap; text-align: right">11.34</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Thailand">Thailand</a></td><td style="white-space: nowrap; text-align: right">14.95</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Costa+Rica">Costa Rica</a></td><td style="white-space: nowrap; text-align: right">8.22</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Sweden">Sweden</a></td><td style="white-space: nowrap; text-align: right">10.80</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Vietnam">Vietnam</a></td><td style="white-space: nowrap; text-align: right">10.04</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Poland">Poland</a></td><td style="white-space: nowrap; text-align: right">7.97</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Bulgaria">Bulgaria</a></td><td style="white-space: nowrap; text-align: right">9.38</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Jordan">Jordan</a></td><td style="white-space: nowrap; text-align: right">5.59</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Kuwait">Kuwait</a></td><td style="white-space: nowrap; text-align: right">5.24</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Nigeria">Nigeria</a></td><td style="white-space: nowrap; text-align: right">4.99</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/country_result.jsp?country=Tunisia">Tunisia</a></td><td style="white-space: nowrap; text-align: right">9.50</td></tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-living/

The document was converted with Code-Format: https://code-format.com/

```
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Croatia">Croatia</a></td><td style="white-
space: nowrap; text-align: right">10.65</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Sri+Lanka">Sri Lanka</a></td><td
style="white-space: nowrap; text-align: right">10.54</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Uruguay">Uruguay</a></td><td style="white-space: nowrap; text-
align: right">11.85</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=United+Kingdom">United Kingdom</a></td><td
style="white-space: nowrap; text-align: right">8.32</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=United+Arab+Emirates">United Arab Emirates</a></td><td
style="white-space: nowrap; text-align: right">9.43</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Kenya">Kenya</a></td><td style="white-space: nowrap; text-
align: right">8.70</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Switzerland">Switzerland</a></td><td
style="white-space: nowrap; text-align: right">23.56</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Spain">Spain</a></td><td style="white-space: nowrap; text-
align: right">12.42</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Lebanon">Lebanon</a></td><td style="white-
space: nowrap; text-align: right">7.68</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Azerbaijan">Azerbaijan</a></td><td
style="white-space: nowrap; text-align: right">3.83</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?country=Cuba">Cuba</
a></td><td style="white-space: nowrap; text-align: right">5.03</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Venezuela">Venezuela</a></td><td style="white-space: nowrap;
text-align: right">5.67</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Czech+Republic">Czech Republic</a></td><td
style="white-space: nowrap; text-align: right">11.10</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Israel">Israel</a></td><td style="white-space: nowrap; text-
align: right">15.69</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
```

The document was converted with **Code-Format**: https://code-format.com/.

country_result.jsp?country=Australia">Australia</a></td><td
style="white-space: nowrap; text-align: right">8.57</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Estonia">Estonia</a></td><td style="white-space: nowrap; text-
align: right">10.44</td></tr>\n<tr><td></td>\n<td style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Hong+Kong+%28China%29">Hong Kong (China)</
a></td><td style="white-space: nowrap; text-align: right">19.71</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Cyprus">Cyprus</a></td><td style="white-space: nowrap; text-
align: right">11.81</td></tr>\n<tr><td></td>\n<td style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Malaysia">Malaysia</a></td><td
style="white-space: nowrap; text-align: right">12.05</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?country=Oman">Oman</
a></td><td style="white-space: nowrap; text-align: right">6.23</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Bosnia+And+Herzegovina">Bosnia And Herzegovina</a></td><td
style="white-space: nowrap; text-align: right">8.04</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Armenia">Armenia</a></td><td style="white-space: nowrap; text-
align: right">6.46</td></tr>\n<tr><td></td>\n<td style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Austria">Austria</a></td><td style="white-
space: nowrap; text-align: right">17.42</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=South+Korea">South Korea</a></td><td
style="white-space: nowrap; text-align: right">11.20</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Luxembourg">Luxembourg</a></td><td style="white-space: nowrap;
text-align: right">18.54</td></tr>\n<tr><td></td>\n<td style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Brazil">Brazil</a></td><td style="white-
space: nowrap; text-align: right">7.85</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Algeria">Algeria</a></td><td
style="white-space: nowrap; text-align: right">7.67</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Slovenia">Slovenia</a></td><td style="white-space: nowrap;
text-align: right">13.10</td></tr>\n<tr><td></td>\n<td style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Colombia">Colombia</a></td><td

The document was converted with Code-Format. https://code-format.com/

```
country_result.jsp?country=Colombia">Colombia</a></td><td
style="white-space: nowrap; text-align: right">4.72</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Ecuador">Ecuador</a></td><td style="white-space: nowrap; text-
align: right">5.41</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Kosovo+%28Disputed+Territory%29">Kosovo
(Disputed Territory)</a></td><td style="white-space: nowrap; text-
align: right">5.80</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Hungary">Hungary</a></td><td style="white-
space: nowrap; text-align: right">8.57</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Japan">Japan</a></td><td
style="white-space: nowrap; text-align: right">9.44</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Moldova">Moldova</a></td><td style="white-space: nowrap; text-
align: right">8.16</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Belarus">Belarus</a></td><td style="white-
space: nowrap; text-align: right">6.73</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Mauritius">Mauritius</a></td><td
style="white-space: nowrap; text-align: right">7.99</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Albania">Albania</a></td><td style="white-space: nowrap; text-
align: right">10.64</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=New+Zealand">New Zealand</a></td><td
style="white-space: nowrap; text-align: right">7.11</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Italy">Italy</a></td><td style="white-space: nowrap; text-
align: right">14.97</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Singapore">Singapore</a></td><td
style="white-space: nowrap; text-align: right">15.72</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Egypt">Egypt</a></td><td style="white-space: nowrap; text-
align: right">2.98</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Bolivia">Bolivia</a></td><td style="white-
space: nowrap; text-align: right">4.32</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Malta">Malta</a></td><td
```

The document was converted with **Code-Format**: https://code-format.com/

style="white-space: nowrap; text-align: right">9.43</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Russia">Russia</a></td><td style="white-space: nowrap; text-
align: right">9.67</td></tr>\n<tr><td></td>\n<td style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Saudi+Arabia">Saudi Arabia</a></td><td
style="white-space: nowrap; text-align: right">6.47</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Netherlands">Netherlands</a></td><td style="white-space:
nowrap; text-align: right">12.39</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Pakistan">Pakistan</a></td><td
style="white-space: nowrap; text-align: right">5.10</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=China">China</a></td><td style="white-space: nowrap; text-
align: right">11.27</td></tr>\n<tr><td></td>\n<td style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Ireland">Ireland</a></td><td style="white-
space: nowrap; text-align: right">9.85</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Qatar">Qatar</a></td><td
style="white-space: nowrap; text-align: right">9.29</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Slovakia">Slovakia</a></td><td style="white-space: nowrap;
text-align: right">12.41</td></tr>\n<tr><td></td>\n<td style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=France">France</a></td><td style="white-
space: nowrap; text-align: right">17.22</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Lithuania">Lithuania</a></td><td
style="white-space: nowrap; text-align: right">10.24</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Serbia">Serbia</a></td><td style="white-space: nowrap; text-
align: right">8.57</td></tr>\n<tr><td></td>\n<td style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Romania">Romania</a></td><td style="white-
space: nowrap; text-align: right">7.65</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Philippines">Philippines</a></td><td
style="white-space: nowrap; text-align: right">6.54</td></
tr>\n<tr><td></td>\n<td style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Uzbekistan">Uzbekistan</a></td><td style="white-space: nowrap;

This document was converted with Code-Format. https://code-format.com/

```
text-align: right">7.01</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Bangladesh">Bangladesh</a></td><td
style="white-space: nowrap; text-align: right">6.96</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Norway">Norway</a></td><td style="white-space: nowrap; text-
align: right">11.18</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Denmark">Denmark</a></td><td style="white-
space: nowrap; text-align: right">12.98</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Dominican+Republic">Dominican
Republic</a></td><td style="white-space: nowrap; text-align:
right">7.17</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Mexico">Mexico</a></td><td style="white-
space: nowrap; text-align: right">8.03</td></tr>\n<tr><td></td>\n<td
style="white-space:nowrap"><a href="https://www.numbeo.com/cost-of-
living/country_result.jsp?country=Zimbabwe">Zimbabwe</a></td><td
style="white-space: nowrap; text-align: right">9.16</td></
tr>\n<tr><td></td>\n<td  style="white-space:nowrap"><a href="https://
www.numbeo.com/cost-of-living/country_result.jsp?
country=Montenegro">Montenegro</a></td><td style="white-space: nowrap;
text-align: right">7.99</td></tr>\n<tr><td></td>\n<td  style="white-
space:nowrap"><a href="https://www.numbeo.com/cost-of-living/
country_result.jsp?country=Indonesia">Indonesia</a></td><td
style="white-space: nowrap; text-align: right">7.45</td></tr>\n</
tbody>\n</table>\n</div> <!-- max div for tables -->\n\n    <script>\n
whenDocReady(function() {\n    var t_t2 = $(\'#t2\').DataTable({\n
\'columnDefs\': [ {\n      \'searchable\': false,\n
\'orderable\': false,\n      \'targets\': 0\n  } ],\n
\'paging\': false,\n        \'order\': [[ 2, "desc" ]]\n,    });\n
t_t2.on( \'order.dt search.dt\', function () {\n        t_t2.column(0,
{search:\'applied\', order:\'applied\'}).nodes().each( function (cell,
i) {\n            cell.innerHTML = i+1;\n        } );\n    } ).draw();
\n  });\n</script>\n\n<p>Last Update: March 14, 2026 10:32 PM,
CET\n<br/>Data Source: Numbeo contributors and official sources\n<br/
>Based on data collected within the last 12 months.\n</div>\n\n<!--
Column 1 end -->\n\n<div class="footer_content">\n<a href="https://
www.numbeo.com/common/about.jsp">About</a><span
class="footer_separator"></span>\n<a href="https://www.numbeo.com/
common/in_the_news.jsp">Numbeo in the News</a><span
class="footer_separator"></span>\n<a href="https://www.numbeo.com/
common/join_community.jsp">Newsletter</a><span
class="footer_separator"></span>\n<a href="https://www.numbeo.com/
premium/advertise">Advertise</a><span class="footer_separator"></
span>\n<a href="https://www.numbeo.com/premium/index">Premium</a><span
class="footer_separator"></span>\n<a href="https://www.numbeo.com/
```

The document was converted with **Code-Format**: https://code-format.com/

```
common/api.jsp" rel="nofollow">API</a><p>\n<span style="font-style:
italic">\nCopyright &copy; 2009-2026 Numbeo. Your use of this service
is subject to our \n<a href="https://www.numbeo.com/common/
terms_of_use.jsp">Terms of Use</a> and \n<a href="https://
www.numbeo.com/common/privacy.jsp">Privacy Policy</a>\n</span></p>\n</
div>\n</div> <!-- end of innerWidth --> \n\n    </body>\n</
html>\n\n\n\n'
```

In [9]:

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(data, "html.parser")
```

In [13]:

```
tables = soup.find_all("table")
table_index = None
for index,table in enumerate(tables):
    if'Local Cheese ' in str(table):
        table_index = index
        break
table_index
```

```
1
```

In [23]:

```
cheese = pd.read_html(io.StringIO(str(tables[table_index])), flavor =
'bs4')[0]
cheese
```

	Rank	Country	Local Cheese (1 kg)
0	NaN	Kazakhstan	8.50
1	NaN	Portugal	10.24
2	NaN	Greece	12.05
3	NaN	Latvia	9.49
4	NaN	Iran	3.43
...	...	...	...
96	NaN	Dominican Republic	7.17
97	NaN	Mexico	8.03
98	NaN	Zimbabwe	9.16
99	NaN	Montenegro	7.99
100	NaN	Indonesia	7.45

101 rows × 3 columns

In [29]:

```
min_val = cheese["Local Cheese (1 kg)"].min()
min_row = cheese[cheese["Local Cheese (1 kg)"] == min_val]
min_row
```

	Rank	Country	Local Cheese (1 kg)
76	NaN	Egypt	2.98

The document was converted with **Code-Format**: https://code-format.com/

In [30]:

```
max_val = cheese["Local Cheese (1 kg)"].max()
max_row = cheese[cheese["Local Cheese (1 kg)"] == max_val]
max_row
```

	Rank	Country	Local Cheese (1 kg)
42	NaN	Switzerland	23.56

In [31]:

```
print(len(cheese["Local Cheese (1 kg)"]))
```

```
101
```

The document was converted with **Code-Format**: https://code-format.com/

Lab3_2

Exported 3/15/2026, 1:08:41 AM

Виконавець: **Базилевич Олексій Андрійович**, група **К-27**
Викладач: **Андрій Ляшко** Частина 2

Варіант 4 (deque)

За даними сайту https://cppreference.com з'ясувати: Member types, Member functions, Non-member functions. За наявності враховувати підрозбиття на категорії. Вивести: відповідні переліки через кому. Також з'ясувати скільки перелічено конструкторів. Для кожного з'ясувати стандарти, в яких він існує (та вивести). Вивести конструктори, що існують в стандарті C++23.

In [32]:

```
import requests
from bs4 import BeautifulSoup

url = "https://en.cppreference.com/w/cpp/container/deque"
soup = BeautifulSoup(requests.get(url).text, "html.parser")
```

In [33]:

```
t_table = soup.find(id="Member_types").find_next("table")
print("Member types:\n" + ", ".join([c.text for c in
t_table.find_all("code")]))
```

```
Member types:
value_type, T, allocator_type, Allocator, size_type, difference_type,
reference, const_reference, pointer, Allocator::pointer,
const_pointer, Allocator::const_pointer, iterator, value_type,
const_iterator, reverse_iterator, const_reverse_iterator
```

The document was converted with **Code-Format**: https://code-format.com/

In [34]:

```
print("Member functions:")
f_table = soup.find(id="Member_functions").find_next("table")
for tr in f_table.find_all("tr"):
    if tr.h5:
        print(f"\n{tr.h5.text}:", end=" ")
    else:
        funcs = [s.text for s in tr.select(".t-lines > span") if
s.text]
        if funcs:
            print(", ".join(funcs), end=", ")
```

```
Member functions:
(constructor), (destructor), operator=, assign, assign_range, (C++23),
get_allocator,
  Element access : at, operator[], front, back,
  Iterators : begin, cbegin, (C++11), end, cend, (C++11), rbegin,
crbegin, (C++11), rend, crend, (C++11),
  Capacity : empty, size, max_size, shrink_to_fit, (DR*),
  Modifiers : clear, insert, insert_range, (C++23), emplace, (C++11),
erase, push_back, emplace_back, (C++11), append_range, (C++23),
pop_back, push_front, emplace_front, (C++11), prepend_range, (C++23),
pop_front, resize, swap,
```

In [35]:

```
n_table = soup.find(id="Non-member_functions").find_next("table")
n_funcs = set(s.text.replace("(std::deque)", "") for s in
n_table.select(".t-lines > span") if s.text)
print("\nNon-member functions:\n" + ", ".join(n_funcs))
```

```

Non-member functions:
operator<=>, std::swap, (removed in C++20), operator<=, operator!=,
erase, erase_if, operator==, operator>=, operator<, operator>, (C++20)
```

The document was converted with **Code-Format**: https://code-format.com/

In [36]:

```
c_soup = BeautifulSoup(requests.get(url + "/deque").text,
"html.parser")
ctors = c_soup.select(".t-dcl-begin tr.t-dcl")

print(f"Конструкторів: {len(ctors)}")

for tr in ctors:
    num = tr.find("td").text.strip(" ()")
    mark = tr.select_one(".t-mark-rev")
    std = mark.text.strip(" ()").replace("since ", "") if mark else
"C++98"
    print(f"{num}: {std}")

cxx23 = [tr.find("td").text.strip(" ()") for tr in ctors]
print(f"C++23: {', '.join(cxx23)}")
```

The document was converted with **Code-Format**: https://code-format.com/

```
Конструкторів: 15
deque() : deque(Allocator()) {}: C++11
explicit deque( const Allocator& alloc = Allocator() );: until C++11
explicit deque( const Allocator& alloc );: C++11
explicit deque( size_type count,
                const Allocator& alloc = Allocator() );: C++11
explicit deque( size_type count, const T& value = T(),
                const Allocator& alloc = Allocator() );: until C++11
deque( size_type count, const T& value,
       const Allocator& alloc = Allocator() );: C++11
template< class InputIt >
deque( InputIt first, InputIt last,

       const Allocator& alloc = Allocator() );: constexpr C++26
template< container-compatible-range<T> R >
deque( std::from_range_t, R&& rg,

       const Allocator& alloc = Allocator() );: C++23
deque( const deque& other );: constexpr C++26
deque( deque&& other );: C++11
deque( const deque& other, const Allocator& alloc );: C++11
deque( const deque& other,
       const std::type_identity_t<Allocator>& alloc );: C++23
deque( deque&& other, const Allocator& alloc );: C++11
deque( deque&& other, const std::type_identity_t<Allocator>& alloc );:
C++23
deque( std::initializer_list<T> init,
       const Allocator& alloc = Allocator() );: C++11
C++23: deque() : deque(Allocator()) {}, explicit deque( const
Allocator& alloc = Allocator() );, explicit deque( const Allocator&
alloc );, explicit deque( size_type count,
                const Allocator& alloc = Allocator() );, explicit
deque( size_type count, const T& value = T(),
                const Allocator& alloc = Allocator() );, deque(
size_type count, const T& value,
       const Allocator& alloc = Allocator() );, template< class
InputIt >
deque( InputIt first, InputIt last,

       const Allocator& alloc = Allocator() );, template< container-
compatible-range<T> R >
deque( std::from_range_t, R&& rg,

       const Allocator& alloc = Allocator() );, deque( const deque&
other );, deque( deque&& other );, deque( const deque& other, const
Allocator& alloc );, deque( const deque& other,
       const std::type_identity_t<Allocator>& alloc );, deque( deque&&
```

The document was converted with **Code-Format**: https://code-format.com/

```
other, const Allocator& alloc );, deque( deque&& other, const
std::type_identity_t<Allocator>& alloc );, deque(
std::initializer_list<T> init,
       const Allocator& alloc = Allocator() );
```

The document was converted with **Code-Format**: https://code-format.com/

Лабораторна робота № 4
варіант № 99 (20; 4)

Лабораторна робота №4
Для кожної задачі здається виконаний з чистого аркушу блокнот, пдф-звіт (отримується за блокнотом) та результуюче зображення у форматі png.
У блокноті обов'язково має бути зазначена інформація про виконавця (ПІБ, група, викладач), варіант та умова задачі. (Це критична вимога, порушення якої занулить бали.)

Зроблено всі п'ять задач, зауважень нема. — макс. бал (15), за умови успішного захисту.

Усі зображення мають бути належно підписані та анотовані (інакше -3 бали).
Якщо в задачі 4 наявні не всі супровідні лінії, то це знімає 3 бали.
Якщо в задачі 5 відсутнє рівняння поверхні (або відображається як plain text), то це знімає 1 бал.

<u>Задача 1.</u>
У завданні 2 лабораторної роботи №1 за датами створення файлів для кожного місяця, починаючи з вересня 2024-го року було знайдено таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний розмір у байтах, кількість їх рядків (враховувати тільки для файлів категорії C; білі та порожні рядки не рахувати), середній розмір.
Для кожної характеристики по місяцях побудувати: графік, графік з накопиченням, стовпчикову діаграму, стовпчикову діаграму з накопиченням. Графіки розташувати на одному зображенні.

З використанням даних завдання 1 лабораторної роботи №1 побудувати кругову діаграму.

<u>Задача 2.</u>
У завданні 4 лабораторної роботи №2 оброблялися значення функції у точках. Побудувати графік цієї функції (кусково-лінійне наближення). На тому самому зображенні іншим кольором побудувати графік знайденої в результаті обчислень функції.

<u>Задача 3.</u>
У завданні 4 лабораторної роботи №2 оброблялися значення функції у точках. Побудувати гістограму, яка відображає розподіл точок по інтервалах.

<u>Задача 4.</u>
Номер варіанту отримується за формулою
(номер варіанту для лабораторної роботи №2) %20 +1.

Створити **статичне** зображення, на якому намалювати криву та супутні елементи (див. посилання). Параметри (якщо наявні) задає користувач. Якщо за посиланням зображення динамічне, то захопити його в якийсь момент часу (ненульовий).

Використання циклів заборонено.

1) Локон Аньєзі

https://uk.wikipedia.org/wiki/%D0%9B%D0%BE%D0%BA%D0%BE%D0%BD_%D0%90%D0%BD%D1%8C%D1%94%D0%B7%D1%96

2) Декартів лист
https://uk.wikipedia.org/wiki/%D0%94%D0%B5%D0%BA%D0%B0%D1%80%D1%82%D1%96%D0%B2_%D0%BB%D0%B8%D1%81%D1%82

3) трисектриса Маклорена
https://uk.wikipedia.org/wiki/%D0%A2%D1%80%D0%B8%D1%81%D0%B5%D0%BA%D1%82%D1%80%D0%B8%D1%81%D0%B0_%D0%9C%D0%B0%D0%BA%D0%BB%D0%BE%D1%80%D0%B5%D0%BD%D0%B0

4) овал Декарта
https://uk.wikipedia.org/wiki/%D0%9E%D0%B2%D0%B0%D0%BB_%D0%94%D0%B5%D0%BA%D0%B0%D1%80%D1%82%D0%B0

5) лемніската Бернуллі
https://uk.wikipedia.org/wiki/%D0%9B%D0%B5%D0%BC%D0%BD%D1%96%D1%81%D0%BA%D0%B0%D1%82%D0%B0_%D0%91%D0%B5%D1%80%D0%BD%D1%83%D0%BB%D0%BB%D1%96

6) циклоїда
https://uk.wikipedia.org/wiki/%D0%A6%D0%B8%D0%BA%D0%BB%D0%BE%D1%97%D0%B4%D0%B0

7) астроїда
https://uk.wikipedia.org/wiki/%D0%90%D1%81%D1%82%D1%80%D0%BE%D1%97%D0%B4%D0%B0

8) трохоїда
https://uk.wikipedia.org/wiki/%D0%A2%D1%80%D0%BE%D1%85%D0%BE%D1%97%D0%B4%D0%B0

9) гіпоциклоїда
https://uk.wikipedia.org/wiki/%D0%93%D1%96%D0%BF%D0%BE%D1%86%D0%B8%D0%BA%D0%BB%D0%BE%D1%97%D0%B4%D0%B0

10) епіциклоїда
https://uk.wikipedia.org/wiki/%D0%95%D0%BF%D1%96%D1%86%D0%B8%D0%BA%D0%BB%D0%BE%D1%97%D0%B4%D0%B0

11) цисоїда Діокла
https://uk.wikipedia.org/wiki/%D0%A6%D0%B8%D1%81%D0%BE%D1%97%D0%B4%D0%B0_%D0%94%D1%96%D0%BE%D0%BA%D0%BB%D0%B0

12) конхоїда Нікомеда
https://uk.wikipedia.org/wiki/%D0%9A%D0%BE%D0%BD%D1%85%D0%BE%D1%97%D0%B4%D0%B0_%D0%9D%D1%96%D0%BA%D0%BE%D0%BC%D0%B5%D0%B4%D0%B0

13) трифолій
https://uk.wikipedia.org/wiki/%D0%A2%D1%80%D0%B8%D0%BF%D0%B5%D0%BB%D1%8E%D1%81%D1%82%D0%BA%D0%BE%D0%B2%D0%B0_%D1%82%D1%80%D0%BE%D1%8F%D0%BD%D0%B4%D0%B0

14) дельтоїда
https://uk.wikipedia.org/wiki/%D0%94%D0%B5%D0%BB%D1%8C%D1%82%D0%BE%D1%97%D0%B4%D0%B0

15) офіуріда
https://uk.wikipedia.org/wiki/%D0%9E%D1%84%D1%96%D1%83%D1%80%D0%B8%D0%B4%D0%B0

16) равлик Паскаля
https://uk.wikipedia.org/wiki/%D0%A0%D0%B0%D0%B2%D0%BB%D0%B8%D0%BA_%D0%9F%D0%B0%D1%81%D0%BA%D0%B0%D0%BB%D1%8F

17) строфоїда
https://uk.wikipedia.org/wiki/%D0%A1%D1%82%D1%80%D0%BE%D1%84%D0%BE%D1%97%D0%B4%D0%B0

18) чотирипелюсткова троянда
https://uk.wikipedia.org/wiki/%D0%A7%D0%BE%D1%82%D0%B8%D1%80%D0%B8%D0%BF%D0%B5%D0%BB%D1%8E%D1%81%D1%82%D0%BA%D0%BE%D0%B2%D0%B0_%D1%82%D1%80%D0%BE%D1%8F%D0%BD%D0%B4%D0%B0

19) кубіка Чірнгауза
https://uk.wikipedia.org/wiki/%D0%9A%D1%83%D0%B1%D0%B8%D0%BA%D0%B0_%D0%A7%D0%B8%D1%80%D0%BD%D0%B3%D0%B0%D1%83%D0%B7%D0%B0

20) кардіоїда
https://uk.wikipedia.org/wiki/%D0%9A%D0%B0%D1%80%D0%B4%D1%96%D0%BE%D1%97%D0%B4%D0%B0

Задача 5.
Номер варіанту отримується за формулою
(номер варіанту для лабораторної роботи №2) %8 +1.

Створити **статичне** зображення, на якому намалювати тривимірну поверхню (як wireframe plot). На зображенні має бути підпис (назва поверхні) та легенда (містить рівняння поверхні).

Використання циклів заборонено.

1) еліптичний параболоїд
https://uk.wikipedia.org/wiki/%D0%9F%D0%B0%D1%80%D0%B0%D0%B1%D0%BE%D0%BB%D0%BE%D1%97%D0%B4

2) гіперболічний параболоїд

https://uk.wikipedia.org/wiki/%D0%9F%D0%B0%D1%80%D0%B0%D0%B1%D0%BE%D0%BB%D0%BE%D1%97%D0%B4

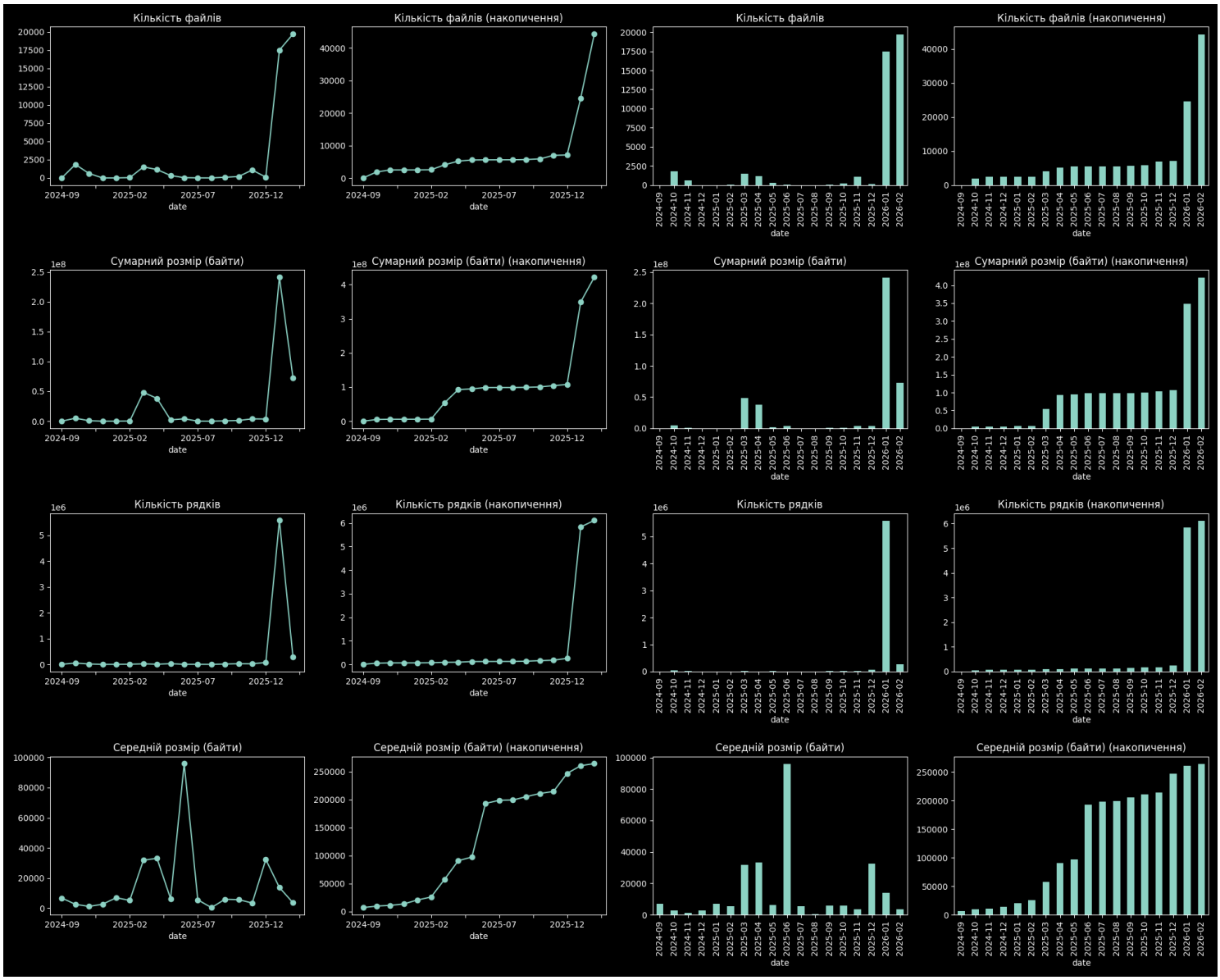
3) однопорожнинний гіперболоїд
https://uk.wikipedia.org/wiki/%D0%93%D1%96%D0%BF%D0%B5%D1%80%D0%B1%D0%BE%D0%BB%D0%BE%D1%97%D0%B4

4) двопорожнинний гіперболоїд
https://uk.wikipedia.org/wiki/%D0%93%D1%96%D0%BF%D0%B5%D1%80%D0%B1%D0%BE%D0%BB%D0%BE%D1%97%D0%B4
5) еліпсоїд
https://uk.wikipedia.org/wiki/%D0%95%D0%BB%D1%96%D0%BF%D1%81%D0%BE%D1%97%D0%B4

6) конус
https://uk.wikipedia.org/wiki/%D0%9A%D0%BE%D0%BD%D1%83%D1%81

7) стрічка Мебіуса
https://uk.wikipedia.org/wiki/%D0%A1%D1%82%D1%80%D1%96%D1%87%D0%BA%D0%B0_%D0%9C%D0%B5%D0%B1%D1%96%D1%83%D1%81%D0%B0

8) пляшка Кляйна
https://uk.wikipedia.org/wiki/%D0%9F%D0%BB%D1%8F%D1%88%D0%BA%D0%B0_%D0%9A%D0%BB%D1%8F%D0%B9%D0%BD%D0%B0


Кількість файлів
Кількість файлів (накопичення)
Кількість файлів
Кількість файлів (накопичення)
Сумарний розмір (байти)
Сумарний розмір (байти) (накопичення)
Сумарний розмір (байти)
Сумарний розмір (байти) (накопичення)
Кількість рядків
Кількість рядків (накопичення)
Кількість рядків
Кількість рядків (накопичення)
Середній розмір (байти)
Середній розмір (байти) (накопичення)
Середній розмір (байти)
Середній розмір (байти) (накопичення)
date

50b51d2f-509e-40fa-9443-f97ed37a6bcb

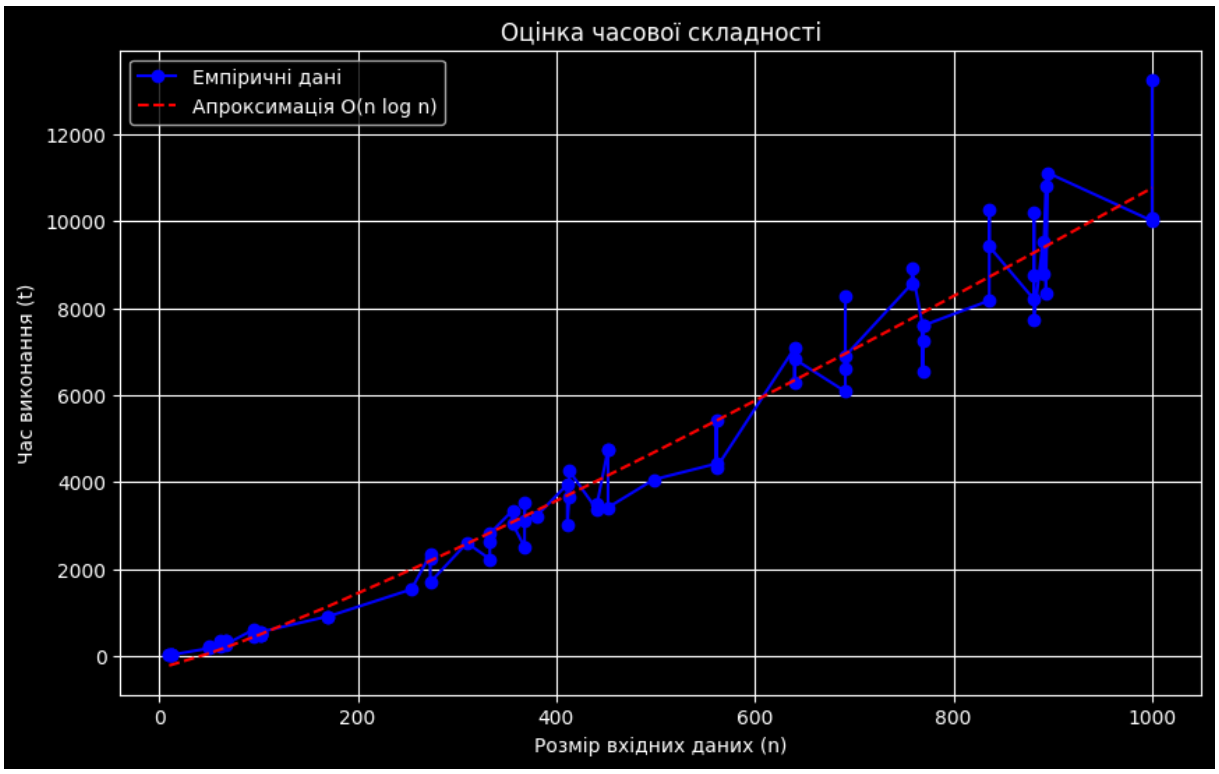
Лабораторна робота №4. Задача 2

Виконавець: Базилевич Олексій, група К27
Викладач: Андрій Ляшко
Умова: За даними 4 завдання лаб. 2 побудувати графік значень функції у точках (кусково-лінійне наближення). На тому ж зображенні іншим кольором побудувати апроксимовану функцію.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
```

In [2]:
```
df = pd.read_csv('lab2_var099.csv')
df = df.sort_values(by='n')
x_data = df['n'].values
y_data = df['t'].values
```

In [3]:
```
fit_func = lambda n, a, b: a * n * np.log(n + 1) + b
popt, _ = curve_fit(fit_func, x_data, y_data)
y_fit = fit_func(x_data, *popt)

plt.figure(figsize=(10, 6))
plt.plot(x_data, y_data, marker='o', linestyle='-', color='blue', label='Емпіричні дані')
plt.plot(x_data, y_fit, linestyle='--', color='red', label='Апроксимація O(n log n)')
plt.xlabel('Розмір вхідних даних (n)')
plt.ylabel('Час виконання (t)')
plt.title('Оцінка часової складності')
plt.legend()
plt.grid(True)
plt.savefig("task2_complexity.png")
plt.show()
```


Оцінка часової складності
Емпіричні дані
Апроксимація O(n log n)
Час виконання (t)
Розмір вхідних даних (n)
12000
10000
8000
6000
4000
2000
0
0
200
400
600
800
1000

db7f3407-d60b-49a5-84ce-40614e2a5f5a

Лабораторна робота №4. Задача 3

Виконавець: Базилевич Олексій, група К27
Викладач: Андрій Ляшко
Умова: Побудувати гістограму, яка відображає розподіл точок по інтервалах на основі даних про успішність/функції.

In [1]:

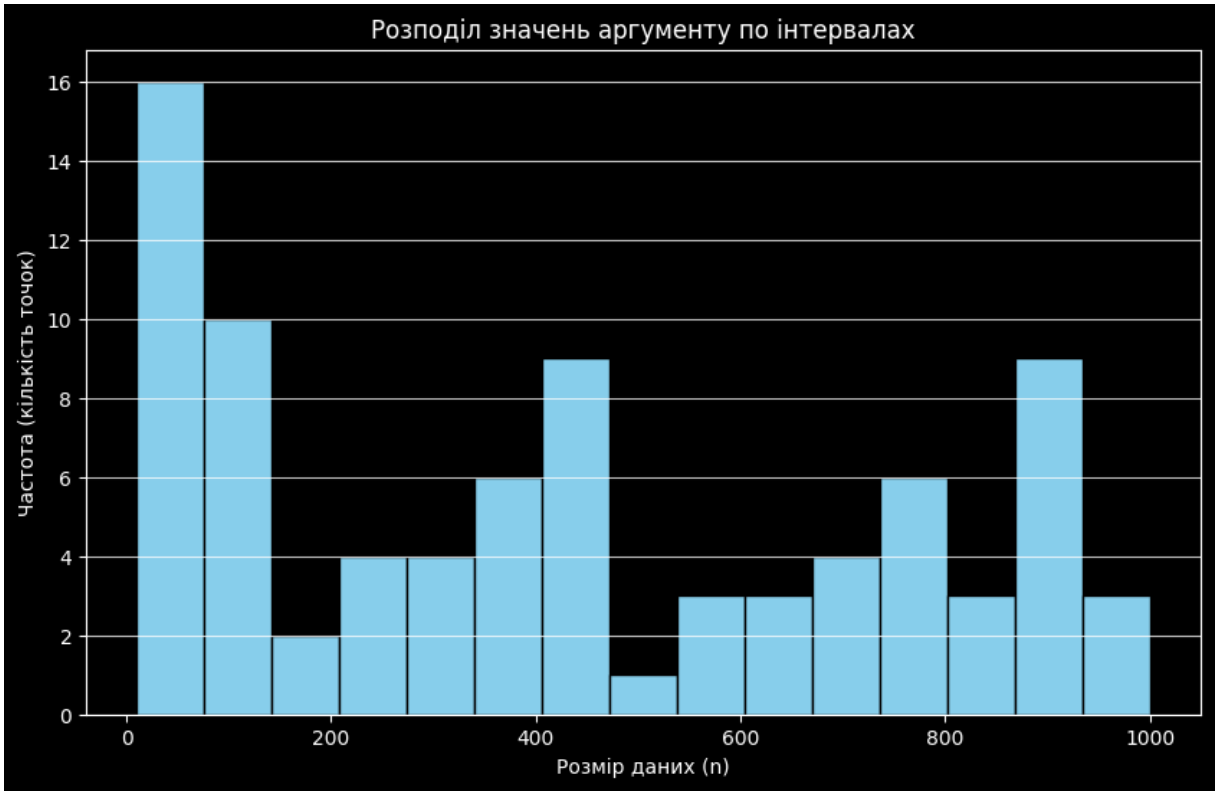
```
import pandas as pd
import matplotlib.pyplot as plt
```

In [2]:

```
df = pd.read_csv('lab2_var099.csv')
bins_count = 15
```

In [3]:

```
plt.figure(figsize=(10, 6))
plt.hist(df['n'], bins=bins_count, edgecolor='black', color='skyblue')
plt.xlabel('Розмір даних (n)')
plt.ylabel('Частота (кількість точок)')
plt.title('Розподіл значень аргументу по інтервалах')
plt.grid(axis='y', alpha=0.75)
plt.savefig("task3_histogram.png")
plt.show()
```



2d79a487-702b-48a4-bc50-e45b2a20d479

Лабораторна робота №4. Задача 4

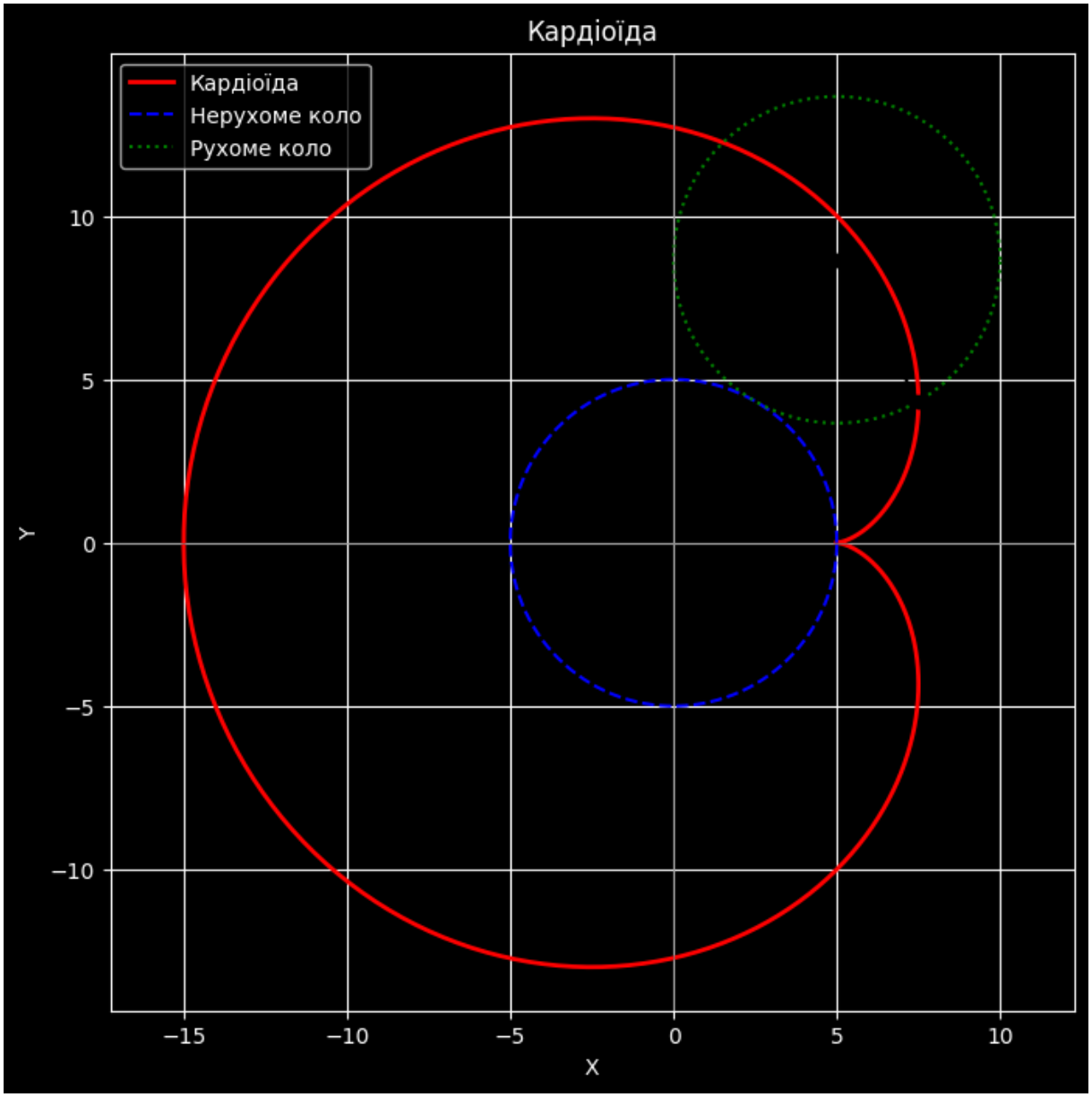
Виконавець: Базилевич Олексій, група К27
Викладач: Андрій Ляшко
Варіант: 20 (Кардіоїда)
Умова: Створити статичне зображення, на якому намалювати криву та супутні елементи. Використання циклів заборонено.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
a = 5
theta = np.linspace(0, 2 * np.pi, 500)
theta_point = np.pi / 3
```

In [3]:
```
x_cardioid = a * (2 * np.cos(theta) - np.cos(2 * theta))
y_cardioid = a * (2 * np.sin(theta) - np.sin(2 * theta))

x_base = a * np.cos(theta)
y_base = a * np.sin(theta)

x_roll_center = 2 * a * np.cos(theta_point)
y_roll_center = 2 * a * np.sin(theta_point)
x_roll = x_roll_center + a * np.cos(theta)
y_roll = y_roll_center + a * np.sin(theta)

px = a * (2 * np.cos(theta_point) - np.cos(2 * theta_point))
py = a * (2 * np.sin(theta_point) - np.sin(2 * theta_point))

plt.figure(figsize=(8, 8))
plt.plot(x_cardioid, y_cardioid, color='red', linewidth=2, label='Кардіоїда')
plt.plot(x_base, y_base, color='blue', linestyle='--', label='Нерухоме коло')
plt.plot(x_roll, y_roll, color='green', linestyle=':', label='Рухоме коло')
plt.plot([x_roll_center, px], [y_roll_center, py], color='black', marker='o')

plt.axhline(0, color='gray', linewidth=0.5)
plt.axvline(0, color='gray', linewidth=0.5)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Кардіоїда')
plt.legend()
plt.axis('equal')
plt.grid(True)
```

```
plt.savefig("task4_cardioid.png")
plt.show()
```



9242fd07-6aa8-4d58-b586-1c42a148c03a

Лабораторна робота №4. Задача 5

Виконавець: Базилевич Олексій, група К27
Викладач: Андрій Ляшко
Варіант: 4 (Двопорожнинний гіперболоїд)
Умова: Намалювати тривимірну поверхню (wireframe plot). На зображенні має бути підпис та легенда з рівнянням поверхні. Використання циклів заборонено.

In [1]:

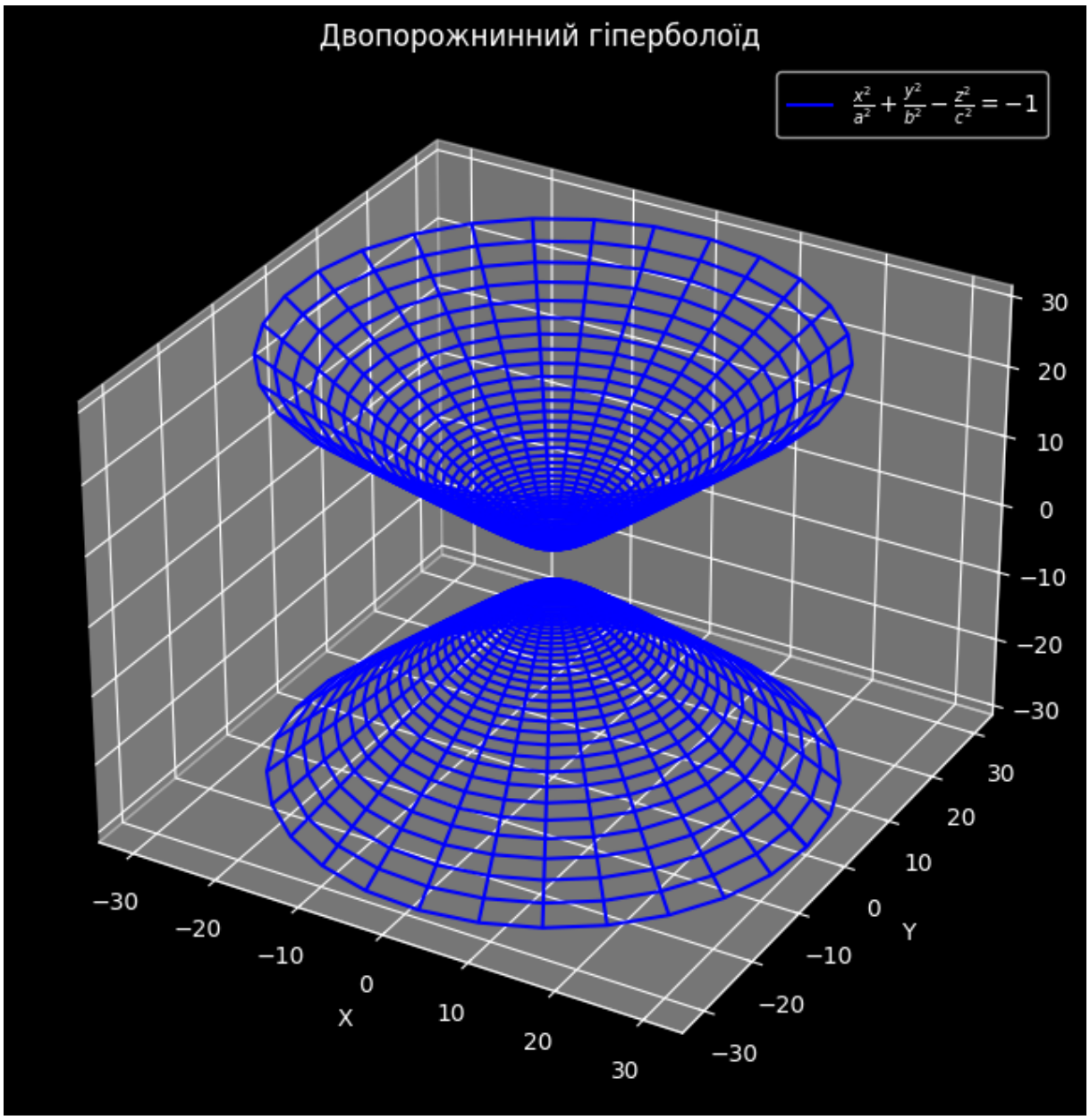
```
import numpy as np
import matplotlib.pyplot as plt
```

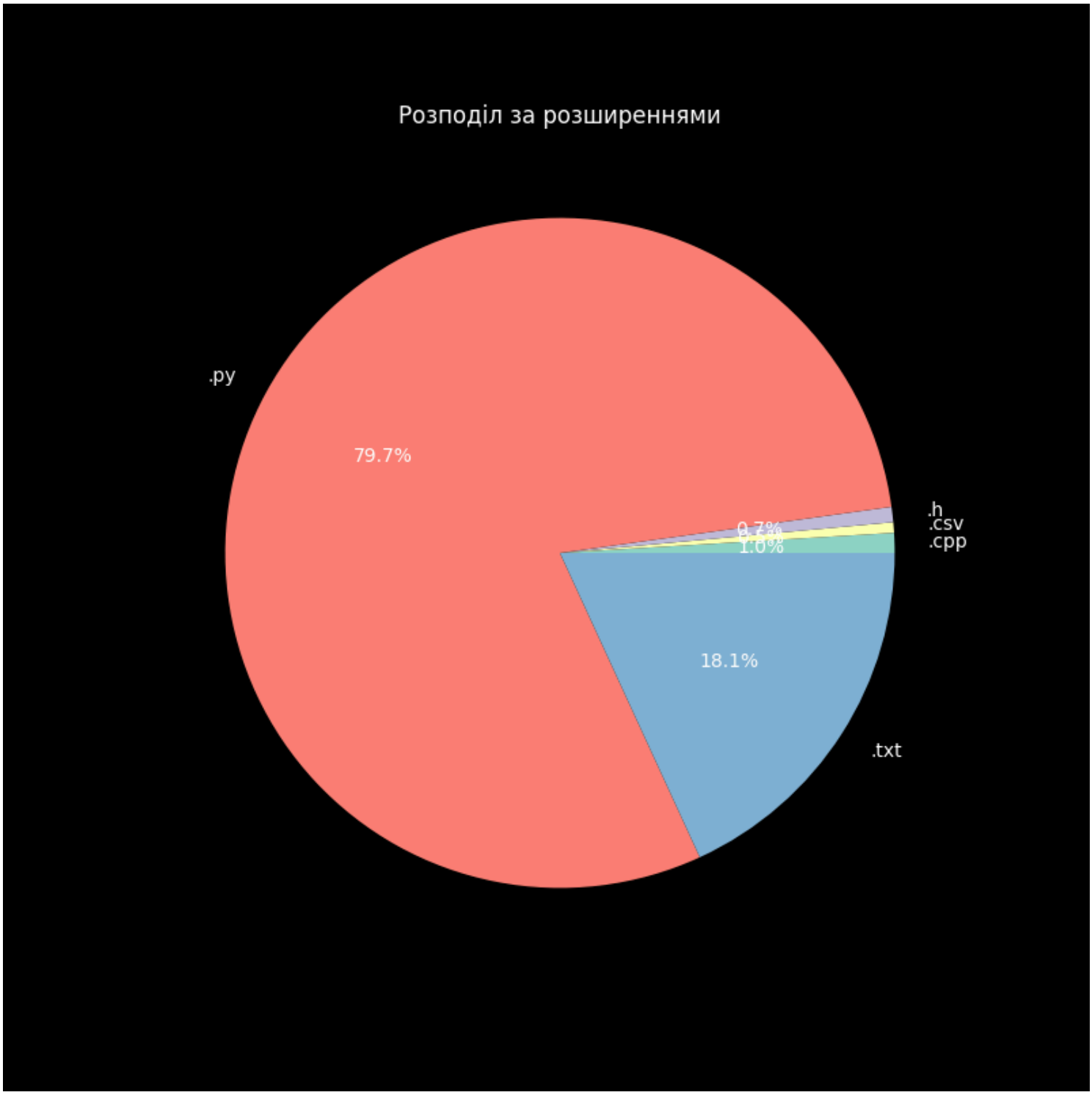
In [2]:

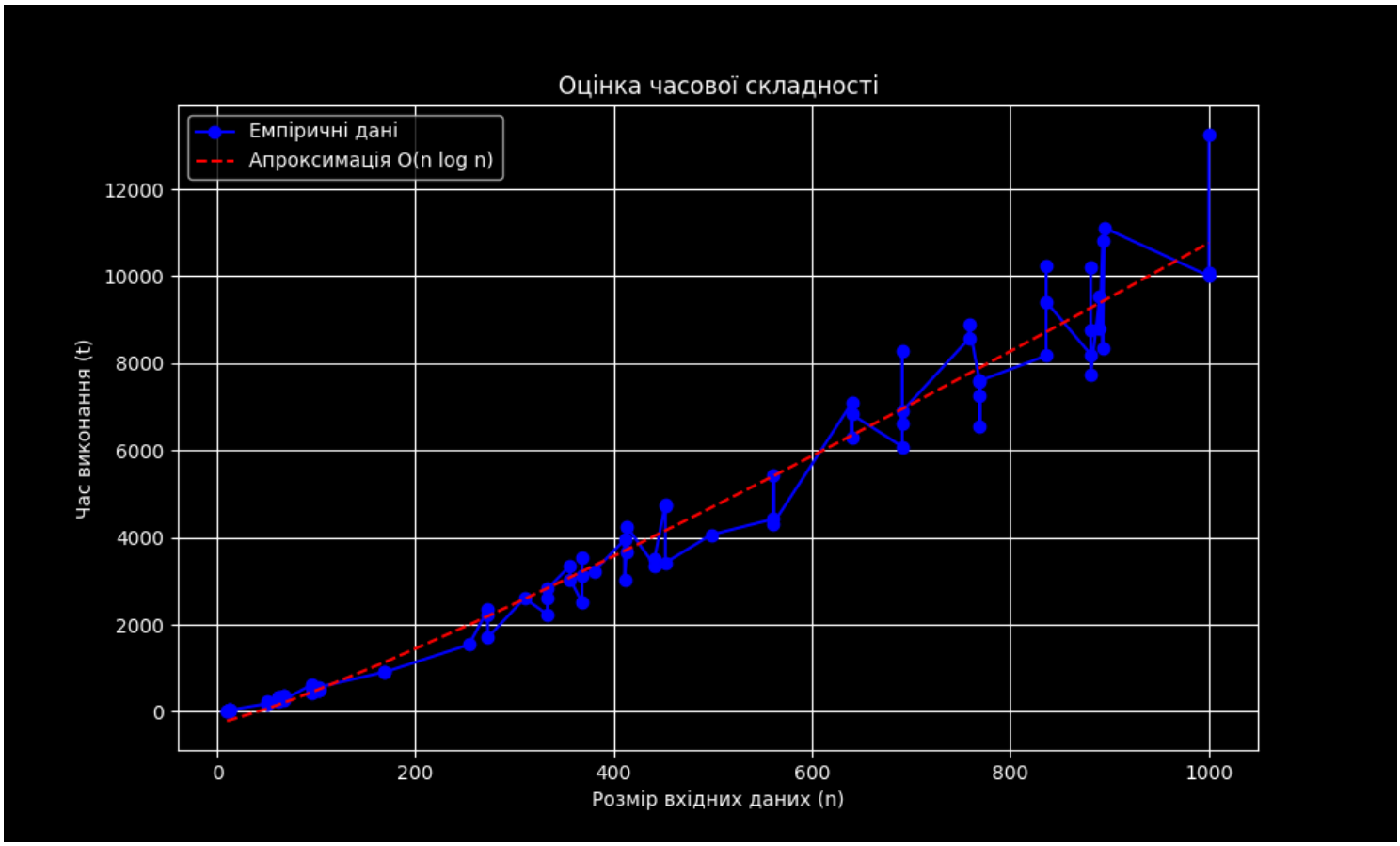
```
a, b, c = 3.0, 3.0, 3.0
grid_points = 30
u = np.linspace(0, 2 * np.pi, grid_points)
v = np.linspace(1.0, 3.0, grid_points)
U, V = np.meshgrid(u, v)
```

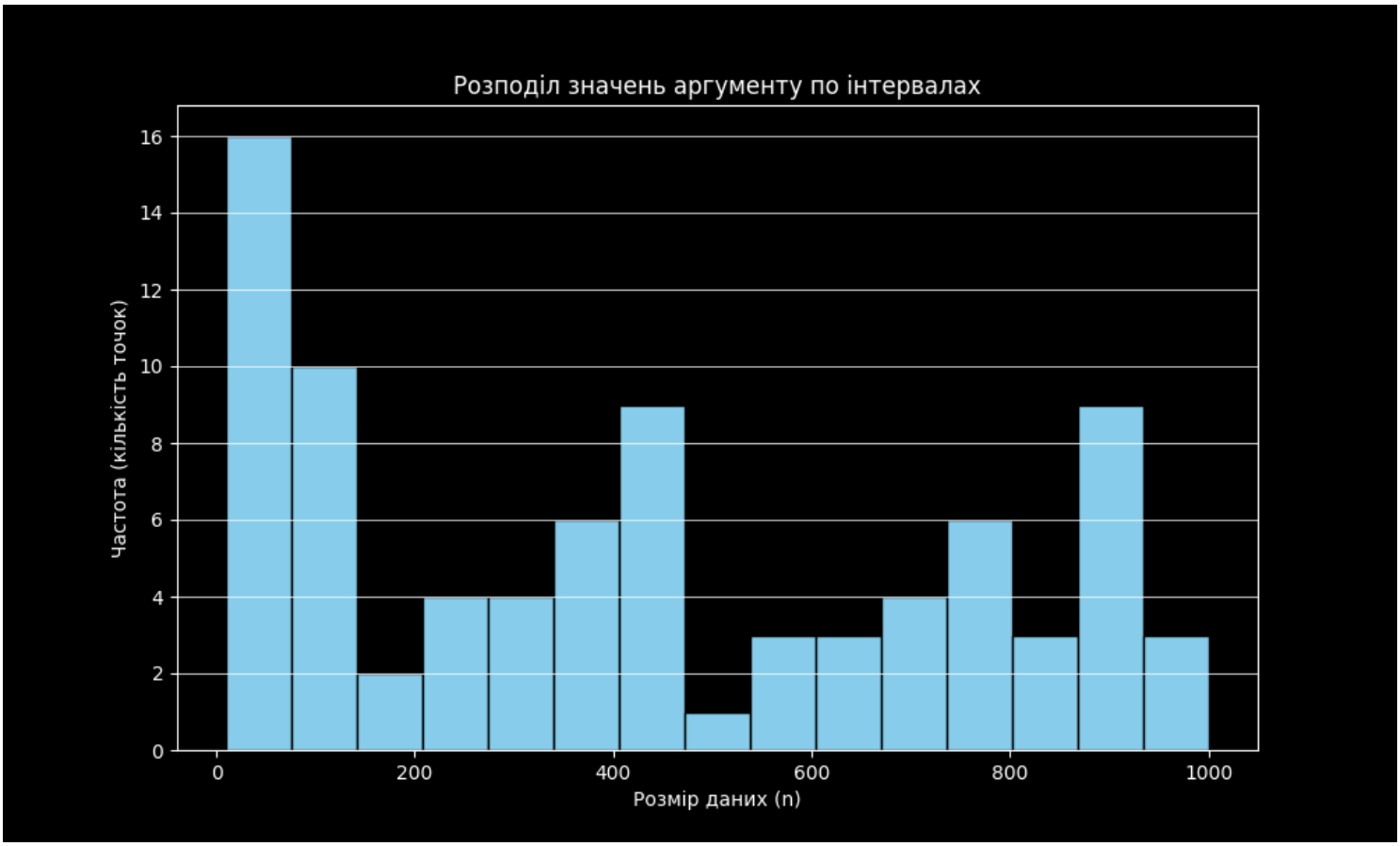
In [3]:

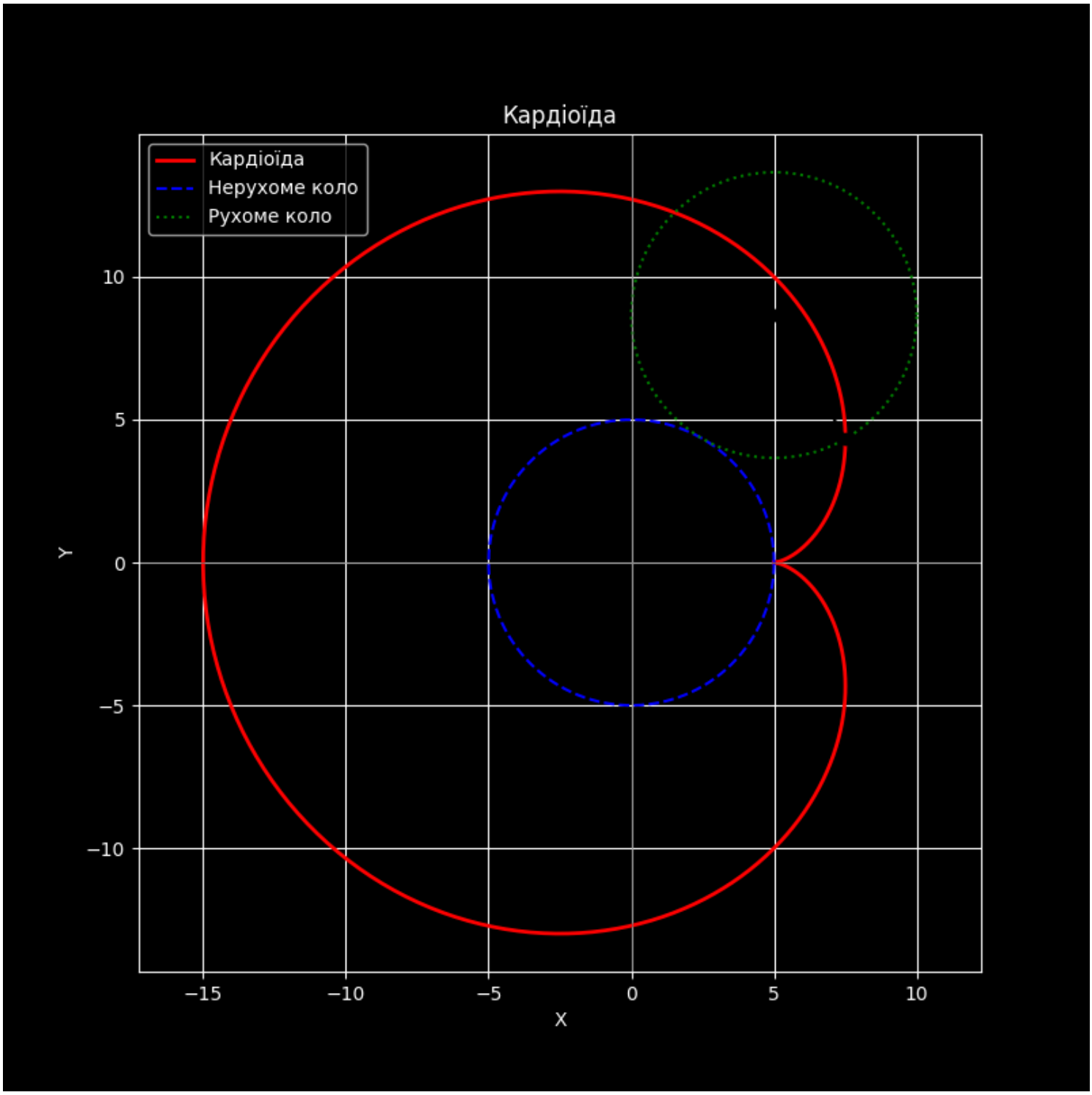
```
X = a * np.sinh(V) * np.cos(U)
Y = b * np.sinh(V) * np.sin(U)
Z1 = c * np.cosh(V)
Z2 = -c * np.cosh(V)

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

ax.plot_wireframe(X, Y, Z1, color='blue', label=r'$\frac{x^2}{a^2} + \frac{y^2}{b^2} - \frac{z^2}{c
ax.plot_wireframe(X, Y, Z2, color='blue')

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Двопорожнинний гіперболоїд')
ax.legend()
plt.savefig("task5_hyperboloid.png")
plt.show()
```

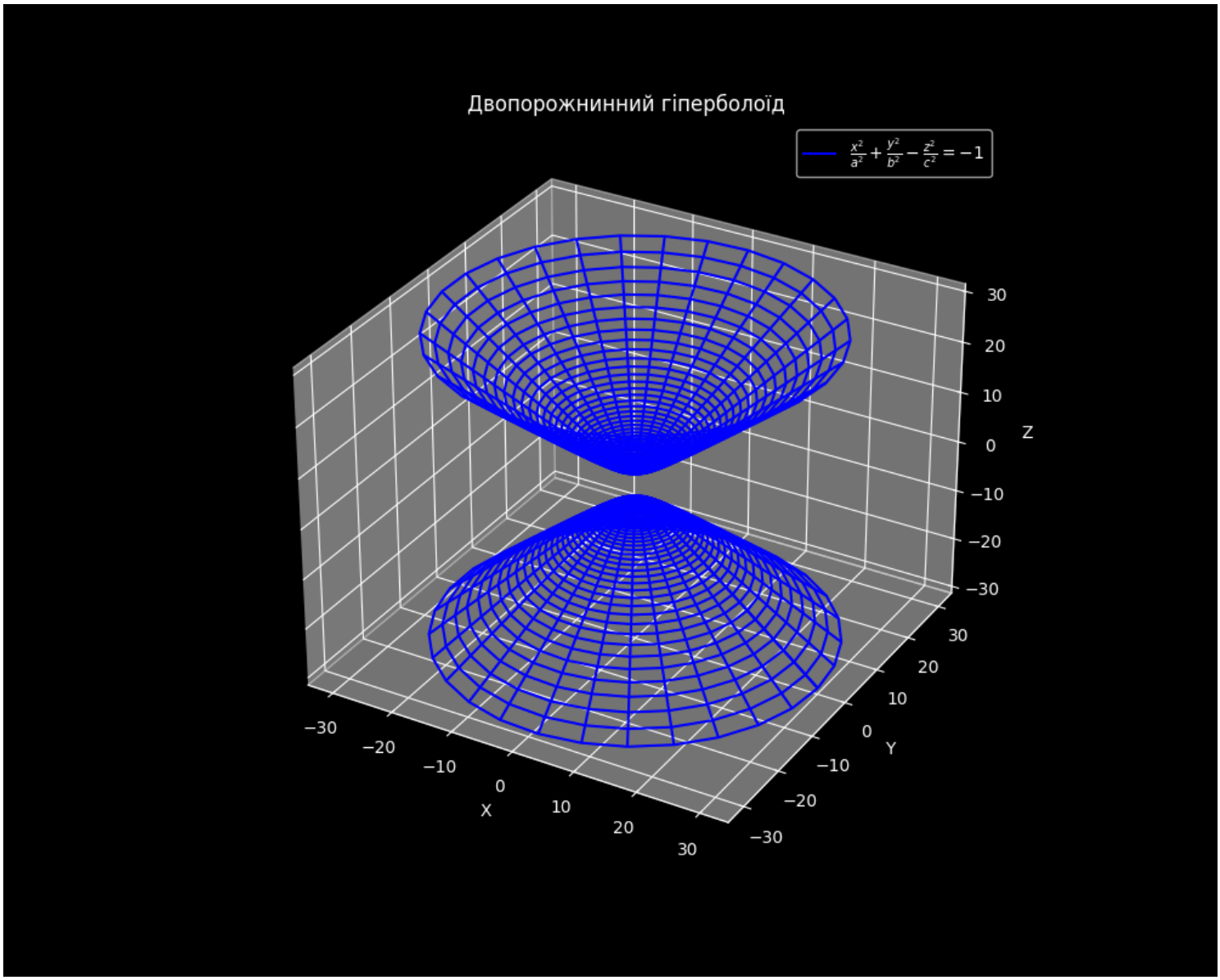

Двопорожнинний гіперболоїд
$\frac{x^2}{a^2}+\frac{y^2}{b^2}-\frac{z^2}{c^2}=-1$
30
20
10
0
−10
−20
−30
X
Y


Розподіл за розширеннями
.py
79.7%
.h
.csv
.cpp
0.7%
1.6%
18.1%
.txt

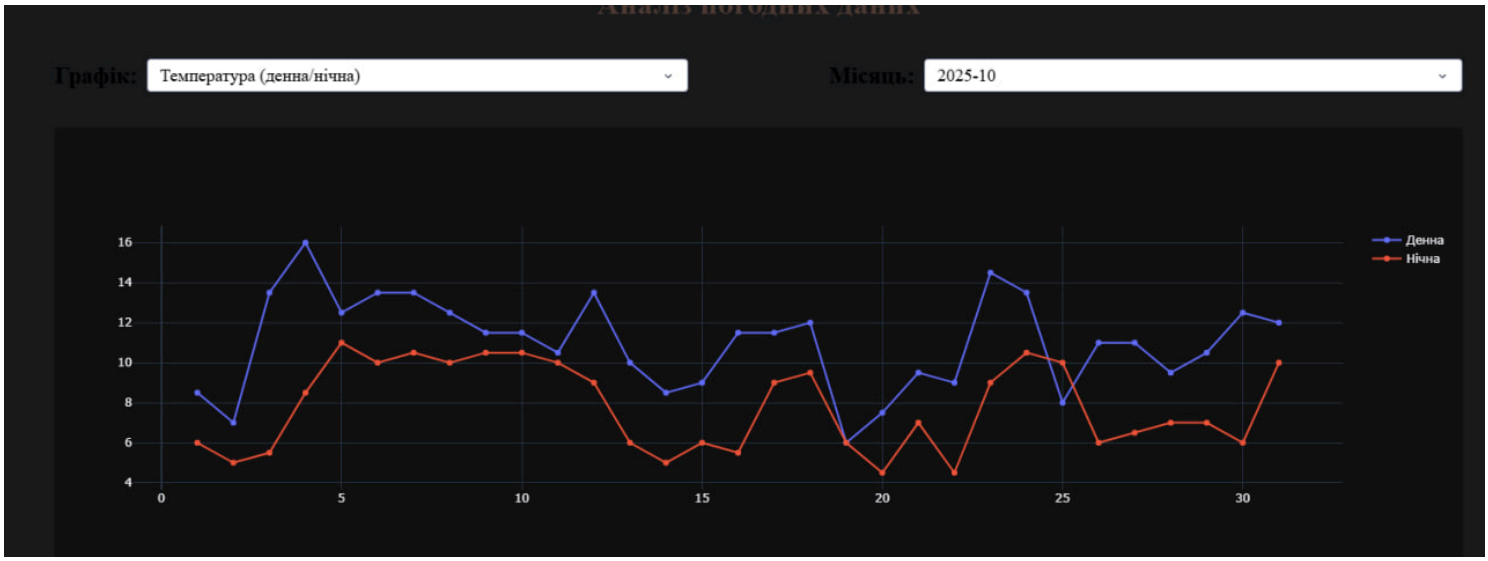

Оцінка часової складності
Емпіричні дані
Апроксимація O(n log n)
Час виконання (t)
Розмір вхідних даних (n)
0
2000
4000
6000
8000
10000
12000
0
200
400
600
800
1000

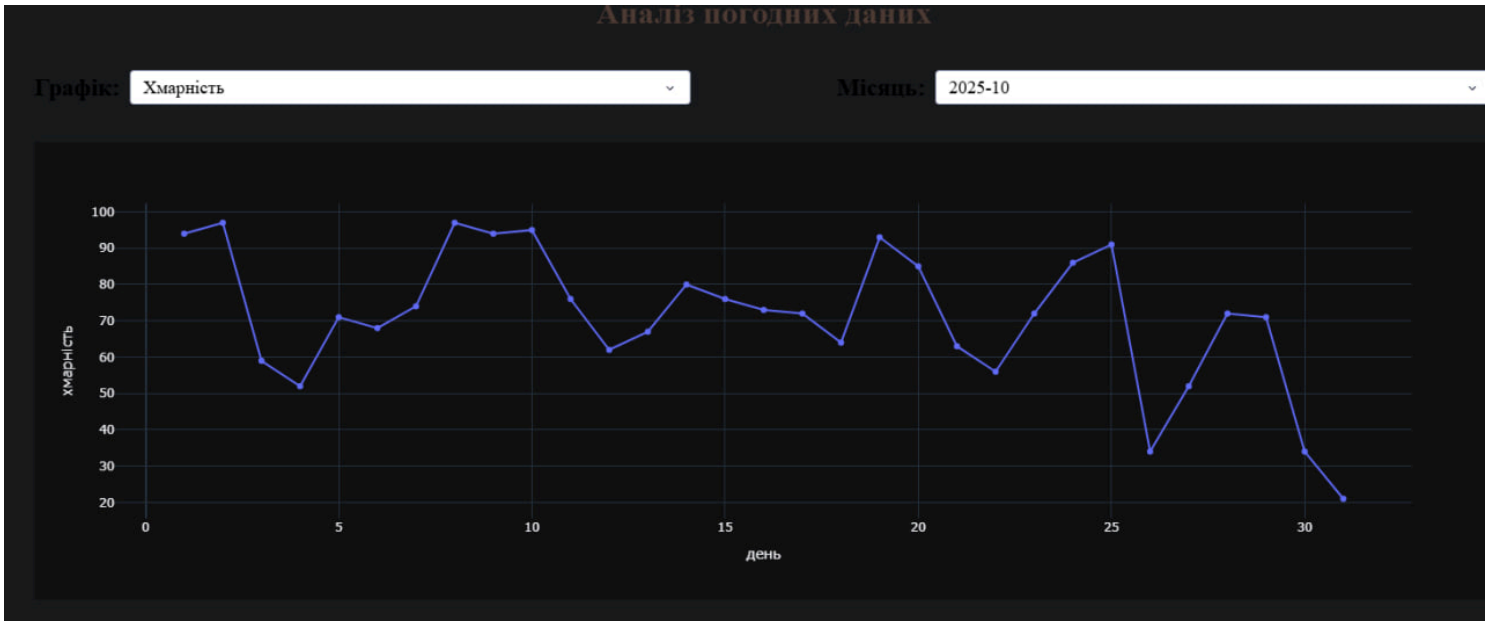

Розподіл значень аргументу по інтервалах
Частота (кількість точок)
Розмір даних (n)
0
2
4
6
8
10
12
14
16
0
200
400
600
800
1000

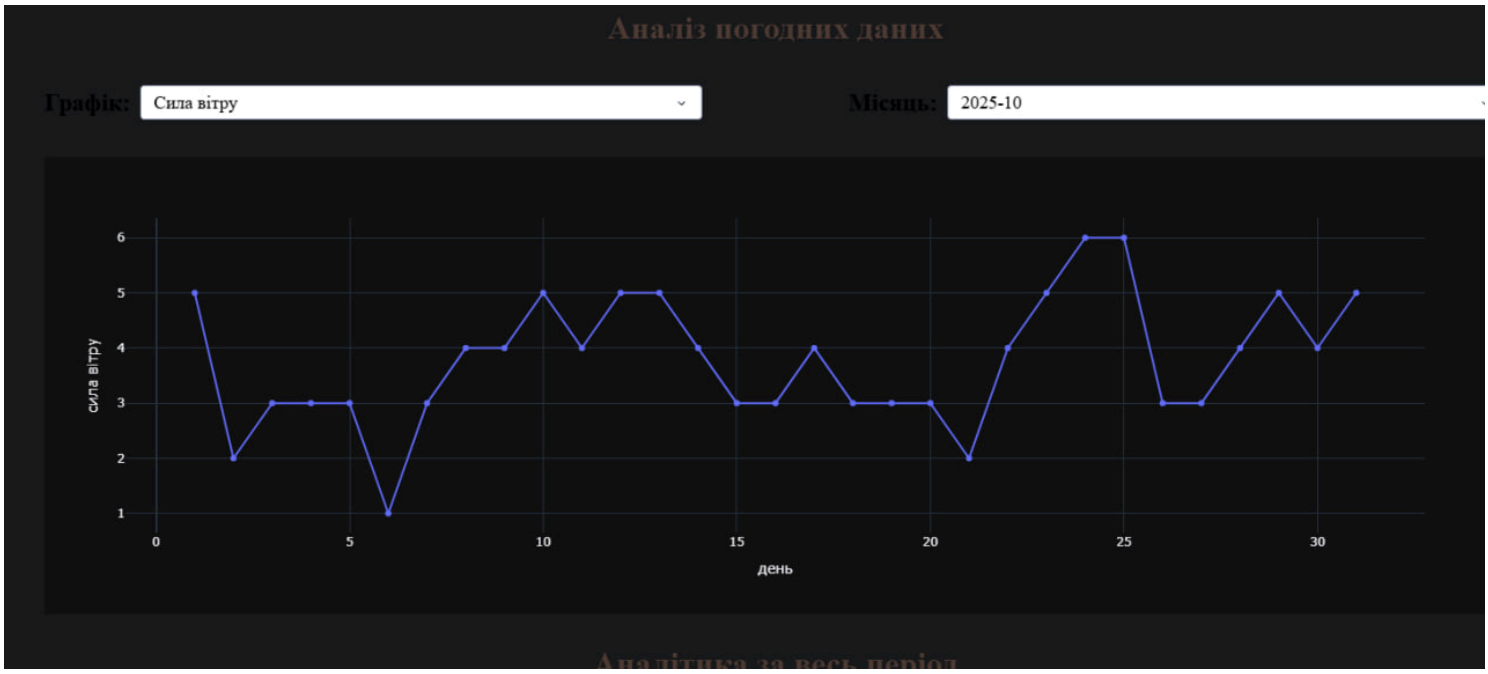

Кардіоїда
Кардіоїда
Нерухоме коло
Рухоме коло
10
5
0
−5
−10
Y
−15
−10
−5
0
5
10
X

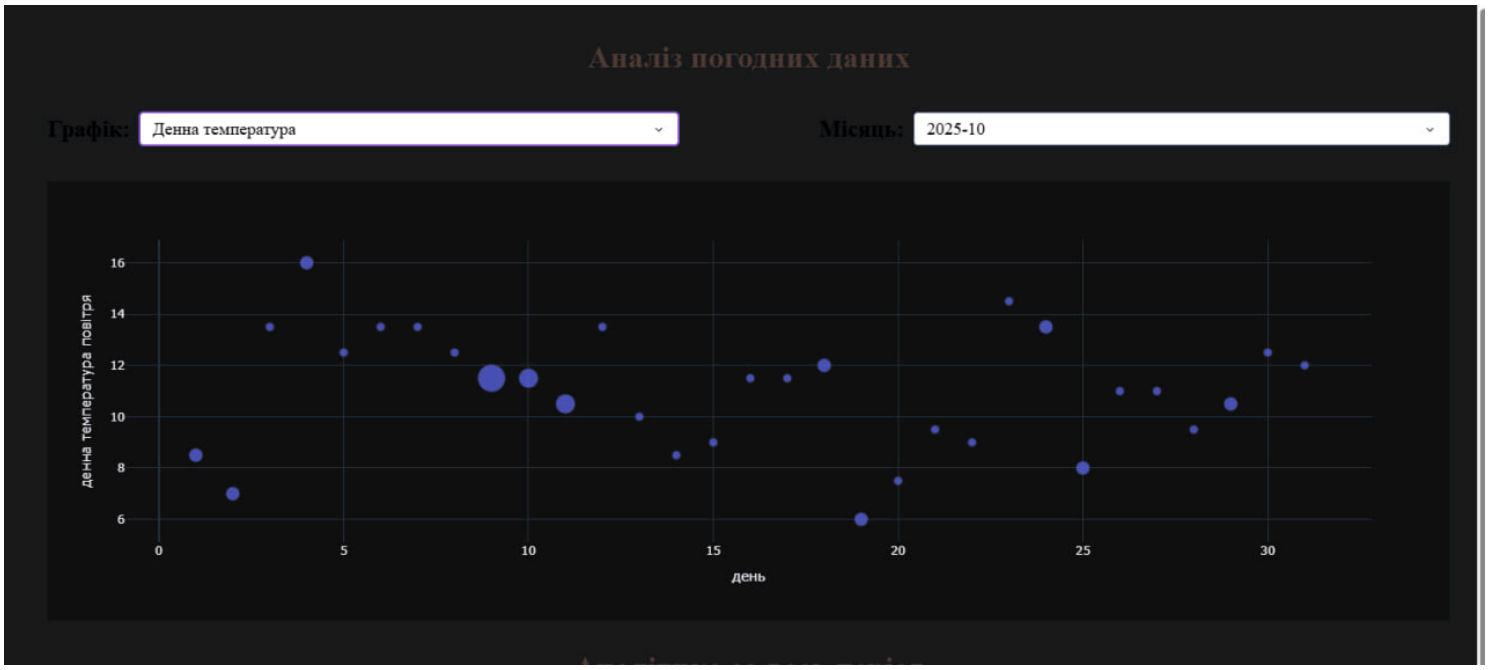

Двопорожнинний гіперболоїд
$\frac{x^2}{a^2} + \frac{y^2}{b^2} - \frac{z^2}{c^2} = -1$
X
Y
Z
−30
−20
−10
0
10
20
30

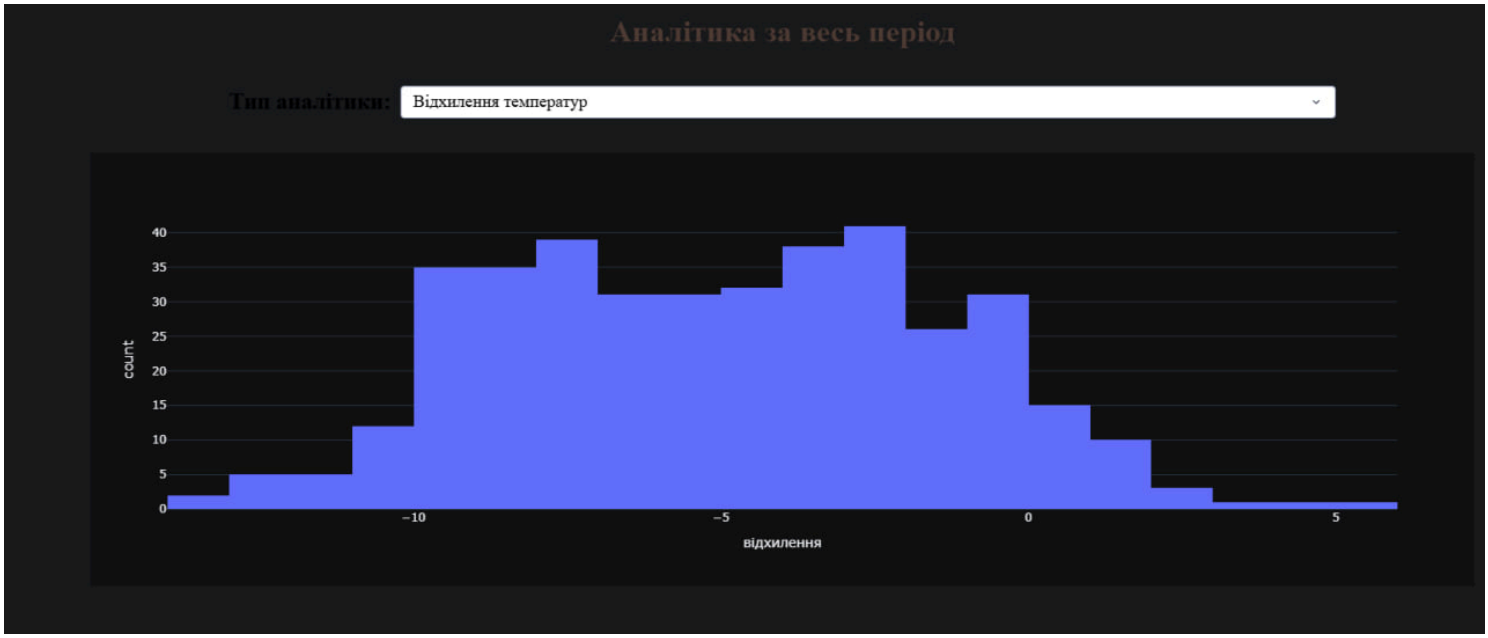
Лабораторна робота № 5
без варіантів


Графік:
Температура (денна/нічна)
Місяць:
2025-10
16
14
12
10
8
6
4
0
5
10
15
20
25
30
Денна
Нічна


Аналіз погодних даних
Графік:
Хмарність
Місяць:
2025-10
100
90
80
70
60
50
40
30
20
хмарність
0
5
10
15
20
25
30
день


Аналіз погодних даних
Графік:
Сила вітру
Місяць:
2025-10
сила вітру
6
5
4
3
2
1
0
5
10
15
20
25
30
день
Аналітика за весь період

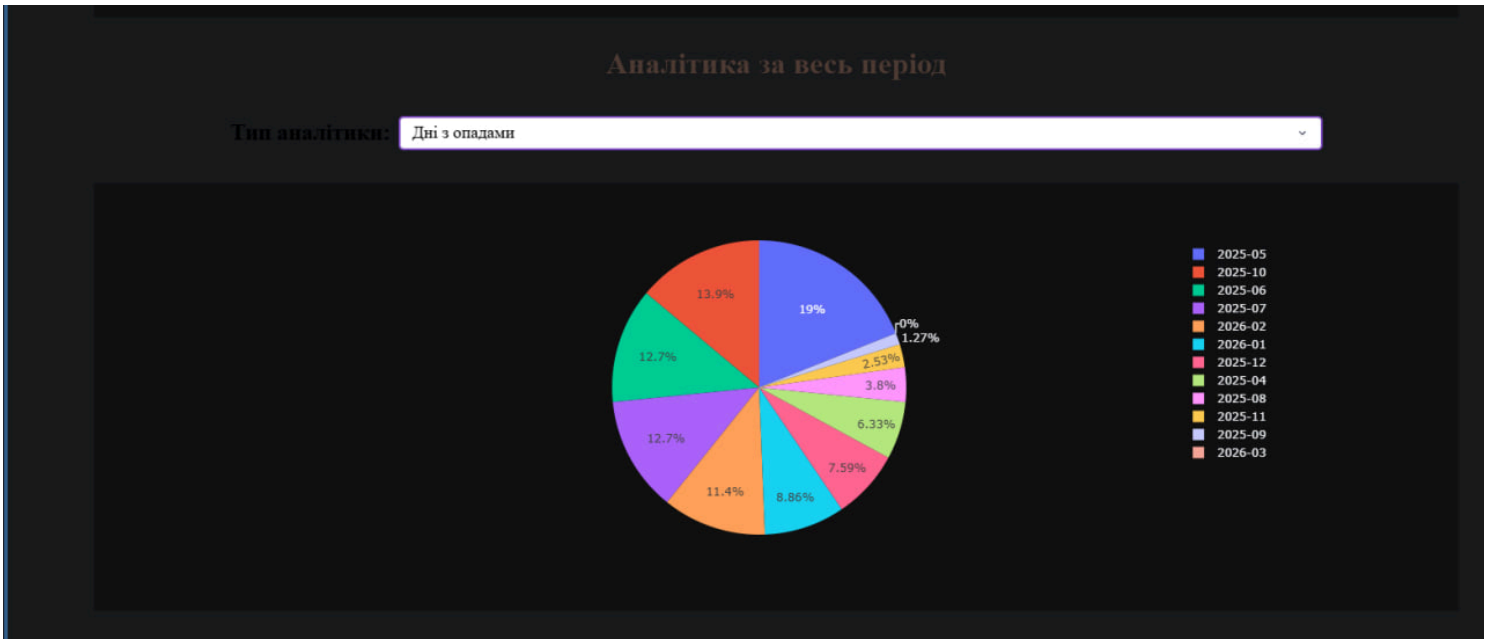

Аналіз погодних даних
Графік:
Денна температура
Місяць:
2025-10
денна температура повітря
16
14
12
10
8
6
0
5
10
15
20
25
30
день


Аналітика за весь період
Тип аналітики:
Відхилення температур
count
40
35
30
25
20
15
10
5
0
−10
−5
0
5
відхилення

Аналітика за весь період
Тип аналітики:
Типи днів
30
25
20
15
10
5
0
count
May 2025
Jul 2025
Sep 2025
Nov 2025
Jan 2026
Mar 2026
період
Хмарність_категорія
сонячний
мінлива хмарність
хмарний

Аналітика за весь період

Тип аналітики: Розподіл хмарності


Аналітика за весь період
Тип аналітики:
Дні з опадами
19%
0%
1.27%
2.53%
3.8%
6.33%
7.59%
8.86%
11.4%
12.7%
12.7%
13.9%
2025-05
2025-10
2025-06
2025-07
2026-02
2026-01
2025-12
2025-04
2025-08
2025-11
2025-09
2026-03

```
import pandas as pd
from dash import Dash, dcc, html, Input, Output
import plotly.express as px
import plotly.graph_objects as go

# ---

def process_data(file_path):
    df = pd.read_csv(file_path, encoding="UTF-8")

    df['хмарність'] = df['хмарність'].str.replace('%', '').astype(int)

    df['опади'] = df['опади'].replace('-', None).fillna('0 м.м.')
    df['опади_num'] = df['опади'].str.replace(' м.м.', '').astype(float)

    df['денна температура повітря'] = df['денна температура повітря'].str.replace('°C', '').astype(float)
    df['нічна температура повітря'] = df['нічна температура повітря'].str.replace('°C', '').astype(float)
    df['сила вітру'] = df['сила вітру'].str.replace(' м/с', '').astype(float)

    df['відхилення'] = df['нічна температура повітря'] - df['денна температура повітря']

    df['Хмарність_категорія'] = pd.cut(df['хмарність'], bins=[-1, 34, 70, 100], labels=['сонячний', 'мінлива хмарність', 'хмар
ний'])

    df['Є_опади'] = df['опади_num'] > 0
    df['Розмір_бульбашки'] = df['опади_num'] * 5
    df.loc[df['опади_num'] == 0, 'Розмір_бульбашки'] = 2

    return df

# ---

df = process_data('weather2026.csv')

# ---

app = Dash(__name__)
app.config.suppress_callback_exceptions = True

graph_menu = dcc.Dropdown(
    id='graph-dropdown',
    options=[
        {'label': 'Температура (денна/нічна)', 'value': 'temp'},
        {'label': 'Хмарність', 'value': 'clouds'},
        {'label': 'Сила вітру', 'value': 'wind'},
        {'label': 'Денна температура', 'value': 'scatter'}
    ],
    value='temp',
    clearable=False,
    style={'width': '100%', 'fontSize': '18px'}
)

period_menu = dcc.Dropdown(
    id='period-dropdown',
    options=[{'label': str(p), 'value': str(p)} for p in df['період'].unique()],
    value=str(df['період'].unique()[0]),
    clearable=False,
    style={'width': '100%', 'fontSize': '18px'}
)

analytics_menu = dcc.Dropdown(
    id='analytics-dropdown',
    options=[
        {'label': 'Відхилення температур', 'value': 'hist'},
        {'label': 'Типи днів', 'value': 'bar'},
        {'label': 'Розподіл хмарності', 'value': 'sunburst'},
        {'label': 'Дні з опадами', 'value': 'pie'}
    ],
    value='hist',
    clearable=False,
    style={'width': '100%', 'fontSize': '18px'}
)

app.layout = html.Div([
```

```
    html.H1('Аналіз погодних даних', style={'textAlign': 'center', 'color': '#503D36'}),

    html.Div([
        html.Div([
            html.H2('Графік:', style={'marginRight': '10px'}),
            html.Div(graph_menu, style={'flex': '1'})
        ], style={'display': 'flex', 'alignItems': 'center', 'width': '45%'}),

        html.Div([
            html.H2('Місяць:', style={'marginRight': '10px'}),
            html.Div(period_menu, style={'flex': '1'})
        ], style={'display': 'flex', 'alignItems': 'center', 'width': '45%'}),
    ], style={'display': 'flex', 'justifyContent': 'space-between', 'marginBottom': '20px'}),

    dcc.Graph(id='main-graph'),

    html.H1('Аналітика за весь період', style={'textAlign': 'center', 'color': '#503D36', 'marginTop': '30px'}),

    html.Div([
        html.H2('Тип аналітики:', style={'marginRight': '10px'}),
        html.Div(analytics_menu, style={'flex': '1'})
    ], style={'display': 'flex', 'alignItems': 'center', 'width': '80%', 'margin': '0 auto', 'marginBottom': '20px'}),

    dcc.Graph(id='analytics-graph')

], style={'padding': '20px'})

@app.callback(
    Output('main-graph', 'figure'),
    [Input('period-dropdown', 'value'), Input('graph-dropdown', 'value')]
)
def update_main_graph(period, graph_type):
    if not period or not graph_type:
        return go.Figure()

    dff = df[df['період'] == period]

    if dff.empty:
        return go.Figure()

    if graph_type == 'temp':
        fig = go.Figure()
        fig.add_trace(go.Scatter(x=dff['день'], y=dff['денна температура повітря'], name='Денна', mode='lines+markers'))
        fig.add_trace(go.Scatter(x=dff['день'], y=dff['нічна температура повітря'], name='Нічна', mode='lines+markers'))
        return fig
    elif graph_type == 'clouds':
        return px.line(dff, x='день', y='хмарність', markers=True)
    elif graph_type == 'wind':
        return px.line(dff, x='день', y='сила вітру', markers=True)
    elif graph_type == 'scatter':
        return px.scatter(dff, x='день', y='денна температура повітря', size='Розмір_бульбашки')

@app.callback(
    Output('analytics-graph', 'figure'),
    [Input('analytics-dropdown', 'value')]
)
def update_analytics_graph(analytics_type):
    if not analytics_type:
        return go.Figure()

    if analytics_type == 'hist':
        return px.histogram(df, x='відхилення')
    elif analytics_type == 'bar':
        grouped = df.groupby(['період', 'Хмарність_категорія'], observed=False).size().reset_index(name='count')
        return px.bar(grouped, x='період', y='count', color='Хмарність_категорія')
    elif analytics_type == 'sunburst':
        grouped = df.groupby(['період', 'Хмарність_категорія'], observed=False).size().reset_index(name='count')
        return px.sunburst(grouped, path=['період', 'Хмарність_категорія'], values='count')
    elif analytics_type == 'pie':
        grouped = df.groupby('період', observed=False)['Є_опади'].sum().reset_index(name='days_with_rain')
        return px.pie(grouped, values='days_with_rain', names='період')

if __name__ == '__main__':
    app.run(jupyter_mode='inline', jupyter_height=1000)
```

Лабораторна робота № 6
без варіантів

Лабораторна робота № 6

Протягом семестру виконувались лабораторні роботи 1-5. Кожна з них мала загальну умову та кілька частин.
Програмно створити загальний звіт за всіма лабораторними роботами.
Має бути:
1) Титульна сторінка, де вказано, що це підсумковий звіт, виконавця (ПІБ), групу, викладача лабораторних занять, рік виконання;
2) на 2й сторінці має бути Зміст.
В покажчик мають увійти всі 6 лабораторних (перший рівень), для кожної (другий рівень) умова і частини та код (ЛР5 та 6). Мають бути проставлені сторінки документу (у змісті!!!).
3) Далі мають розташовуватись звіти з лабораторних робіт.
Для ЛР №1-4 використовуються звіти, що здавались (загальну умову лр3-6 можна вставити як пдф-ку, отриману за docx-файлом зовнішніми засобами (наприклад, роздрукувати в пдф-файл). Для ЛР5 вставляються скріншоти, що здавались, та код. Для ЛР6 наводиться тільки умова та код.
Кожній ЛР у файлі має передувати «початковий» аркуш вигляду

Лабораторна робота № ...
варіант № ...

(крупним шрифтом, по центру; якщо варіантів не було, то зазначити «без варіантів»)

4) У файлі мають бути розставлені закладки на лабораторні роботи та закладки вкладеного рівня на частини лабораторних робіт.

Можна використовувати готове: звіти, скріншоти з ЛР5, конвертовані файли загальних умов. Все інше (зміст, початкові аркуші, код, закладки) має формуватись у результуючому файлі програмно.

Якщо відповідна частина якоїсь ЛР чи вся ЛР не була зроблена. Тоді згенерувати замість звіту відповідну заглушку (або у звіт вставити заглушку для певної частини), замість відсутнього коду можна взяти довільний код мовою python (до 60 рядків), щоб використати ці заглушки в ЛР6.

Максимум = 15 балів.
Звіт читабельний, його згенеровано програмно, з-під python, з використанням бібліотек роботи з пдф-файлами; наявні всі лабораторні та всі їх частини, для кожної ЛР у звіті сформовано «початковий» аркуш --- 9 балів
додатково до попереднього:
Наявний зміст (з усіма елементами 1-го та 2-го рівня) з правильними сторінками --- +3 бали
Наявні закладки (усі та правильні), але тільки однорівневі --- +1 бал
Наявні закладки (усі та правильні) (дворівневі) --- +3 бали
Неохайно відформатовано титульний аркуш або «початкові» аркуші або рисунки вибиваються за межі аркушів --- -3 бали

Hint: ніде не сказано, що все треба зробити однією бібліотекою. Можна використати кілька.

```
import os
import json
import fitz
import requests
from reportlab.lib.pagesizes import A4
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.enums import TA_CENTER, TA_LEFT
from reportlab.lib.units import cm


def get_font():
    f_p = 'DejaVuSans.ttf'
    if not os.path.exists(f_p):
        u = "https://raw.githubusercontent.com/prawnpdf/prawn/master/data/fonts/DejaVuSans.ttf"
        try:
            r = requests.get(u, timeout=10)
            with open(f_p, 'wb') as f:
                f.write(r.content)
        except:
            return 'Helvetica'
    pdfmetrics.registerFont(TTFont('DejaVuSans', f_p))
    return 'DejaVuSans'


def get_full_code(p):
    if not os.path.exists(p): return ["# Файл коду не знайдено"]
    if p.endswith('.ipynb'):
        try:
            with open(p, 'r', encoding='utf-8') as f:
                d = json.load(f)
            lines = []
            for c in d.get('cells', []):
                if c.get('cell_type') == 'code':
                    src = c.get('source', [])
                    if src:
                        lines.extend(src)
                        lines.append("\n\n# --- \n\n")
            return lines if lines else ["# Notebook порожній"]
        except:
            return ["# Помилка читання .ipynb"]
    with open(p, 'r', encoding='utf-8') as f:
        return f.readlines()


def make_code_pdf(fn, lines, f_n):
    doc = SimpleDocTemplate(fn, pagesize=A4, leftMargin=1.5 * cm, rightMargin=1 * cm, topMargin=1.5 * cm,
                            bottomMargin=1.5 * cm)
    st = ParagraphStyle(name='C', fontName=f_n, fontSize=8, leading=10)
    els = []
    txt = "".join(lines).replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
    for l in txt.split('\n'):
        c = l.replace(" ", " ")
        els.append(Paragraph(c if c else " ", st))
    doc.build(els)


def make_title(fn, f_n):
    doc = SimpleDocTemplate(fn, pagesize=A4, leftMargin=0, rightMargin=0)
    st_t = ParagraphStyle(name='T', fontName=f_n, fontSize=24, alignment=TA_CENTER, leading=30)
    st_n = ParagraphStyle(name='N', fontName=f_n, fontSize=14, alignment=TA_CENTER, leading=20)
    els = [
        Spacer(1, 9 * cm),
        Paragraph("Підсумковий звіт", st_t),
        Paragraph("з лабораторних робіт 1-6", st_t),
        Spacer(1, 4 * cm),
        Paragraph("Виконавець: Базилевич Олексій", st_n),
        Paragraph("Група: К-27", st_n),
        Paragraph("Викладач: Андрій Ляшко", st_n),
        Spacer(1, 3 * cm),
        Paragraph("2026", st_n)
    ]
```

```
    doc.build(els)


def make_sep(fn, n, v, f_n):
    doc = SimpleDocTemplate(fn, pagesize=A4, leftMargin=0, rightMargin=0)
    st = ParagraphStyle(name='S', fontName=f_n, fontSize=28, alignment=TA_CENTER, leading=35)
    vt = f"варіант № {v}" if v else "без варіантів"
    els = [
        Spacer(1, 11 * cm),
        Paragraph(f"Лабораторна робота № {n}", st),
        Paragraph(vt, st)
    ]
    doc.build(els)


def fit_img(ip, op):
    idoc = fitz.open(ip)
    pb = idoc.convert_to_pdf()
    idoc.close()
    timg = fitz.open("pdf", pb)
    fdoc = fitz.open()
    a4w, a4h = fitz.paper_size("A4")
    for p in timg:
        np = fdoc.new_page(width=a4w, height=a4h)
        r = p.rect
        s = min((a4w - 40) / r.width, (a4h - 40) / r.height)
        tw, th = r.width * s, r.height * s
        tr = fitz.Rect((a4w - tw) / 2, (a4h - th) / 2, (a4w + tw) / 2, (a4h + th) / 2)
        np.show_pdf_page(tr, timg, p.number)
    fdoc.save(op)
    fdoc.close()
    timg.close()


def main():
    fn = get_font()
    res = fitz.open()
    tmp = []

    t_f = "t_title.pdf"
    make_title(t_f, fn)
    tmp.append(t_f)
    tdoc = fitz.open(t_f)
    res.insert_pdf(tdoc)
    tdoc.close()

    cfg = [
        {"n": 1, "v": "", "p": "Lab1", "c": "Lab1/MainTaskForLab1/LAB1 робочий зошит.pdf", "code": [], "media": True},
        {"n": 2, "v": "99", "p": "Lab2", "c": "Lab2/MainTaskForLab2/LAB2/LSM.pdf", "code": [], "media": True},
        {"n": 3, "v": "4", "p": "Lab3", "c": "Lab3/MainTaskForLab3/Лаб 3.docx.pdf", "code": [], "media": True},
        {"n": 4, "v": "99 (20; 4)", "p": "Lab4", "c": "Lab4/MainTaskForLab4/Lab4.docx.pdf", "code": [], "media": True},
        {"n": 5, "v": "", "p": "Lab5", "c": "Lab5/MainTaskForLab5/LAB5/lab5.pdf", "code": ["Lab5/Lab5_1.ipynb"],
         "media": True, "no_report": True},
        {"n": 6, "v": "", "p": "Lab6", "c": "Lab6/MainTaskForLab6/lab6.pdf", "code": ["lab6.py"], "media": False}
    ]

    toc, pg = [], 3
    for l in cfg:
        sf = f"t_s_{l['n']}.pdf"
        make_sep(sf, l['n'], l['v'], fn)
        tmp.append(sf)
        sdoc = fitz.open(sf)
        res.insert_pdf(sdoc)
        toc.append([1, f"Лабораторна робота № {l['n']}", pg])
        pg += len(sdoc)
        sdoc.close()

        cp = l["c"]
        if os.path.exists(cp):
            toc.append([2, "Умова", pg])
            cdoc = fitz.open(cp)
            res.insert_pdf(cdoc)
            pg += len(cdoc)
            cdoc.close()
```

```
        pd = os.path.join(l['p'], "Pdf")
        if l.get("media") and os.path.exists(pd):
            for f in sorted(os.listdir(pd)):
                fp = os.path.join(pd, f)
                to = f"t_m_{l['n']}_{f}.pdf"
                if f.lower().endswith('.pdf'):
                    if l.get("no_report"): continue
                    toc.append([2, f"Звіт: {f}", pg])
                    md = fitz.open(fp)
                    res.insert_pdf(md)
                    pg += len(md)
                    md.close()
                elif f.lower().endswith(('.png', '.jpg', '.jpeg')):
                    toc.append([2, f"Скріншот: {f}", pg])
                    fit_img(fp, to)
                    tmp.append(to)
                    md = fitz.open(to)
                    res.insert_pdf(md)
                    pg += len(md)
                    md.close()

        for part in l['code']:
            cp = part if os.path.exists(part) else os.path.join(l['p'], part)
            if os.path.exists(cp):
                toc.append([2, f"Код: {os.path.basename(cp)}", pg])
                tf = f"t_c_{l['n']}_{os.path.basename(cp).replace('.', '_')}.pdf"
                tmp.append(tf)
                make_code_pdf(tf, get_full_code(cp), fn)
                cd = fitz.open(tf)
                res.insert_pdf(cd)
                pg += len(cd)
                cd.close()

    st_tt = ParagraphStyle(name='TT', fontName=fn, fontSize=18, alignment=TA_CENTER, spaceAfter=20)
    t_els = [Paragraph("Зміст", st_tt)]
    for lvl, t, p in toc:
        ind = 20 if lvl == 2 else 0
        st_l = ParagraphStyle(name=f'L{lvl}', fontName=fn, fontSize=11, leftIndent=ind)
        t_els.append(Paragraph(f"{t} . . . . . {p}", st_l))

    t_toc = "t_toc.pdf"
    SimpleDocTemplate(t_toc, pagesize=A4).build(t_els)
    tmp.append(t_toc)
    tdoc = fitz.open(t_toc)
    res.insert_pdf(tdoc, start_at=1)
    tdoc.close()

    res.set_toc(toc)
    res.save("Final_Report_Bazylevich_Alex.pdf")
    res.close()

    for f in tmp:
        if os.path.exists(f):
            try:
                os.remove(f)
            except:
                pass
    print("Звіт згенеровано успішно: Final_Report_Bazylevich_Alex.pdf")


if __name__ == "__main__":
    main()
```